



DevOps series

Kubernetes





 Somkiat Puisungnoen

 Somkiat Home





Somkiat Puisungnoen

Update Info 1View Activity Log 10+...

Timeline

About

Friends 3,138

Photos

More ▾

 When did you work at Opendream? X

... 22 Pending Items

 Intro

Software Craftsmanship

 Software Practitioner at สยามชำนาญกิจ พ.ศ. 2556

 Agile Practitioner and Technical at SPRINT3r

 Post |  Photo/Video |  Live Video |  Life Event

 What's on your mind?

 Public ▾

Post

 **Somkiat Puisungnoen**

15 mins · Bangkok · 🌐 ▾

Java and Bigdata

...



Facebook interface for the page **somkiat.cc**. The top navigation bar includes the Facebook logo, the page name, a search bar, and icons for Home, Messages, Notifications (3), Insights, Publishing Tools, Settings, and Help.

The main content area displays a large video of a man in a white Superman t-shirt posing against a white wall. A blue call-to-action button is overlaid on the video, reading "Help people take action on this Page." with a close icon. Below the video, there are buttons for "Liked", "Following", "Share", and a menu icon.

The left sidebar shows the page name **somkiat.cc** and handle **@somkiat.cc**. Below this is a menu with options: Home, Posts, Videos, and Photos.



Software delivery

Design

Develop

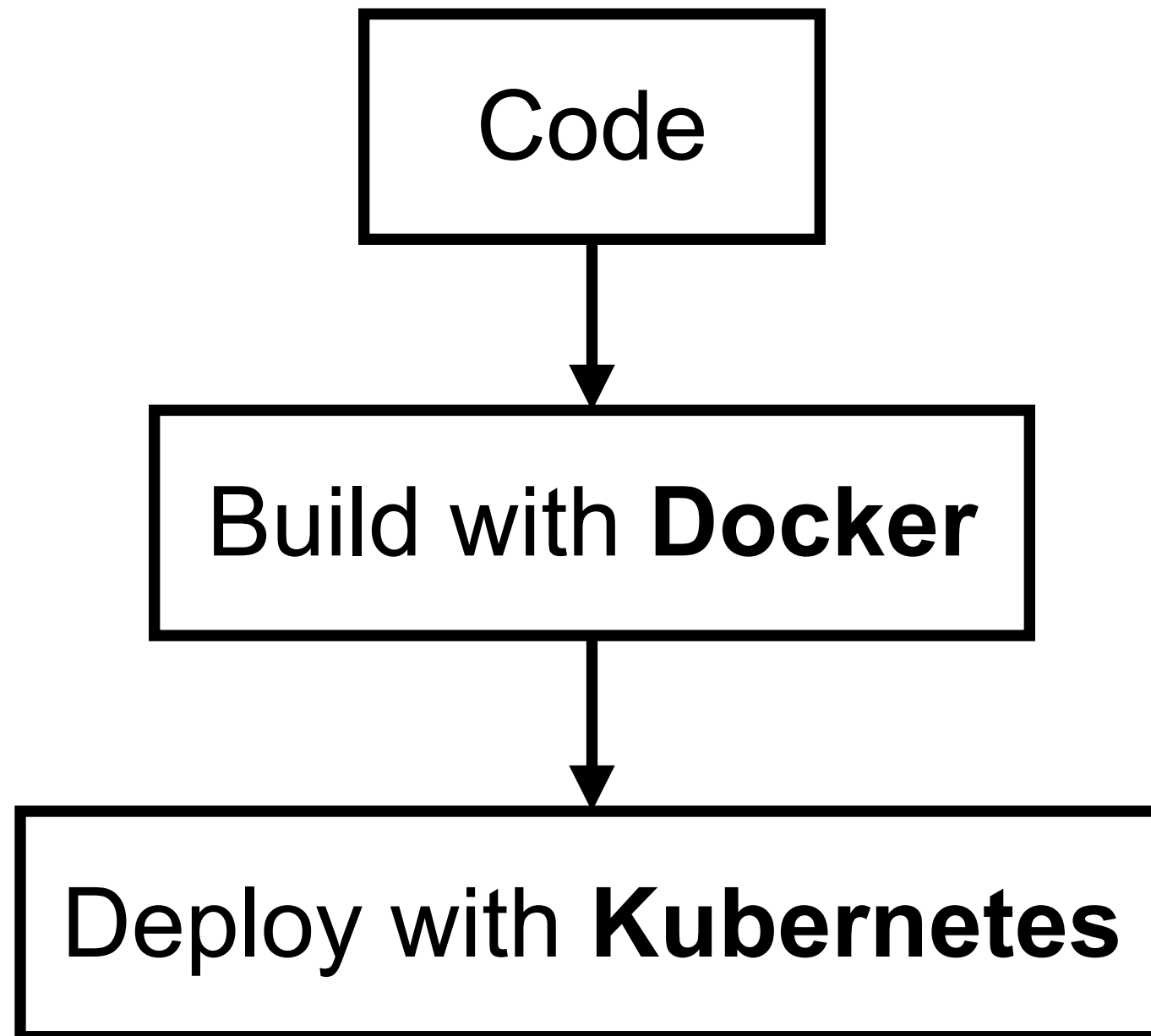
Test

Deploy

Docker and Kubernetes

Monitoring and Observability





Section 1 (Container and Docker)

1. Introduction to containers
2. Container design patterns
3. Manage containers with Docker
4. Docker ecosystem
5. Workshop with Docker



Section 2 (Kubernetes)

1. Introduction to Kubernetes
2. Kubernetes architecture
3. Components in Kubernetes
4. Install and config Kubernetes cluster
5. Networking
6. Stateful application
7. Security and Monitoring cluster



**[https://github.com/up1/
course-kubernetes](https://github.com/up1/course-kubernetes)**



Software Delivery

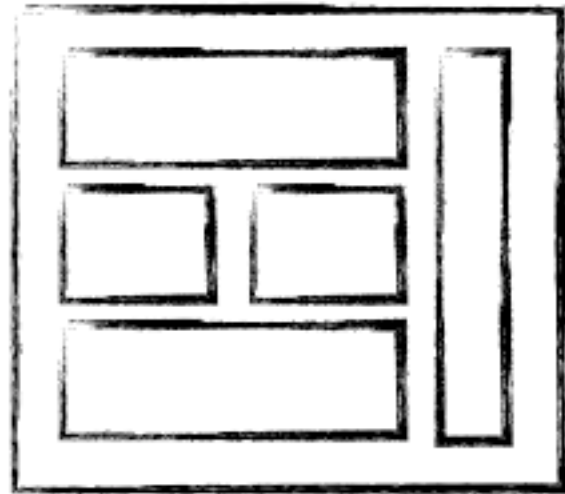


Manage containers



Why docker ?

Software industry has changed !!



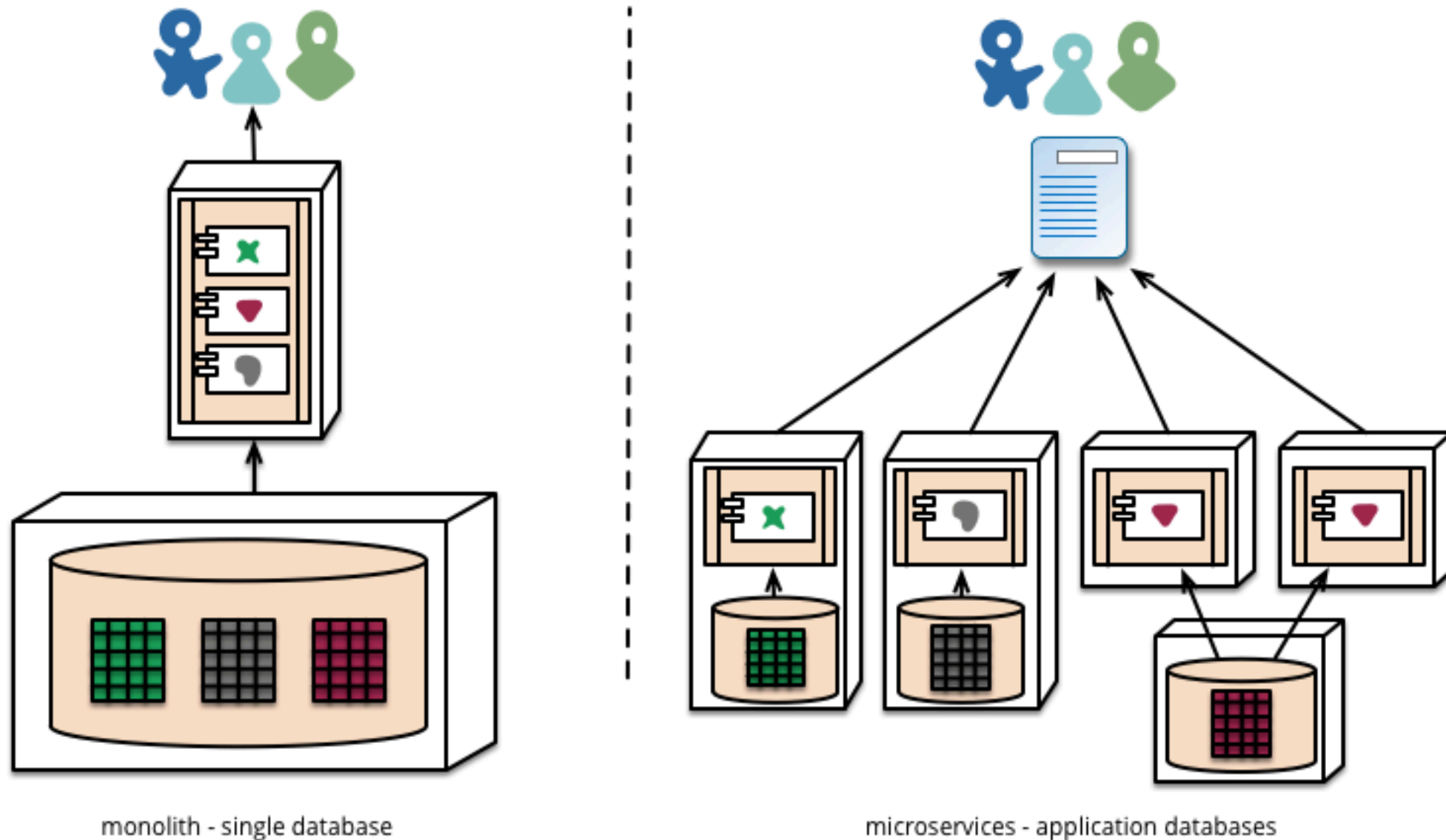
MONOLITHIC/LAYERED



MICRO SERVICES



Microservice architecture

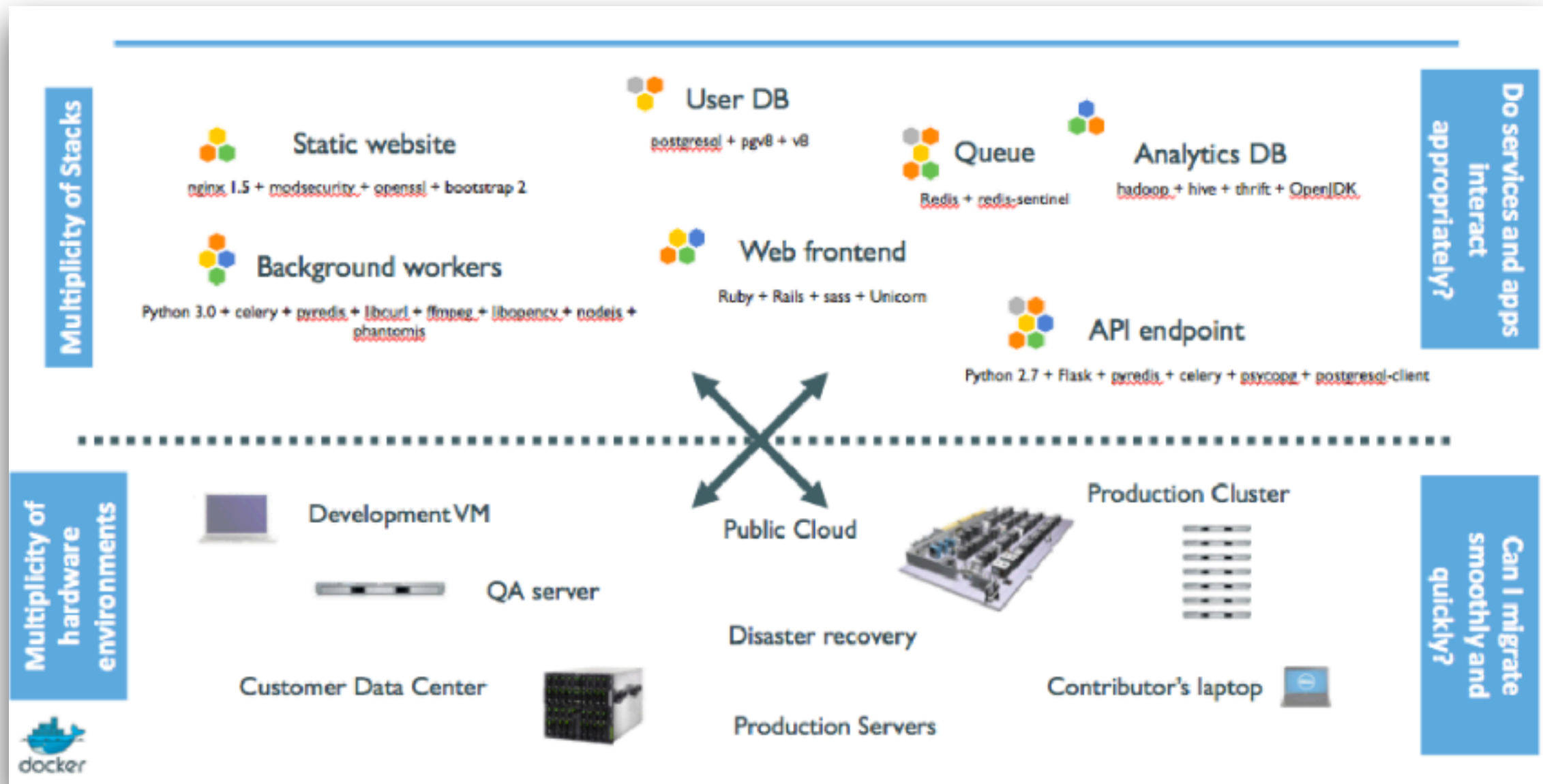


<https://martinfowler.com/articles/microservices.html>



Why docker ?

We have problem !!

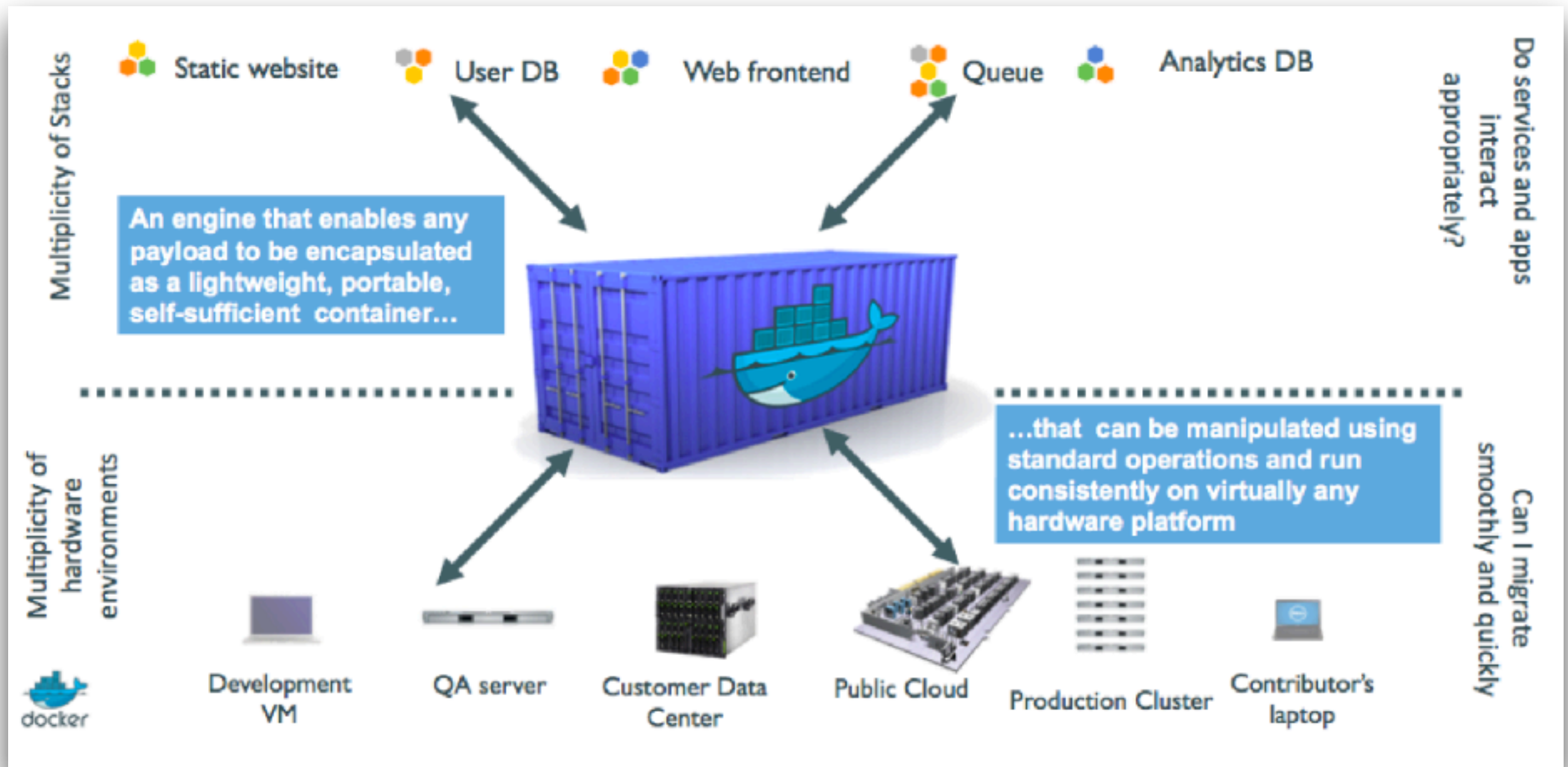


Problem ?

	Static website	?	?	?	?	?	?	?
	Web frontend	?	?	?	?	?	?	?
	Background workers	?	?	?	?	?	?	?
	User DB	?	?	?	?	?	?	?
	Analytics DB	?	?	?	?	?	?	?
	Queue	?	?	?	?	?	?	?
		Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
								



Solution ?

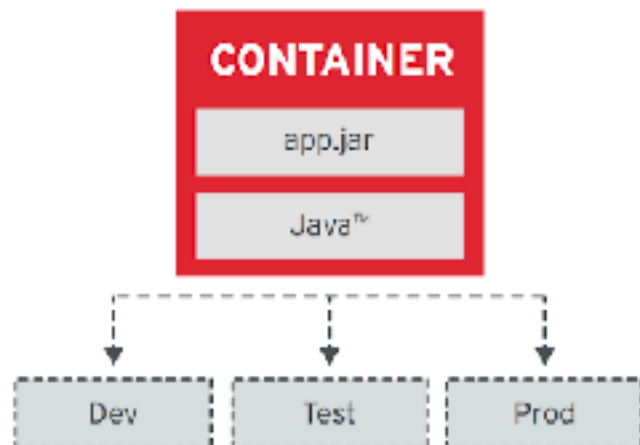


	Static website						
	Web frontend						
	Background workers						
	User DB						
	Analytics DB						
	Queue						
		Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop



Container Design Principles

Image Immutability Principle



High Observability Principle



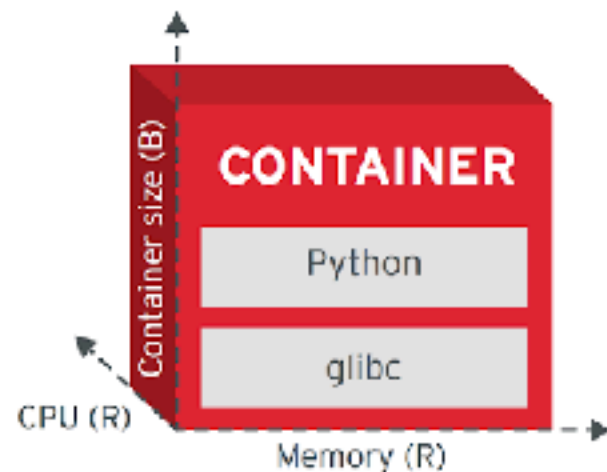
Process Disposability Principle



Lifecycle Conformance Principle



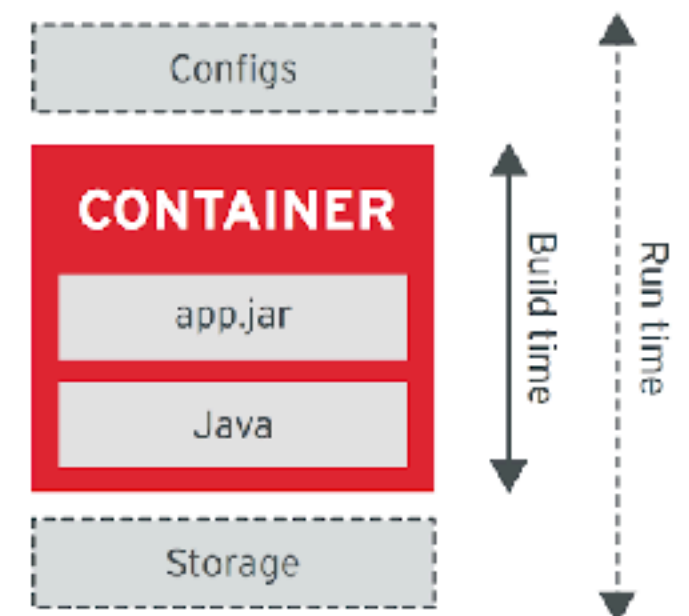
Runtime Confinement Principle



Single Concern Principle



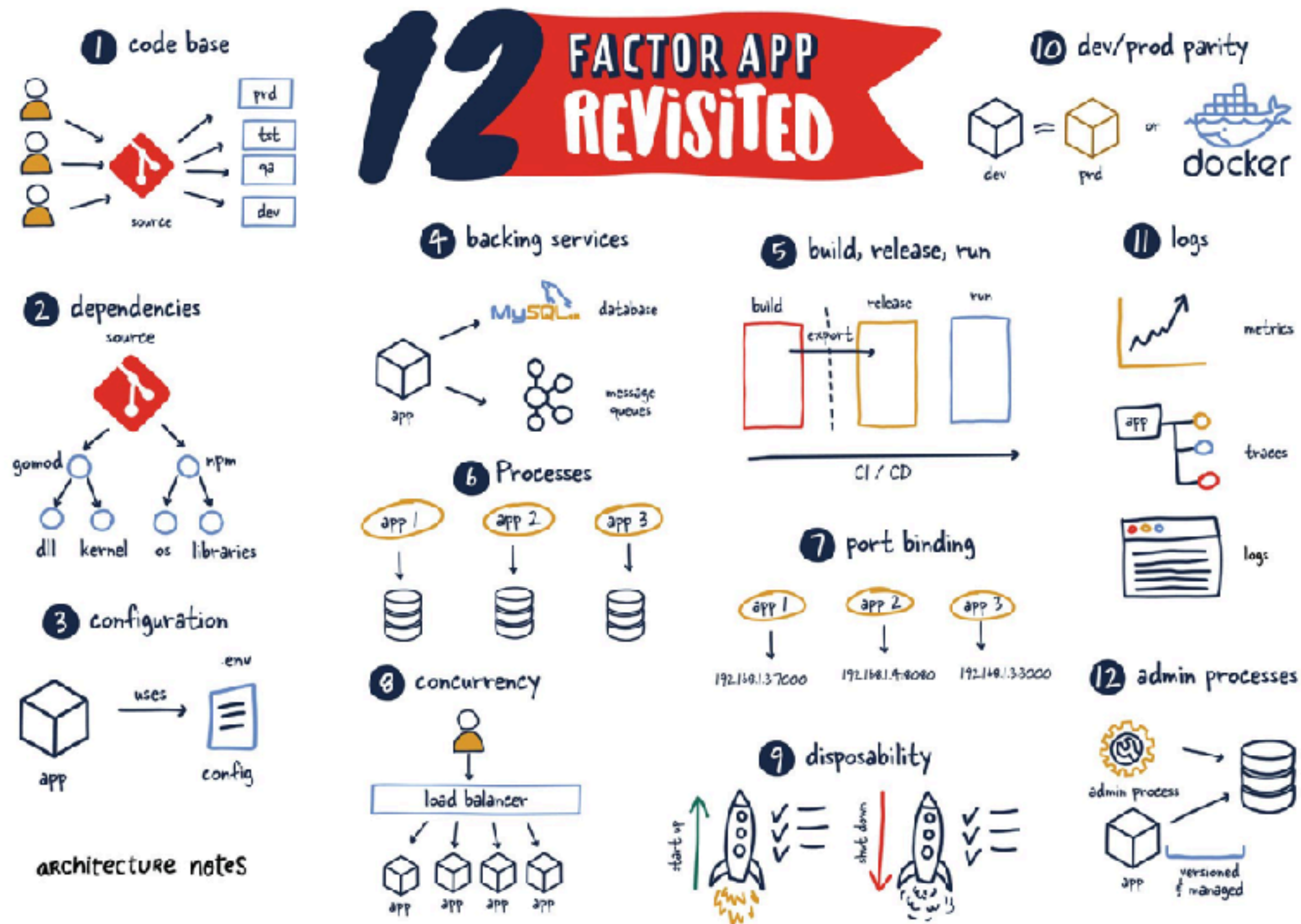
Self-Containment Principle



<https://kubernetes.io/blog/2018/03/principles-of-container-app-design/>



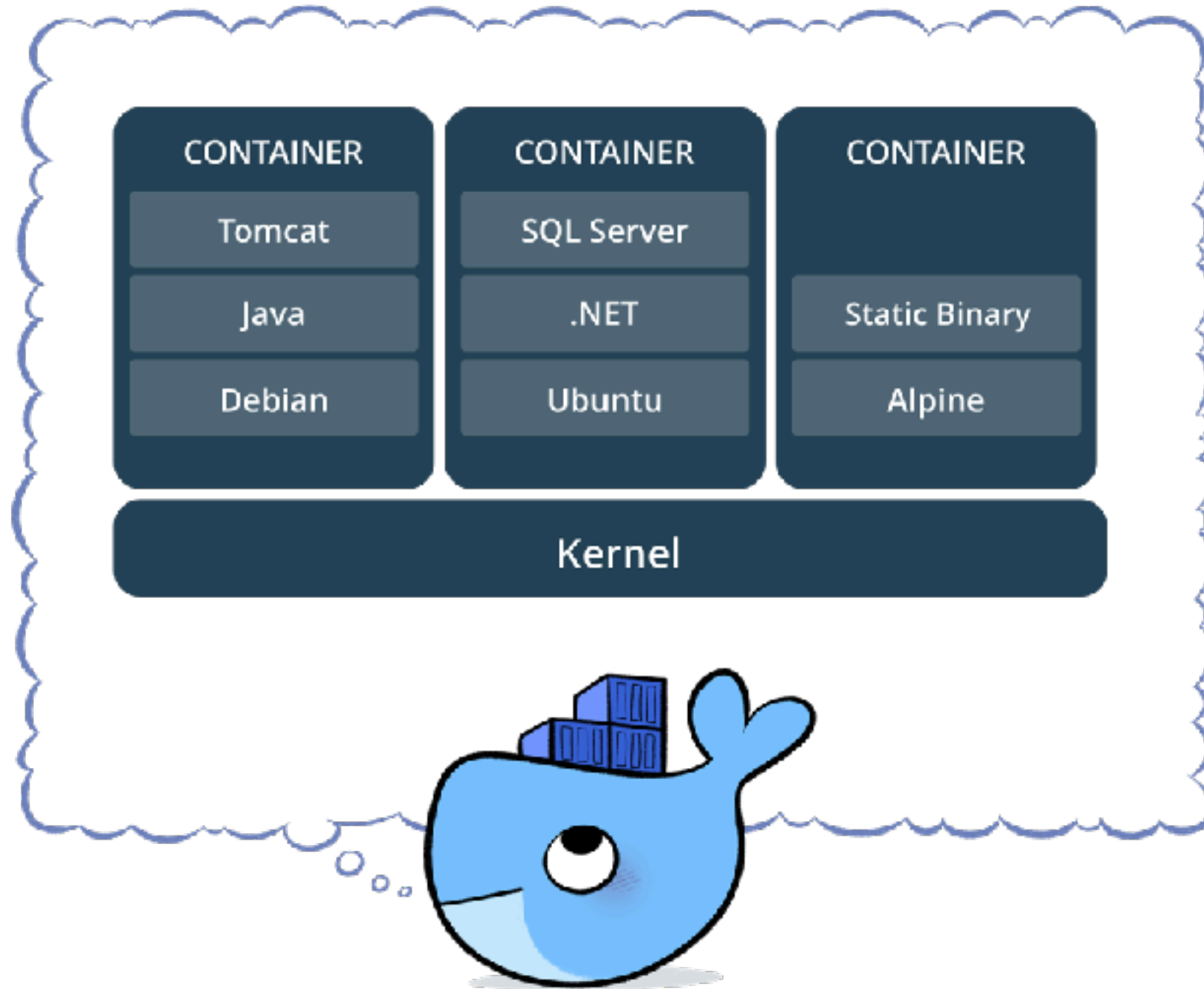
The Twelve-Factor App



<https://12factor.net/>



Container with Docker



Basic of Docker

Image
Container
Dockerfile
Registry
Volume and network



Image vs Container

Image like template/blueprint
Containers are created from image



Docker image

Collection of files and some meta data

Made of **layers**

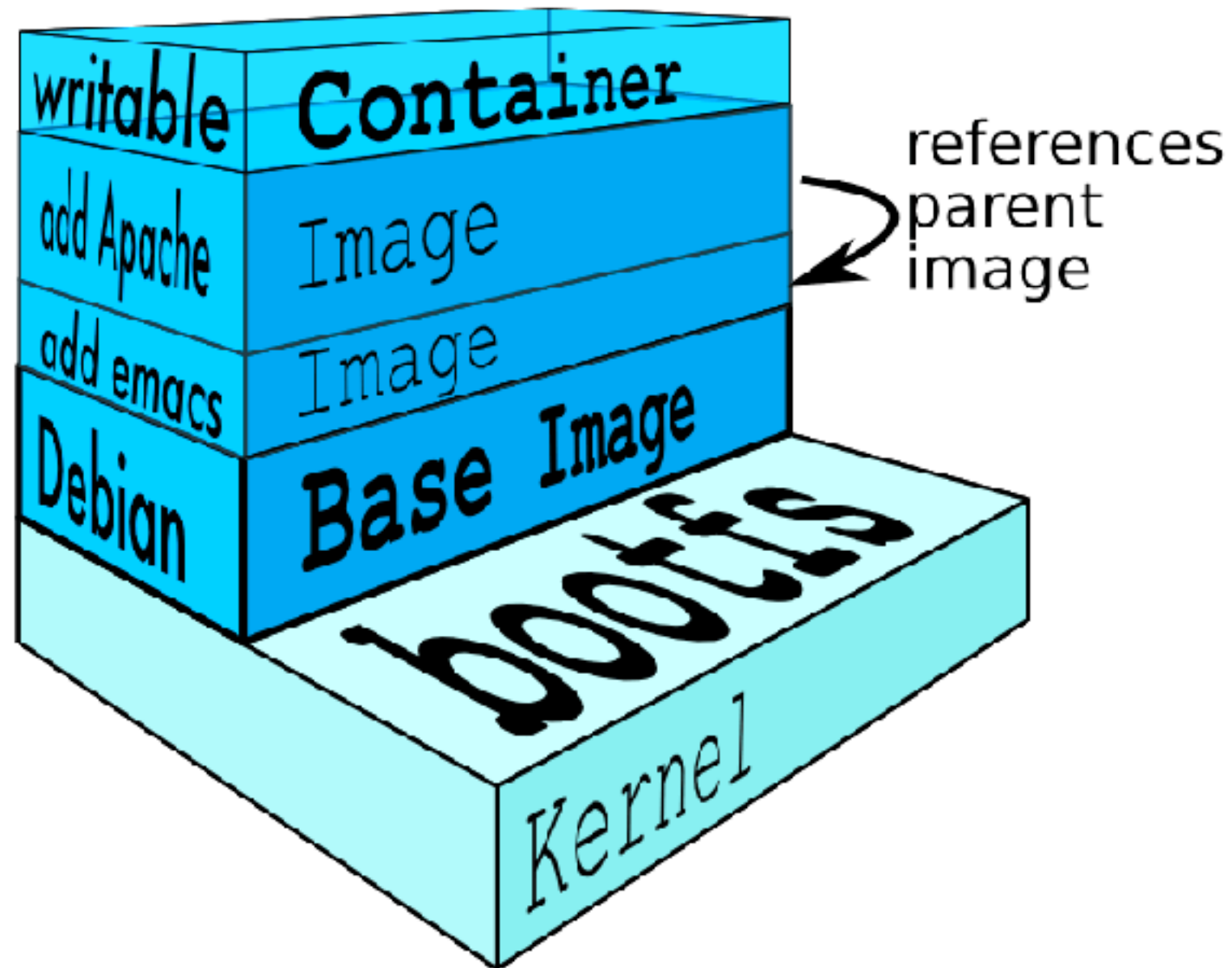
Each layer can add/change/remove files

Image can share layers

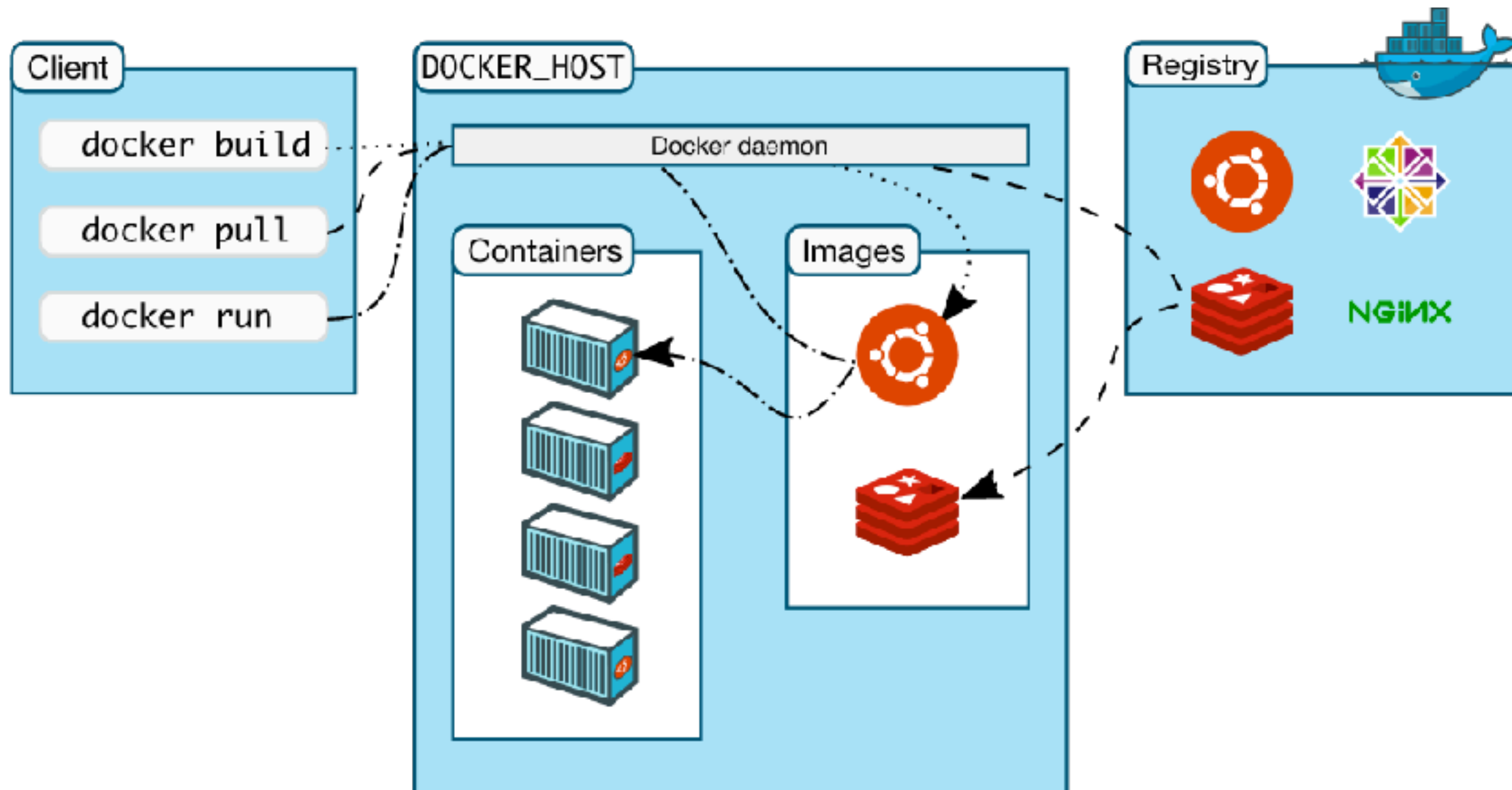
Read-only file system



Docker image layer



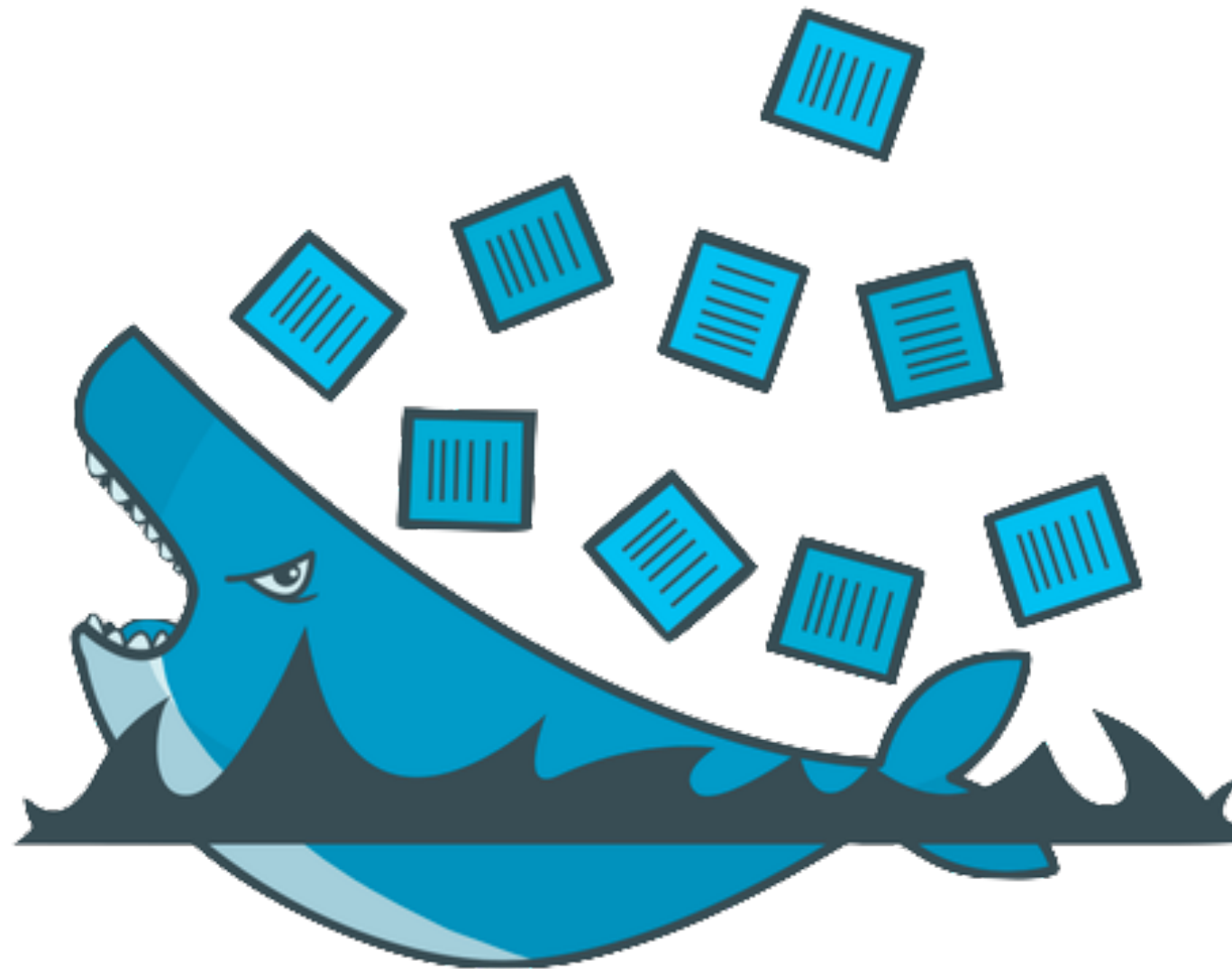
How docker works ?



<https://docs.docker.com/get-started/overview/>



Workshop



Working with Docker compose

<https://docs.docker.com/compose/>



Multiple containers app!!

Difficult to create and manage !!

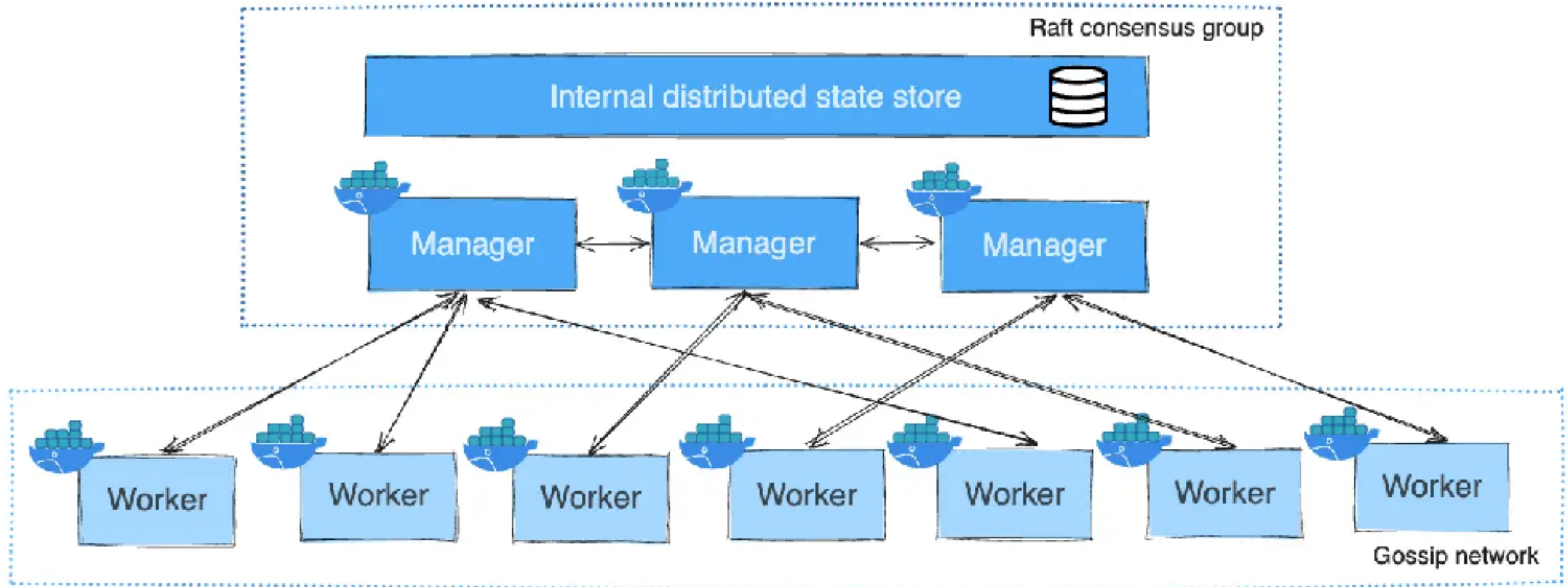


Mutiple containers app!!

Build images from Dockerfile
Pull images from Hub/private/cache
Configuration and create containers
Start and stop containers
Stream their logs



Docker Swarm



<https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/>



Introduction



kubernetes





Why we need Kubernetes ?

Managing containers for production is challenging
Need something to manage beyond a container
engine !!



Key capabilities was missing !!

Using multiple containers with **shared** resources

Monitoring running containers

Handling **dead containers**

Autoscaling container instances to handle load

Making the container services **easily** accessible

Handling **dead connecting** containers to a variety of external data sources

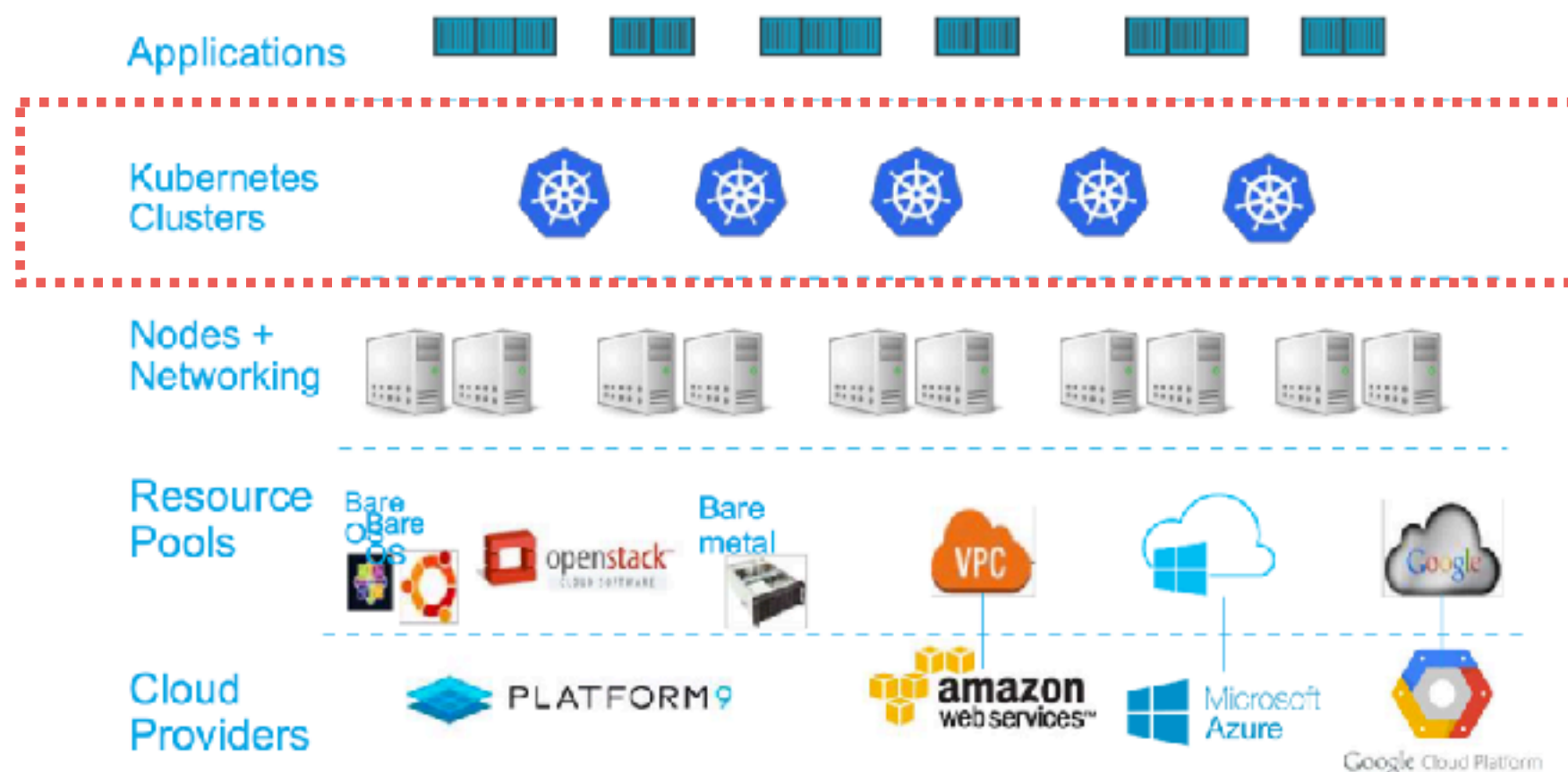


Write once, run anywhere

Eliminate infrastructure lock-in

Use containers

Provides management for containers



Deployment, not infrastructure

Software deployment is hard
Infrastructure provisioning/re-provisioning
Configuration networking and load balance
Redundancy (scale-out)
Lifecycle management (Software update)



K8s support for deployment

Scale-out service

Rolling update for new version

Rollback to a previous version

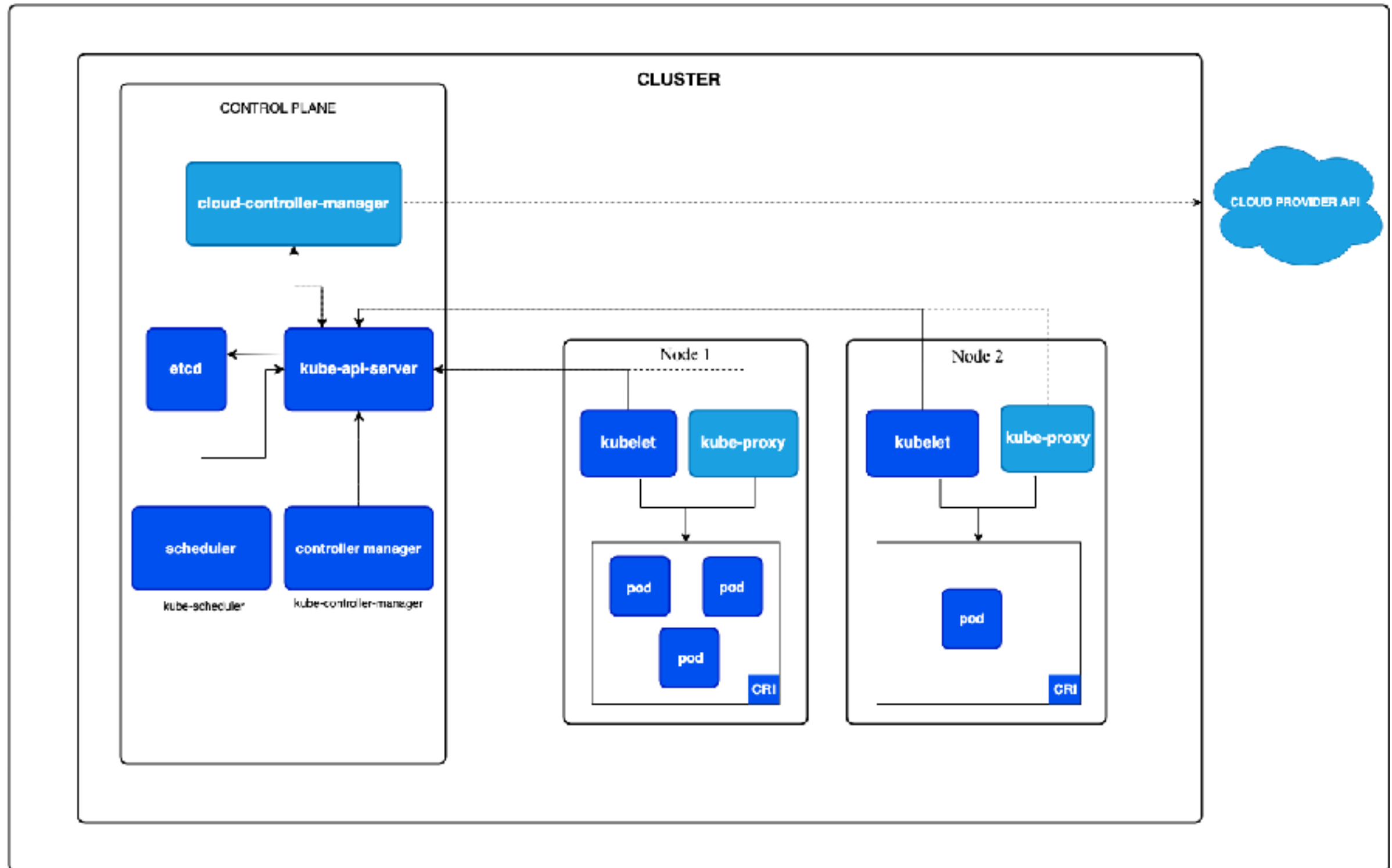
Pause and resume a deployment

Horizontal auto-scaling

Canary deployment



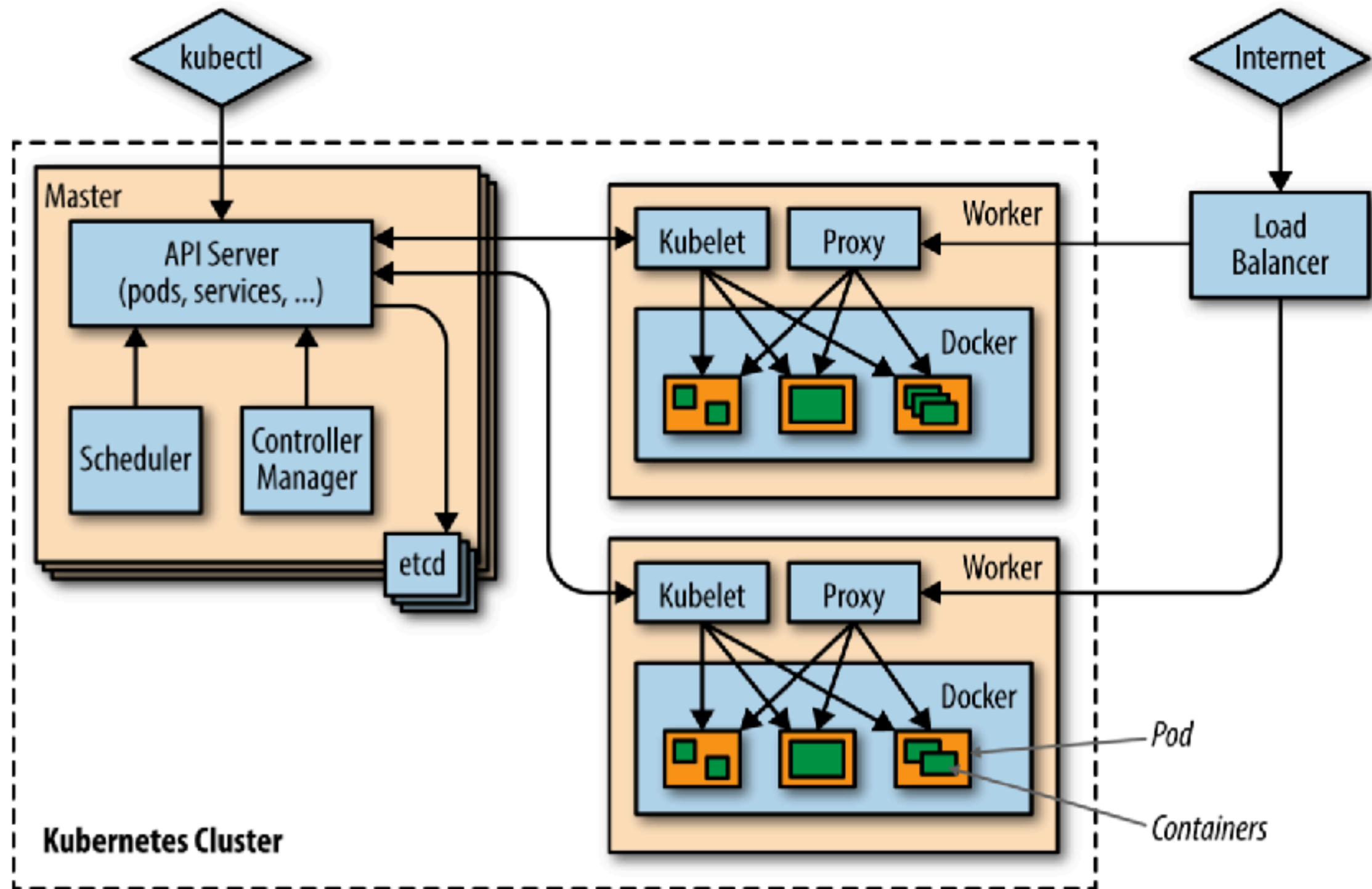
Kubernetes Architecture



<https://kubernetes.io/docs/concepts/architecture/>

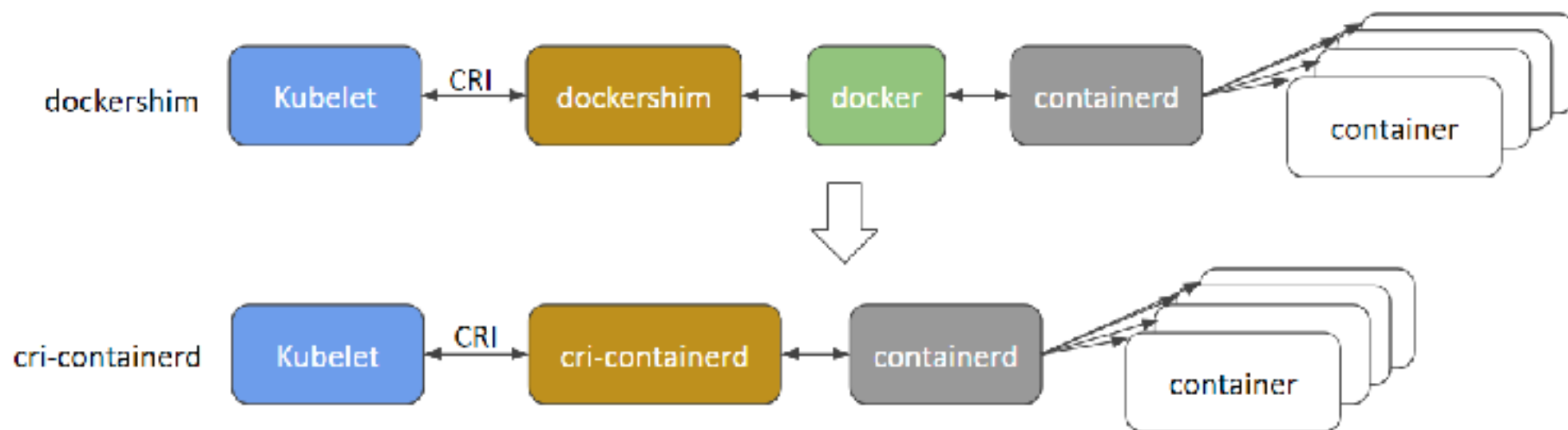


Kubernetes Architecture



Container runtime ?

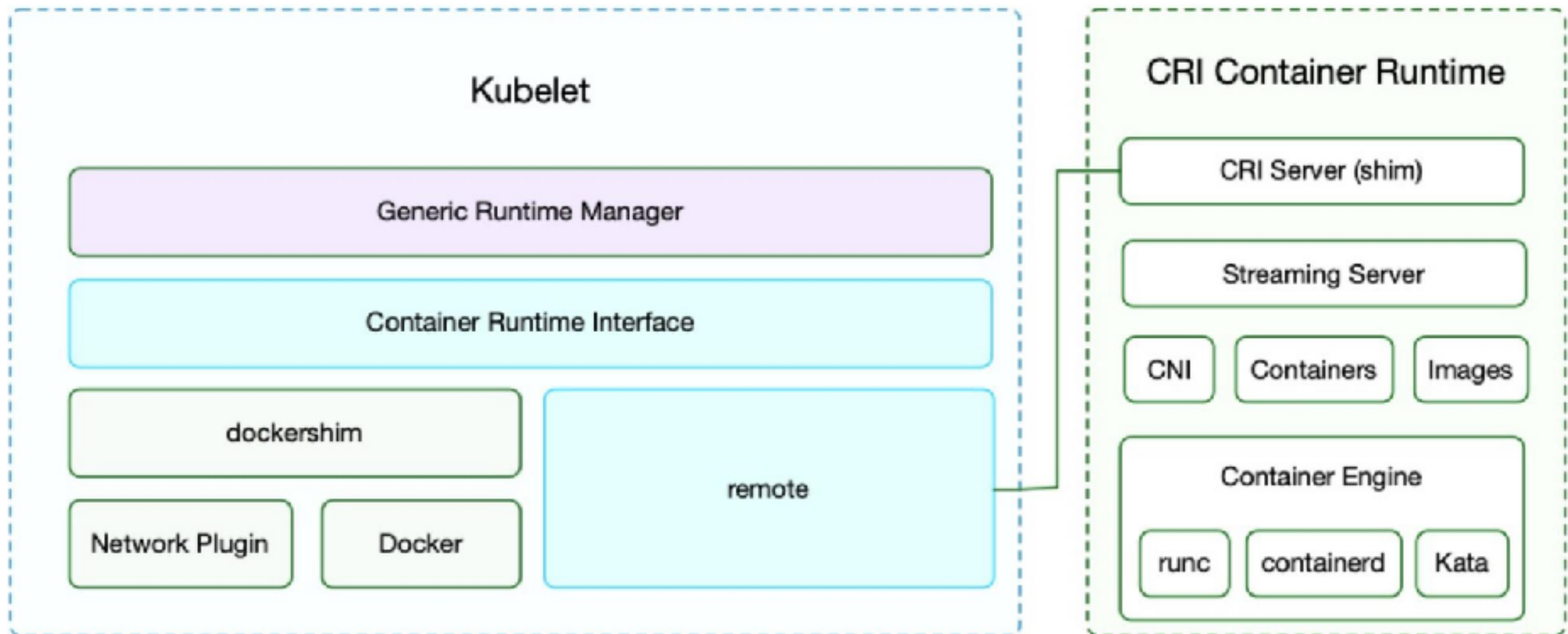
Need to install container runtime into every nodes
Support Container Runtime Interface (CRI)
Kubernetes removed dockershim !!



<https://kubernetes.io/docs/setup/production-environment/container-runtimes/>



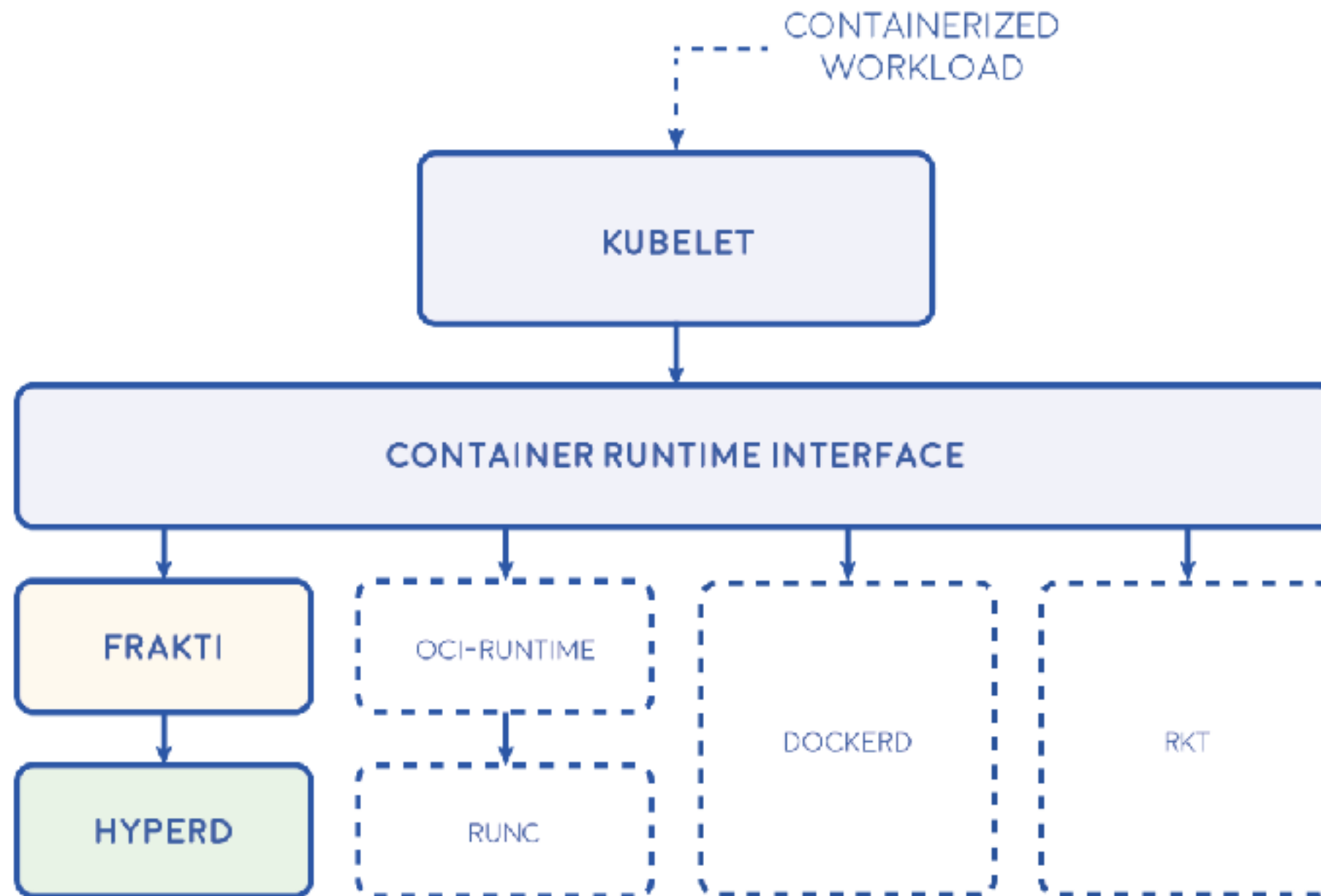
Container Runtime Interface



<https://kubernetes.io/docs/setup/production-environment/container-runtimes/>



Container Runtime Interface



<https://www.devopsschool.com/blog/list-of-top-container-runtime-interface-projects/>

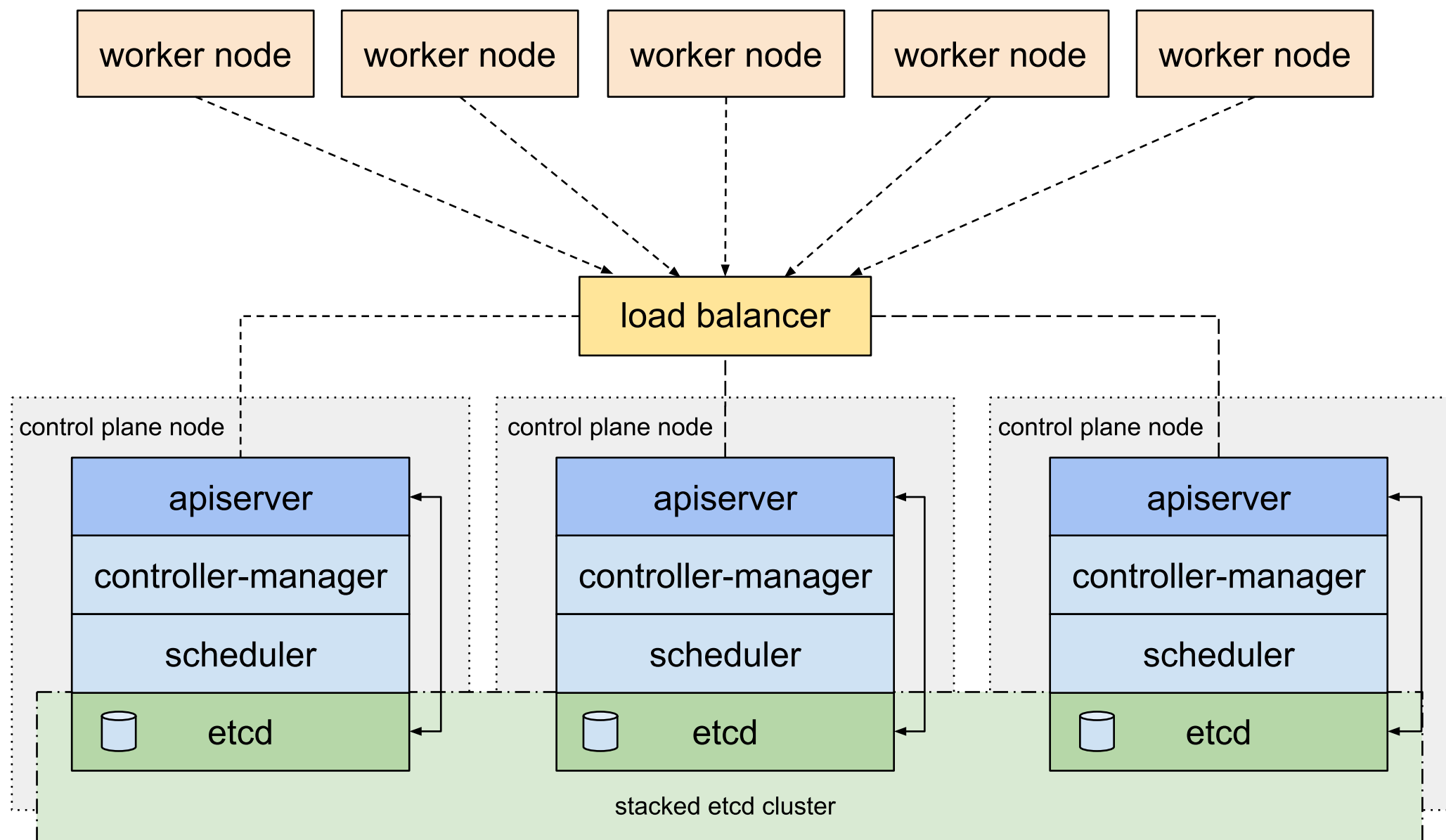


High Availability Cluster



High Availability Cluster (1)

kubeadm HA topology - stacked etcd

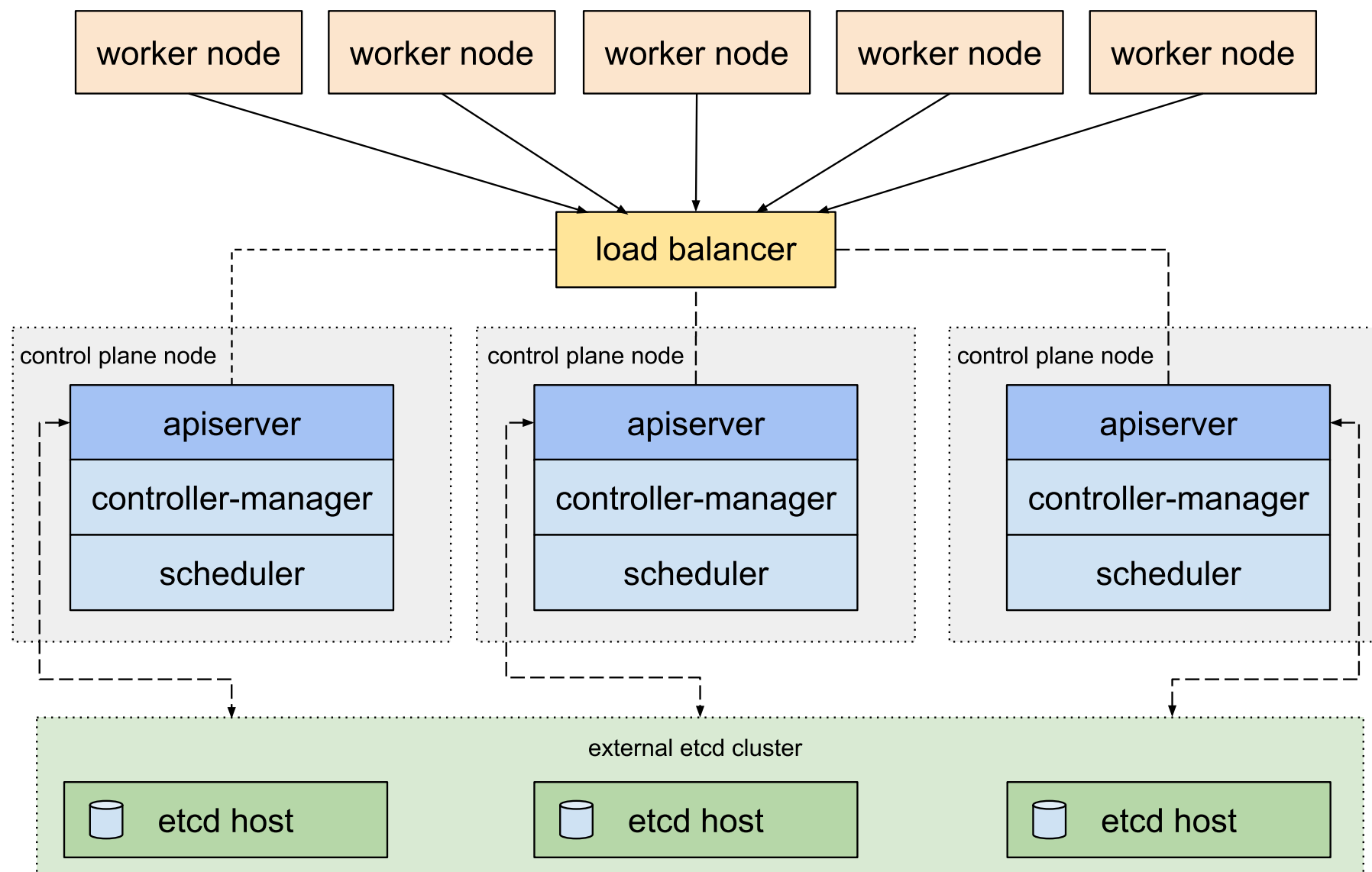


<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/ha-topology/>



High Availability Cluster (2)

kubeadm HA topology - external etcd

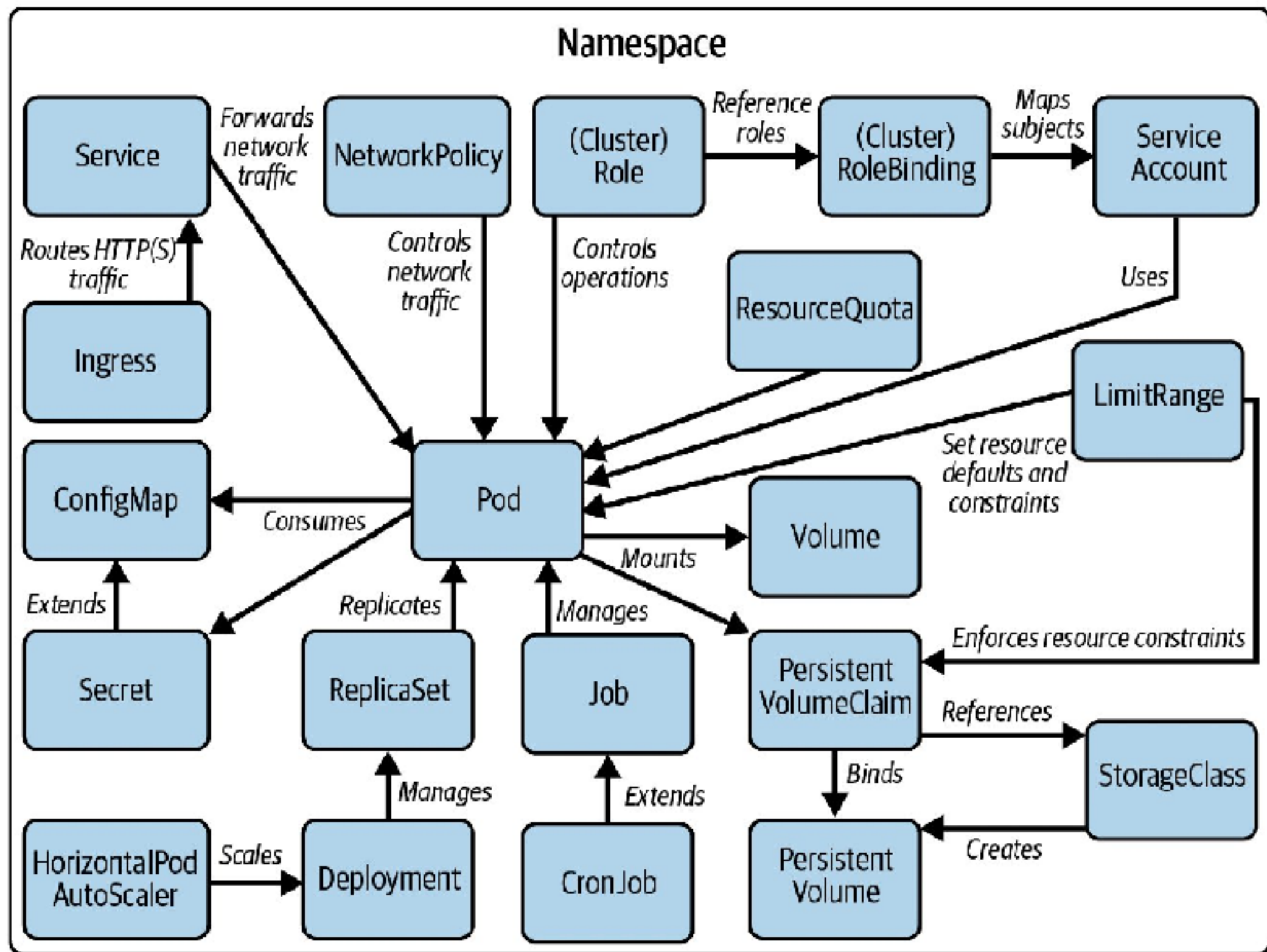


<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/ha-topology/>



K8s components





K8s components

Pods

Services

Deployments

ConfigMap and Secret

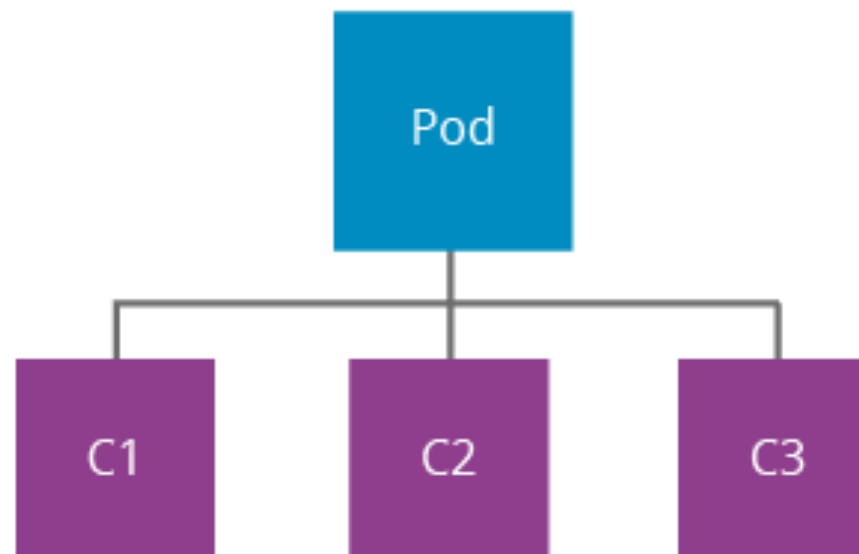
Volumes

StatefulSets



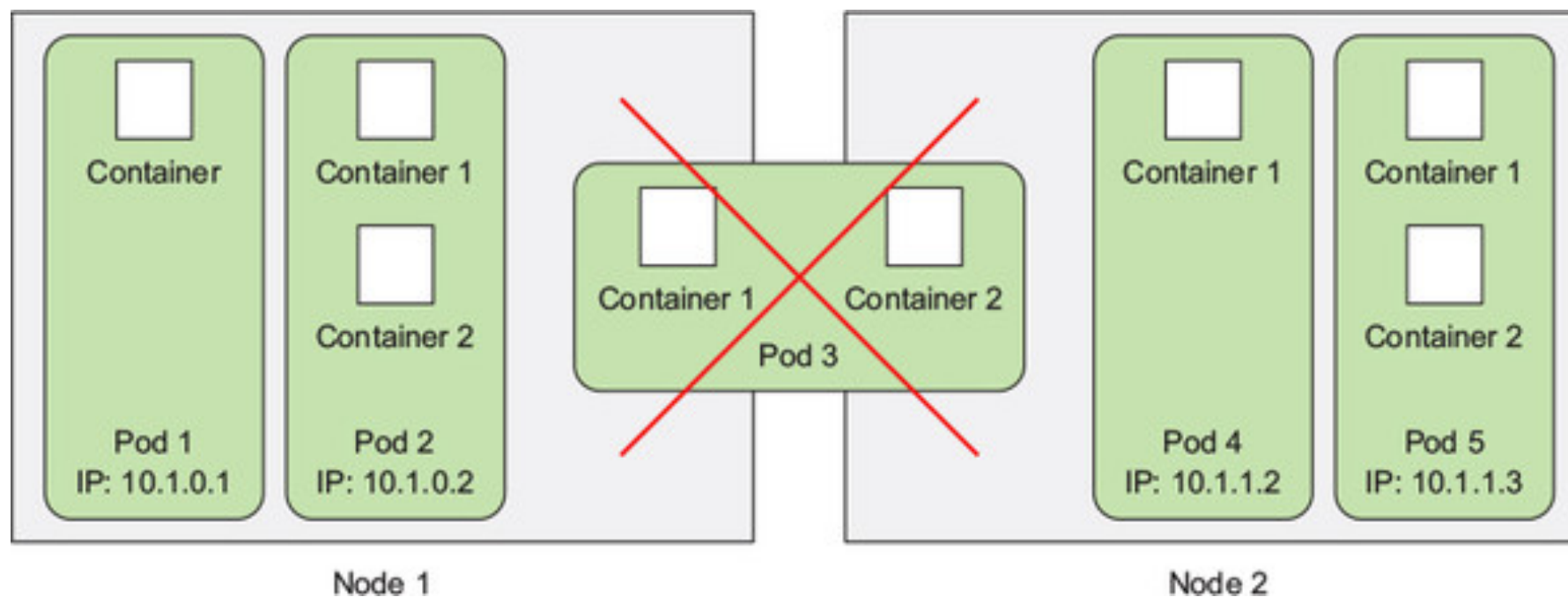
Pods

Small group of co-located containers
Optionally shared volume between containers
Basic deployment unit in Kubernetes



Pods

1 pods = 1 container
1 pods = N containers



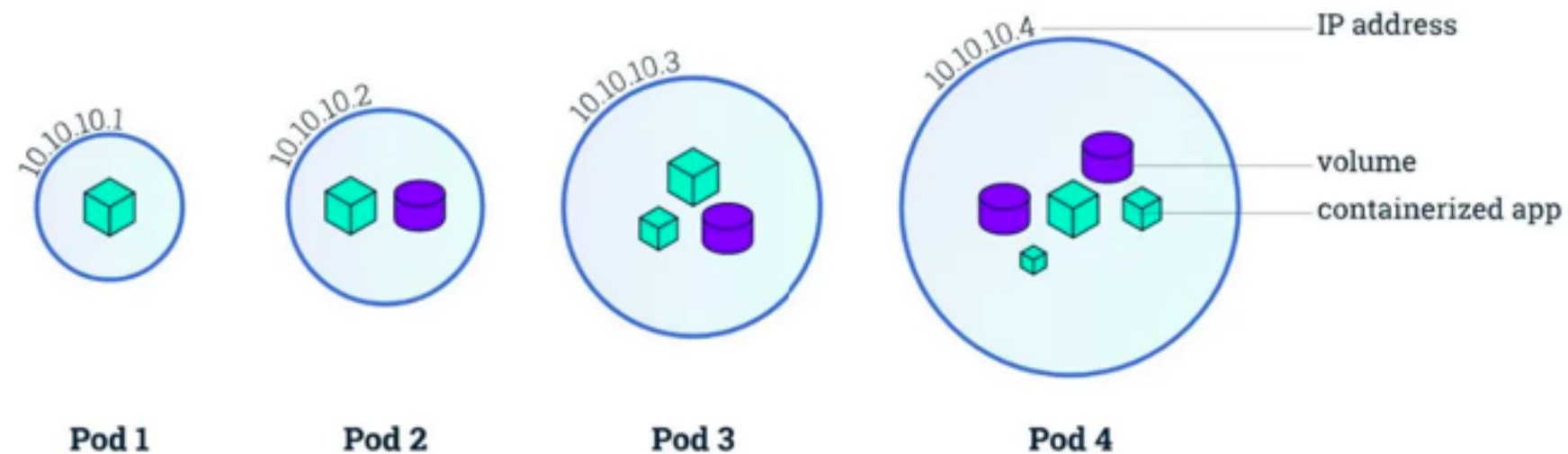
All containers in same Pods

Share process ID

Share network interface

Share Hostname/IP/Port

Share Unix Time Sharing (UTS)



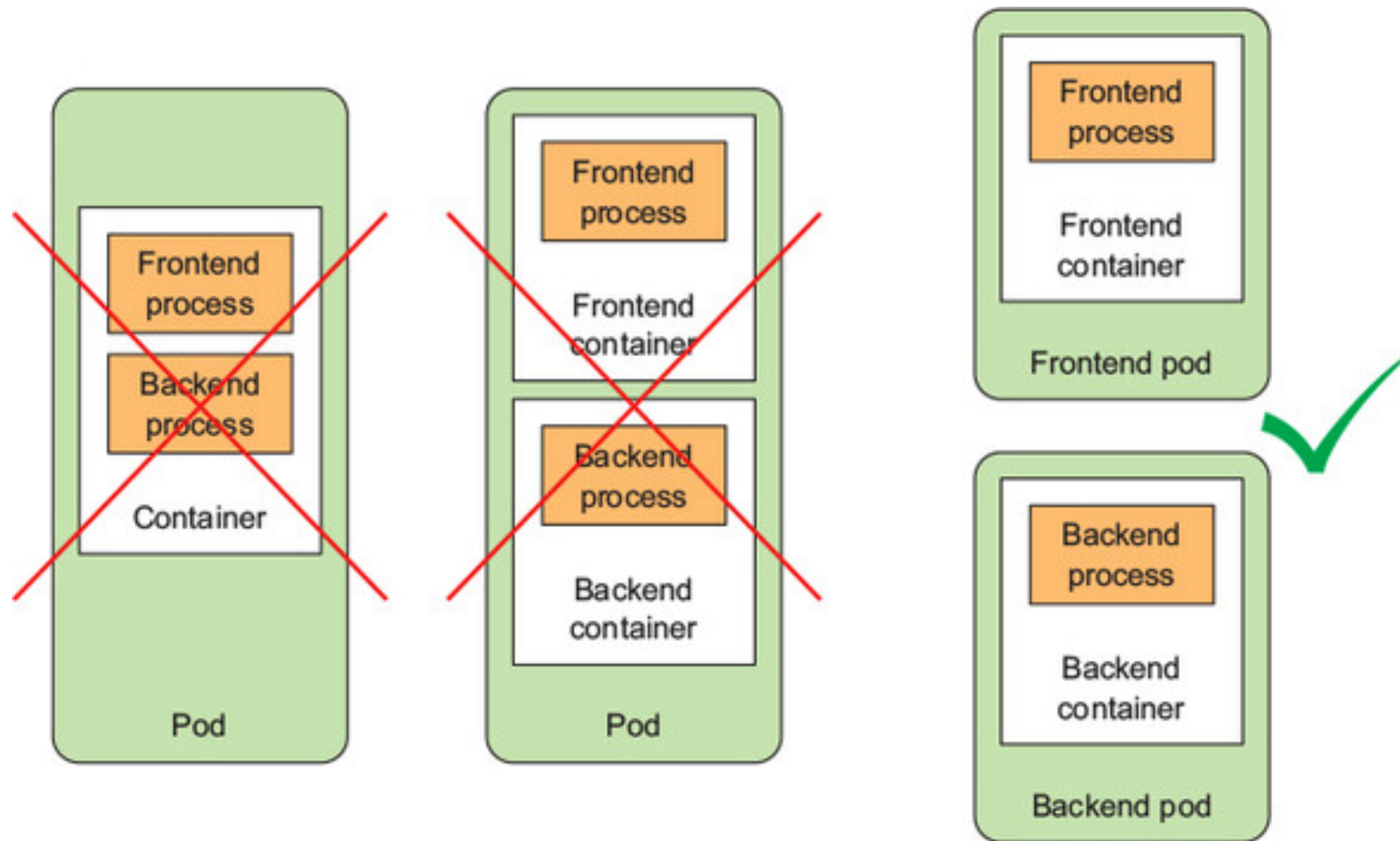
Pods Lifecycle

Phase	Description
Pending	Accepted by kubernetes but container not created yet
Running	Pods bound to the node, all containers created and at least one container is running/starting/restarting
Succeeded	Containers exited with status 0
Failed	All containers exit and at least one exited with non-zero status
Unknow	State of Pods can't be determined due to communication issues with its node



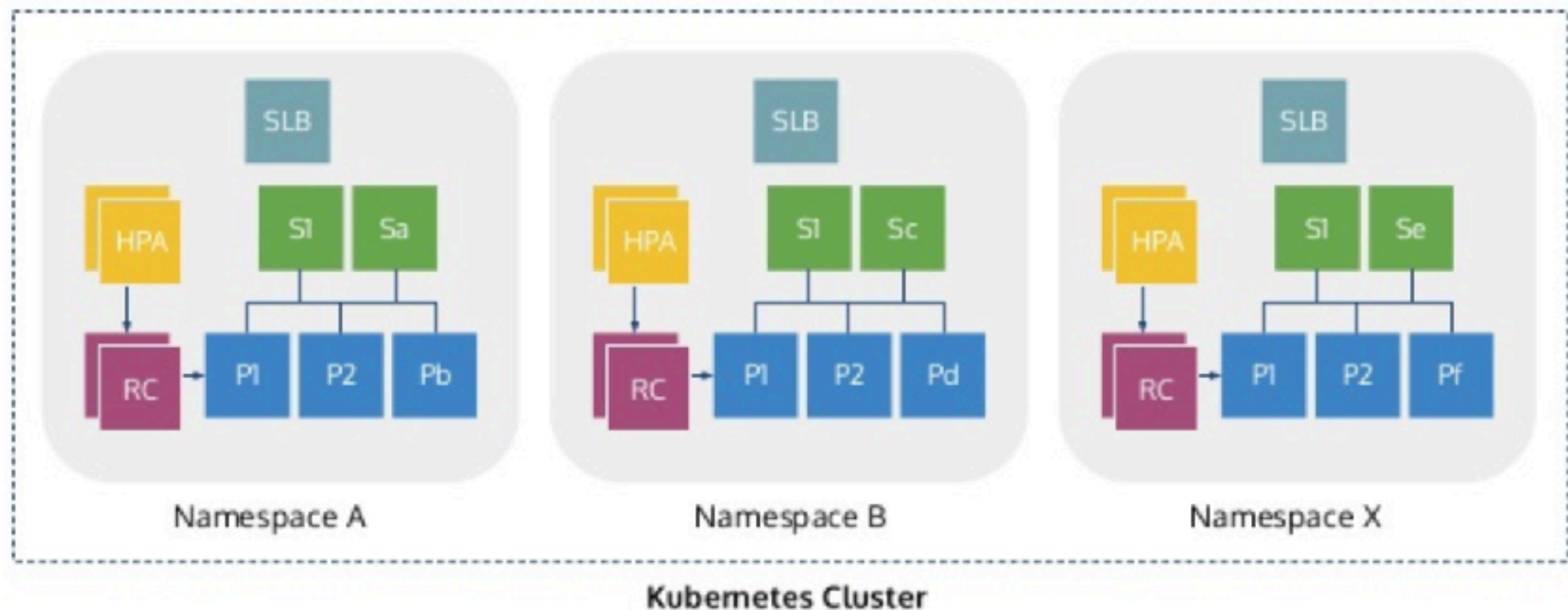
Organize container across Pods

Split multi-tiers app into multiple pods



Pods Namespaces

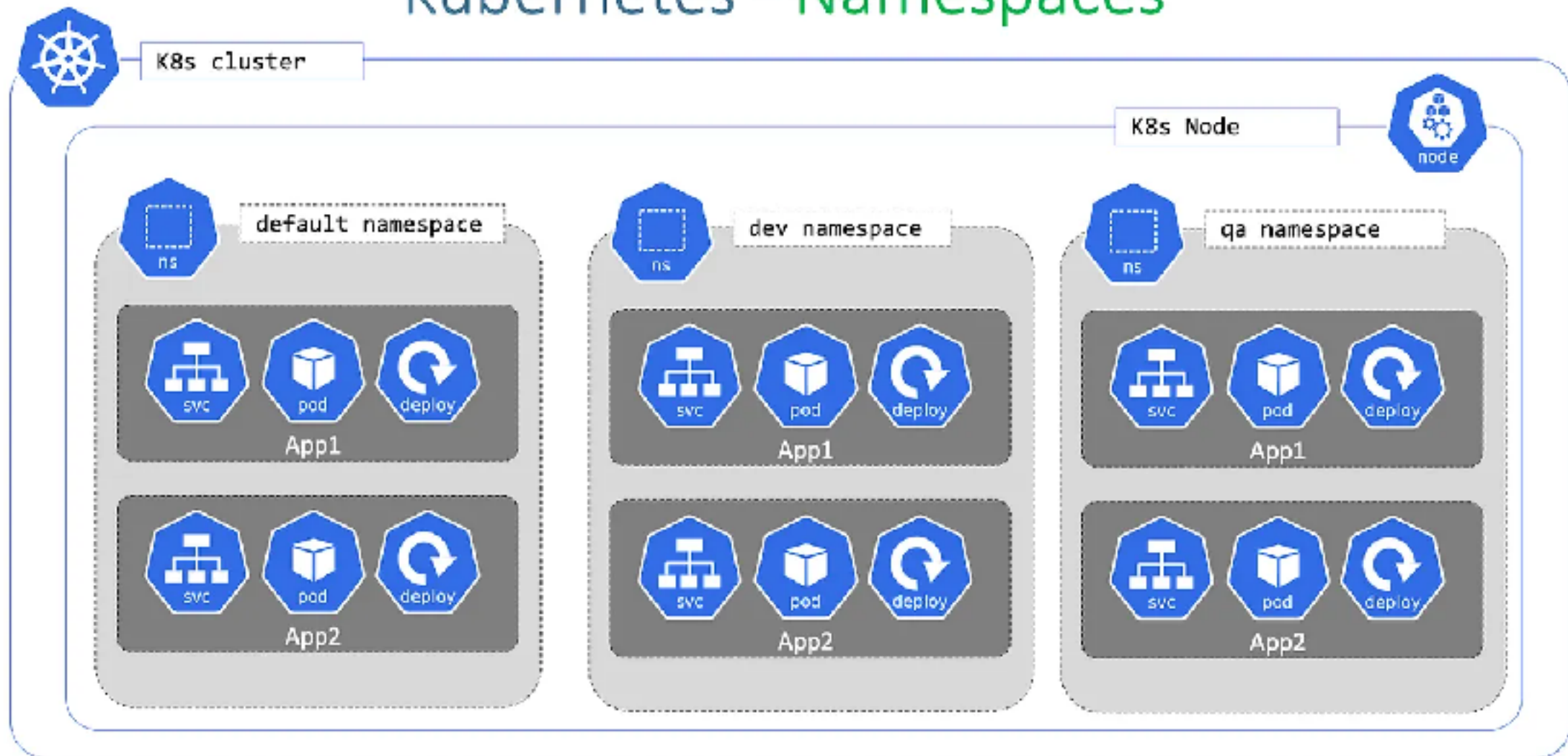
Allow different teams to use the same cluster



Pods Namespaces

Manage by environments

Kubernetes - Namespaces



<https://blog.devops.dev/day18-kubernetes-resource-quota-and-namespace-6a21045b0d97>



Deployment ReplicaSet (RS)



Deployment and ReplicaSet (RS)

Next-generation of Replication Controller (RC)
Provide function to maintain versioning of Pods
 Update new version (Rollout)
 Revert to old version (Rollback)
 Scale a deployment
 Pause/Resume process

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>



Kubernetes Deployment

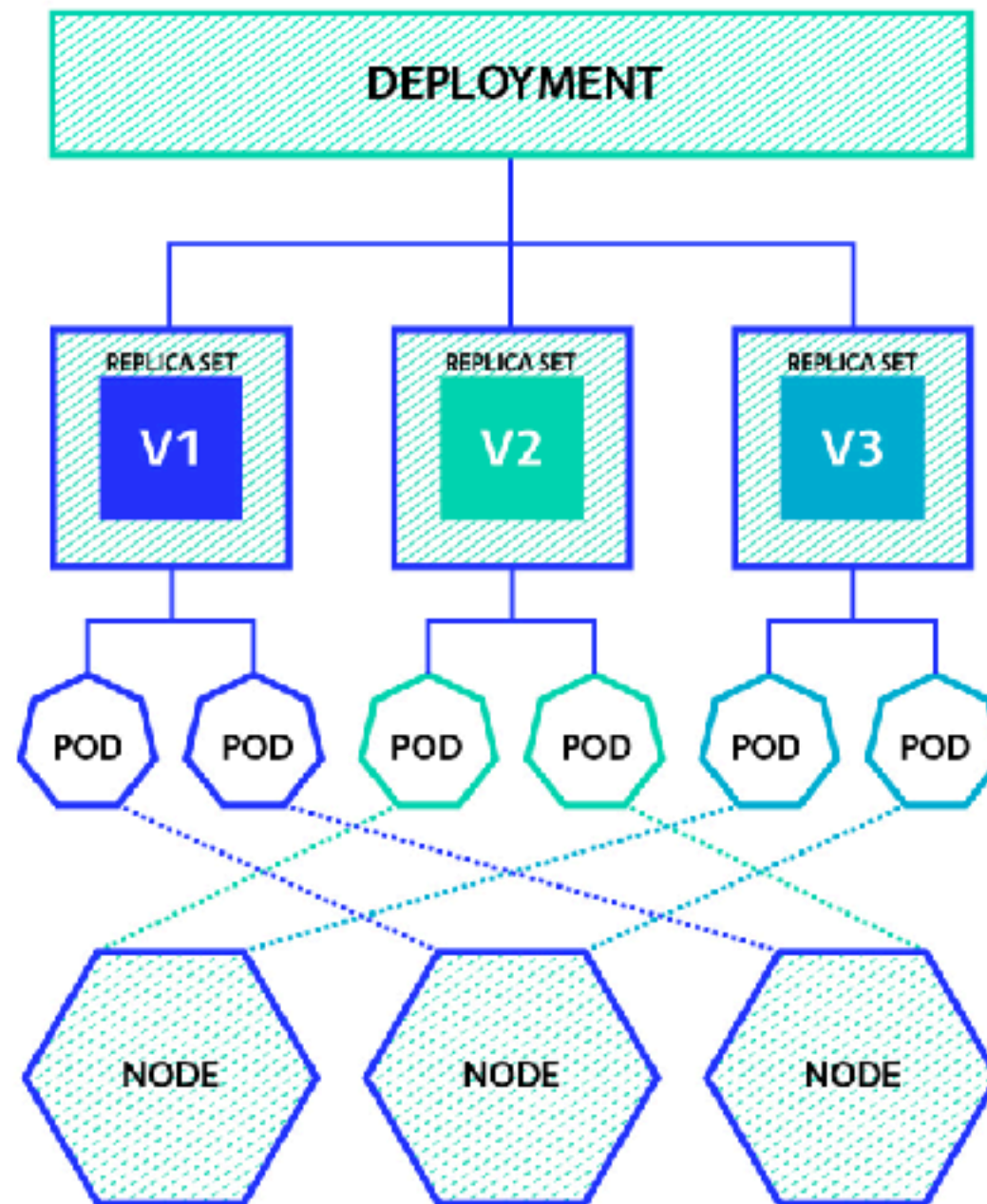
Define the application lifecycle
How to update application ?



<https://www.plural.sh/blog/kubernetes-deployments-guide/>



Deployment and ReplicaSet



Deployment strategies ?

Control the behavior of Pod replacement

Ensure availability

Minimal disruption



Deployment strategies ?

**Rolling update
(Default)**

Recreate

Blue/Green

Canary

A/B testing

Ramped slow
rollout

Shadow
deployment

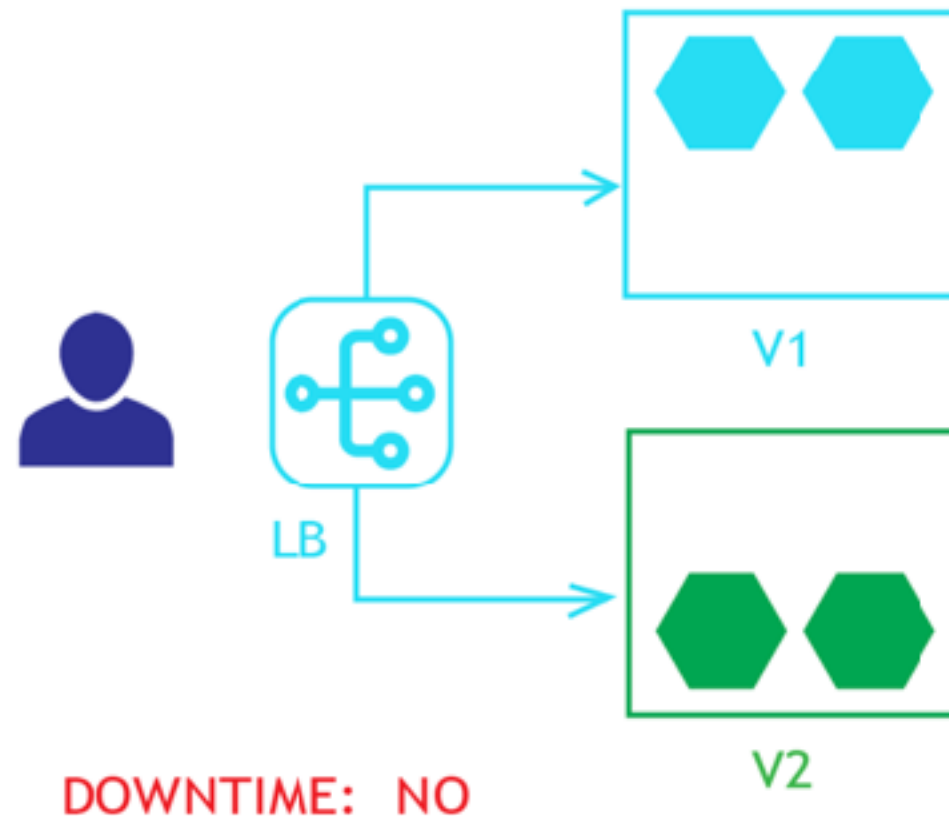
Best-effort
controlled rollout

<https://spacelift.io/blog/kubernetes-deployment-strategies>



Rollout update (default)

Gradually replaces old pods with new ones
Minimize downtime

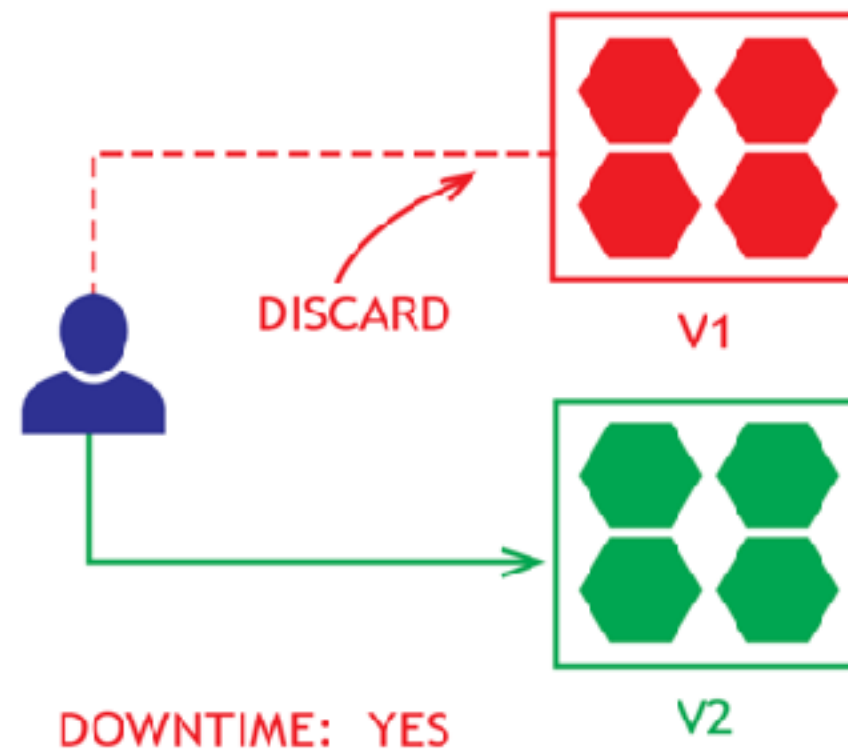


<https://2cloud.io/blog/kubernetes-deployment-strategies>



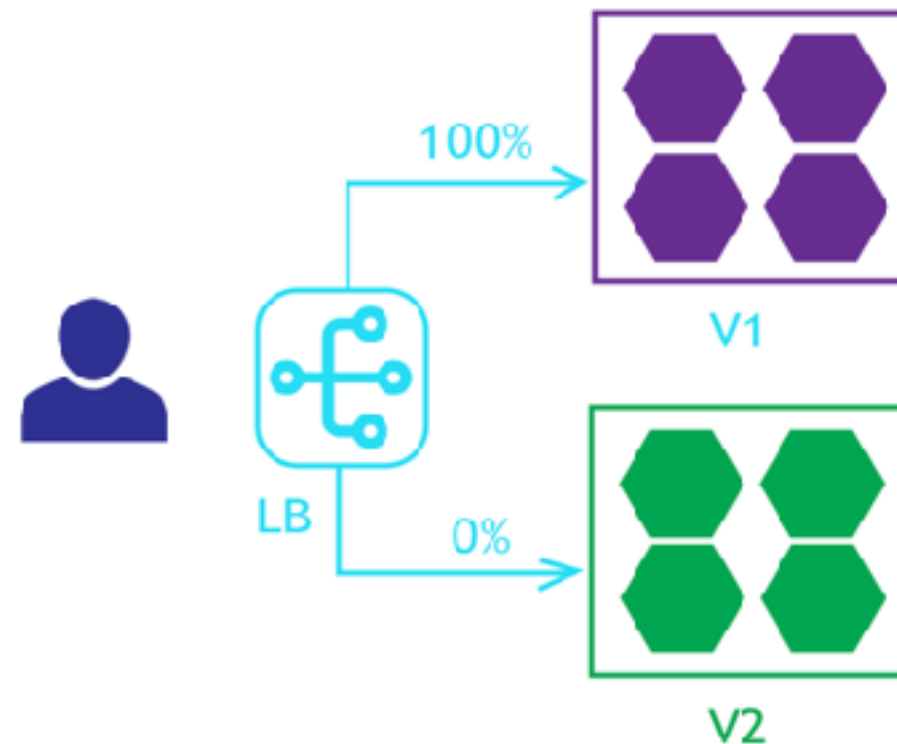
Recreate

Shutdown all existing pods before start new ones
Simple but causes downtime



Blue/Green

Deploy a **new version** alongside the **current**
Switch traffic once stable

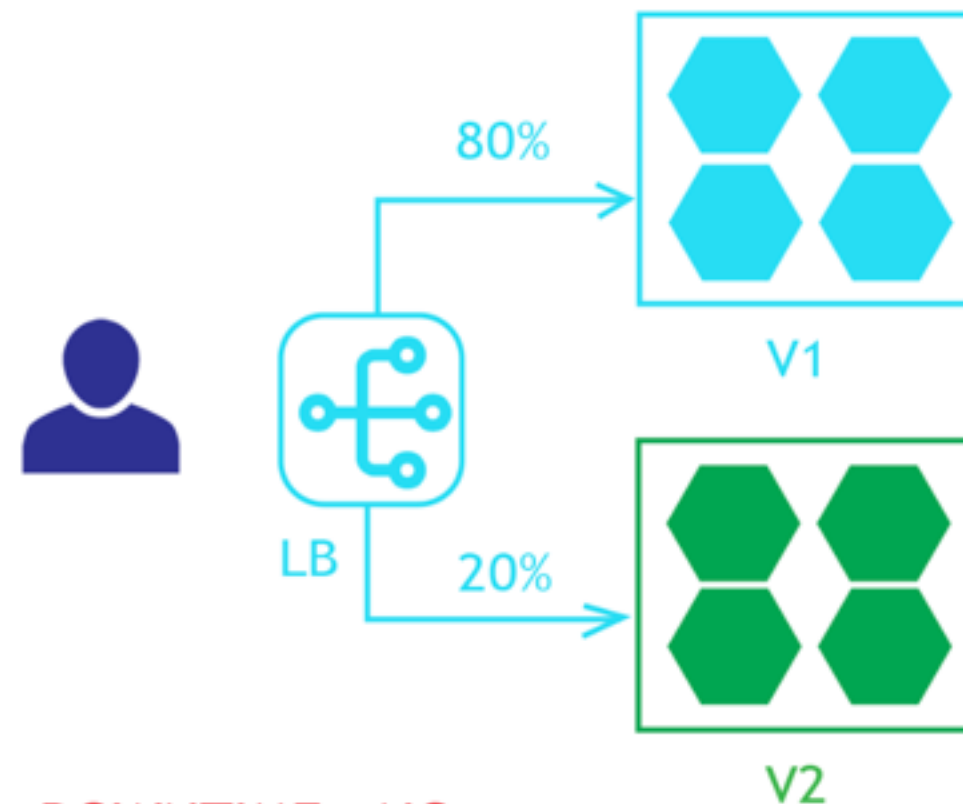


DOWNTIME: NO



Canary

Send a small percentage of traffic to new version
Gradually increasing after validation

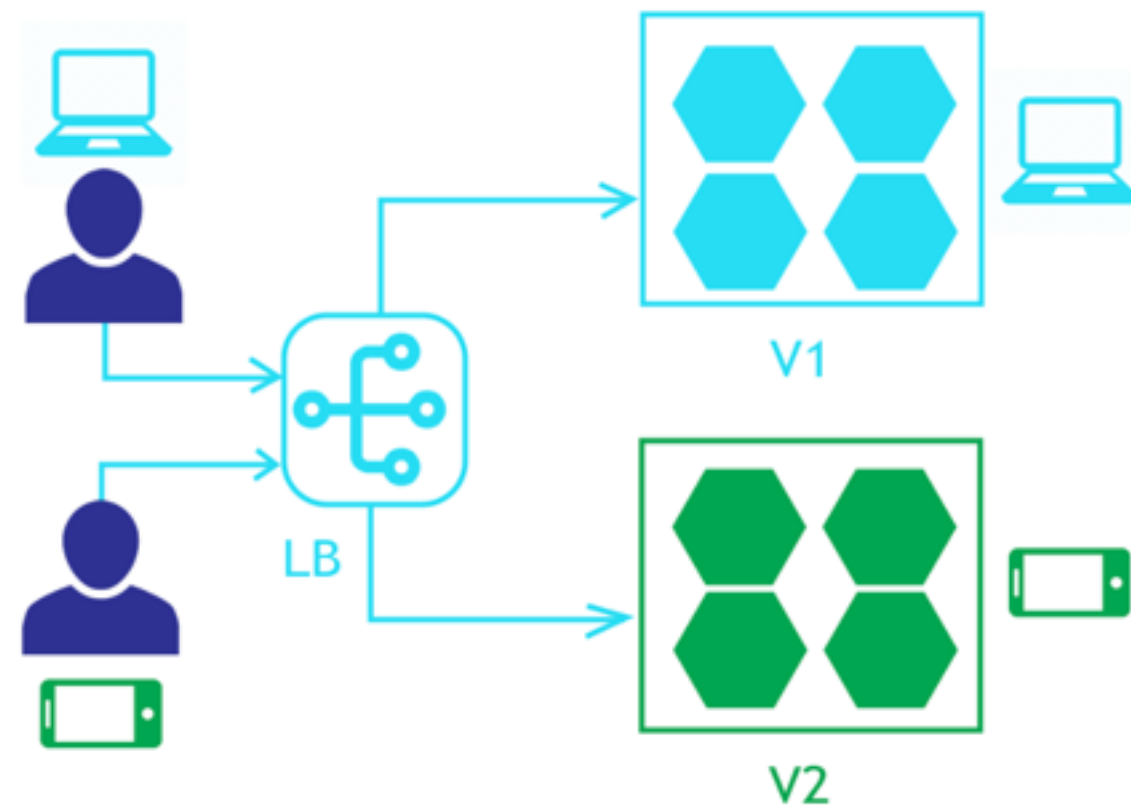


DOWNTIME: NO



A/B Testing

Route traffic to different versions based on **rules**
User segmentation, geolocation

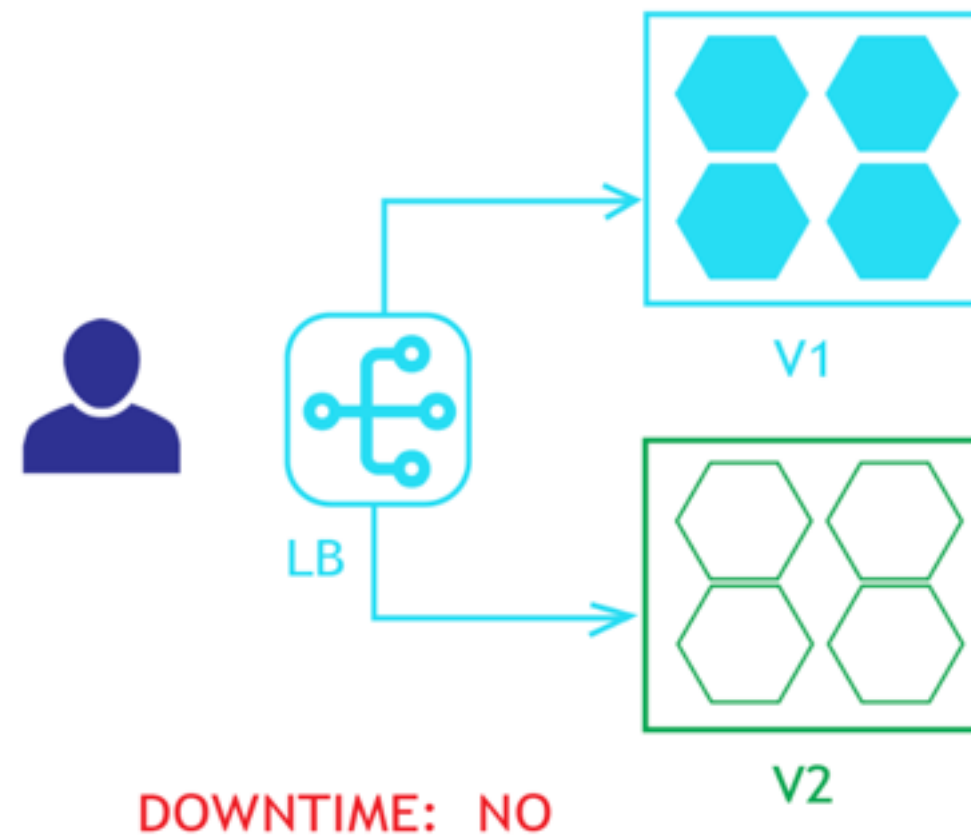


DOWNTIME: NOT APPLICABLE



Shadow deployment

Clone live traffic to a new version for testing
Without effecting users



Services discovery and Load balancing



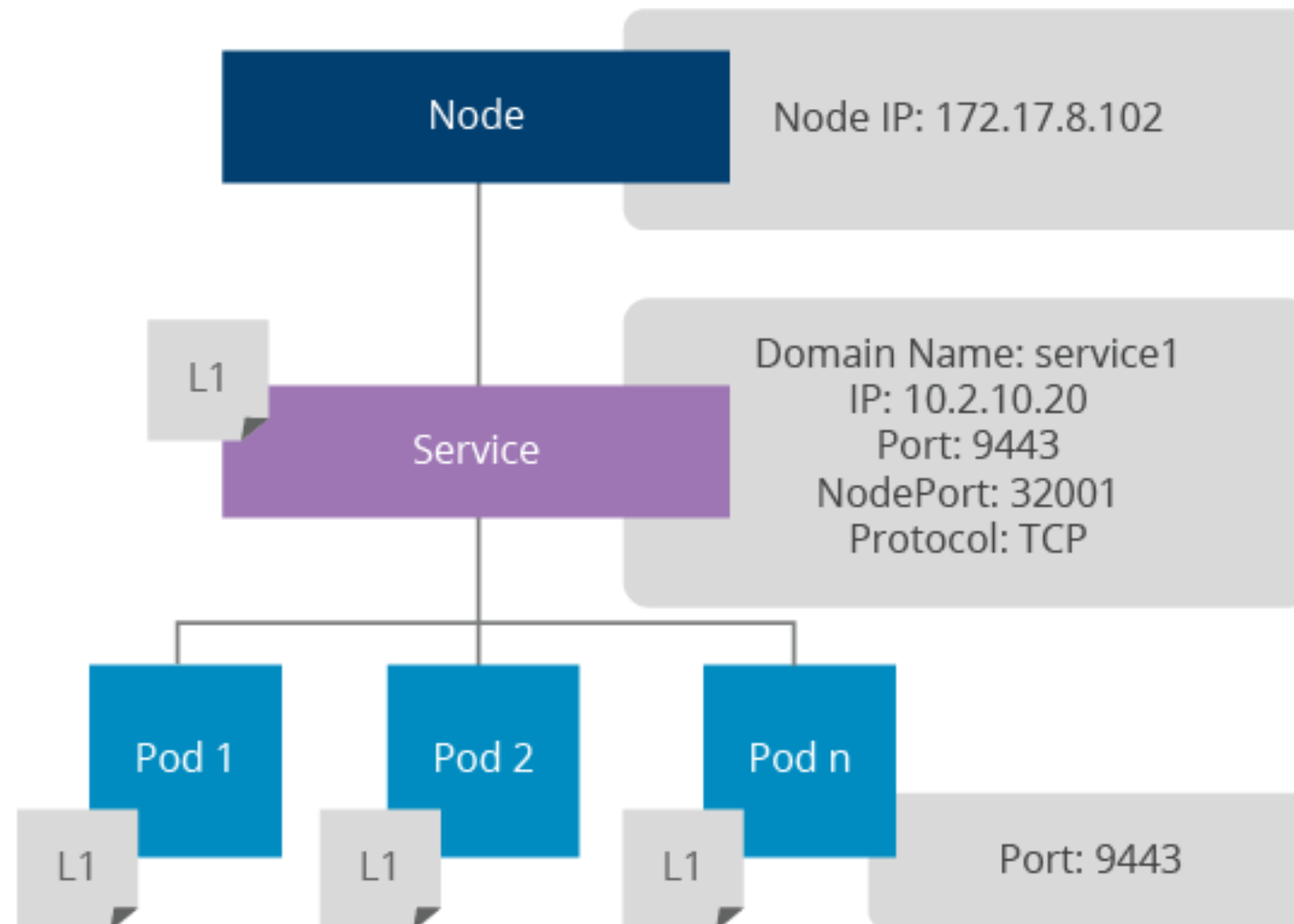
Services

Independent from Pods
Abstraction layer of Pods
Provide load balance
Expose access Pods/Load balance
Find Pods by label selector

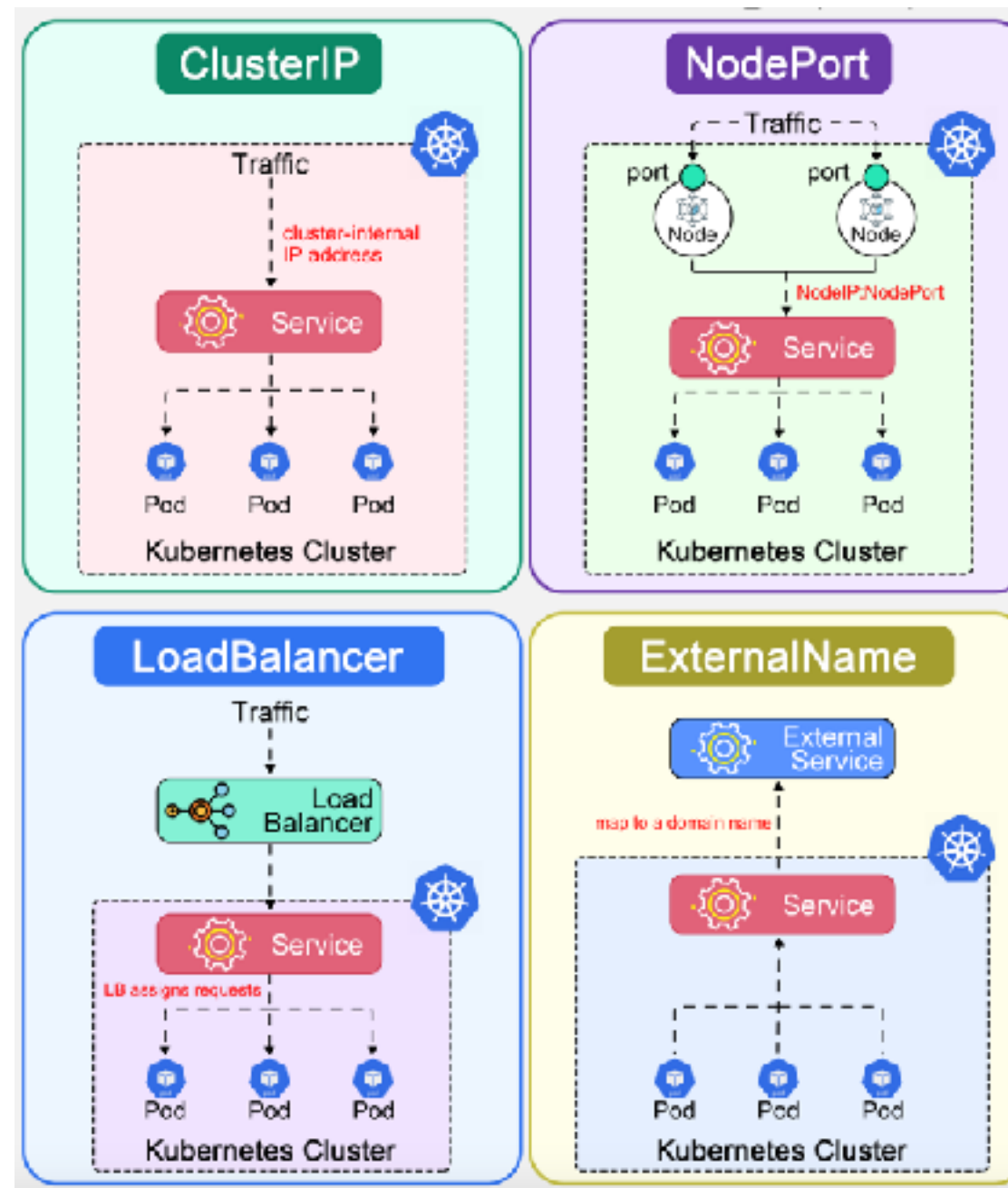
<https://kubernetes.io/docs/concepts/services-networking/service/>



Services



Types of services



<https://bytebytego.com/guides/top-4-kubernetes-service-types-in-one-diagram/>

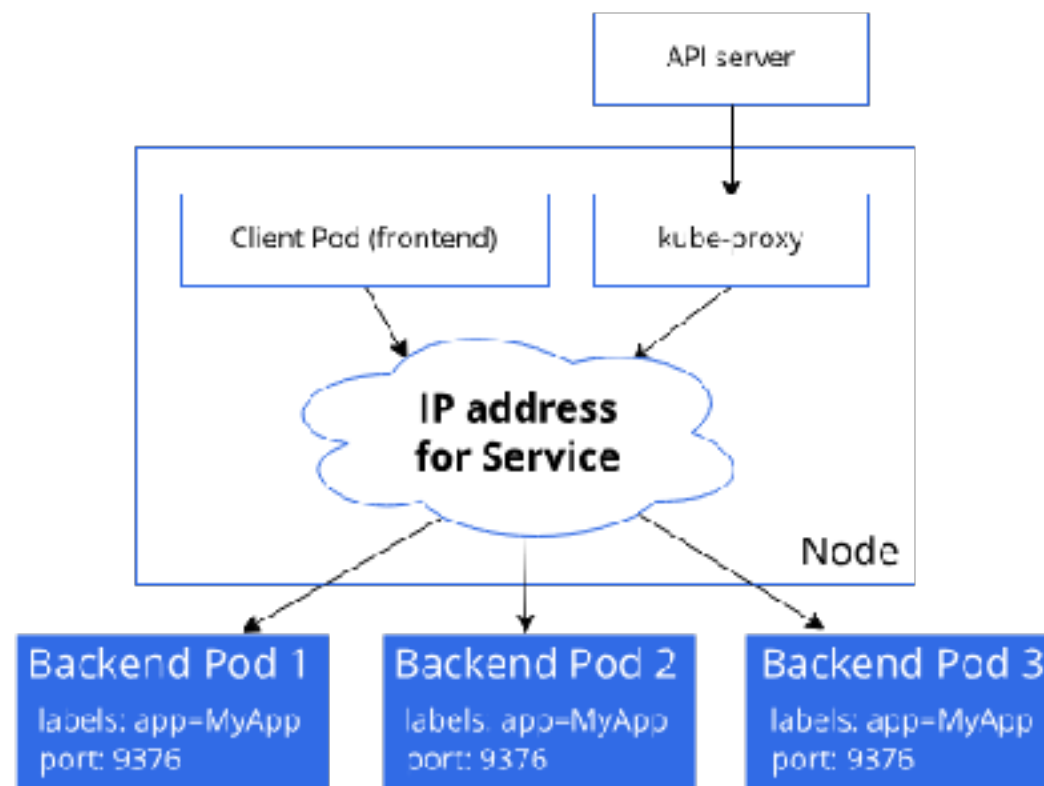


ClusterIP

Default of services

Access only from services in the same cluster

You can expose ClusterIP service with **kube-proxy**

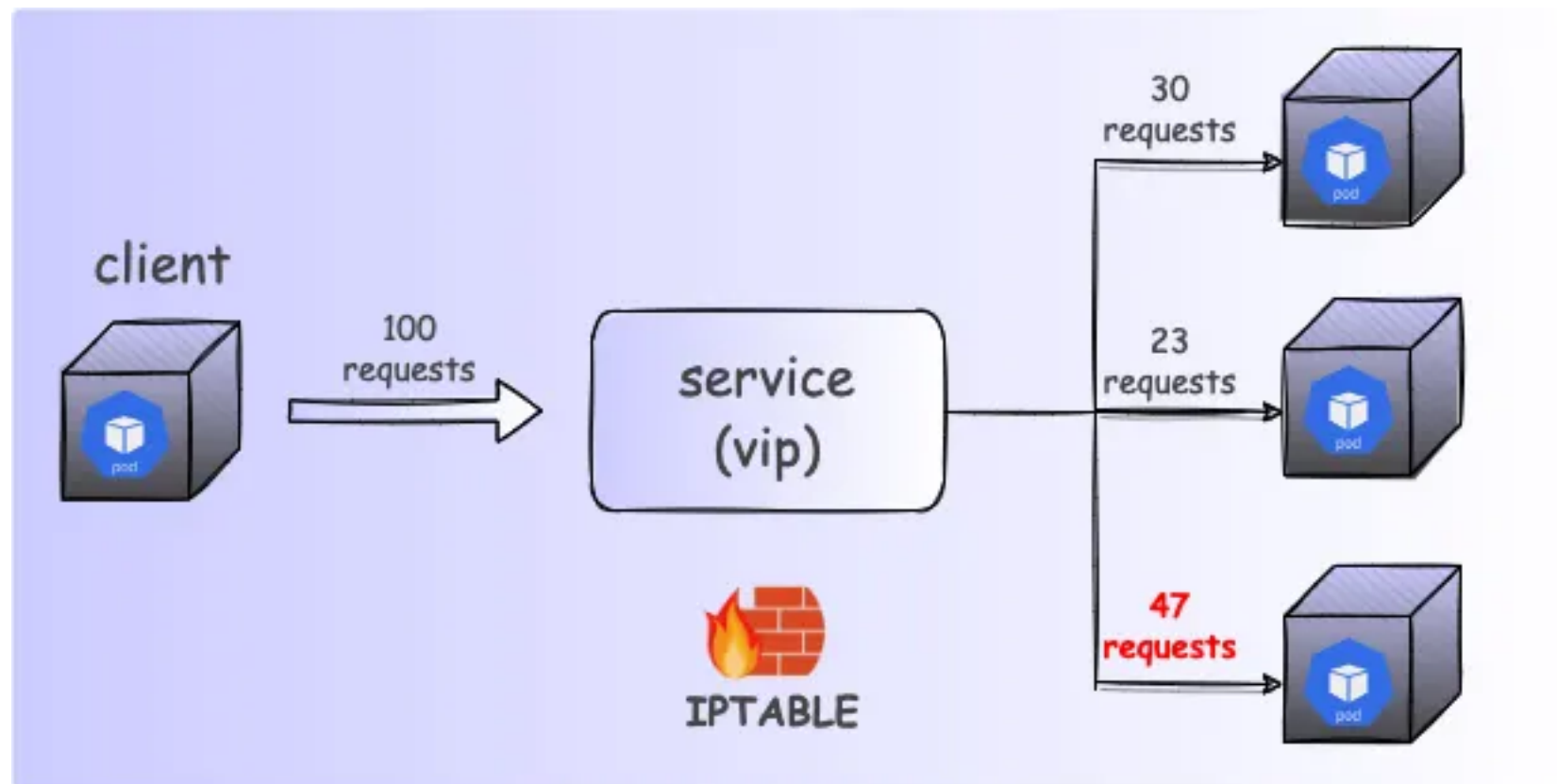


<https://kubernetes.io/docs/concepts/services-networking/cluster-ip-allocation/>



Kube-proxy

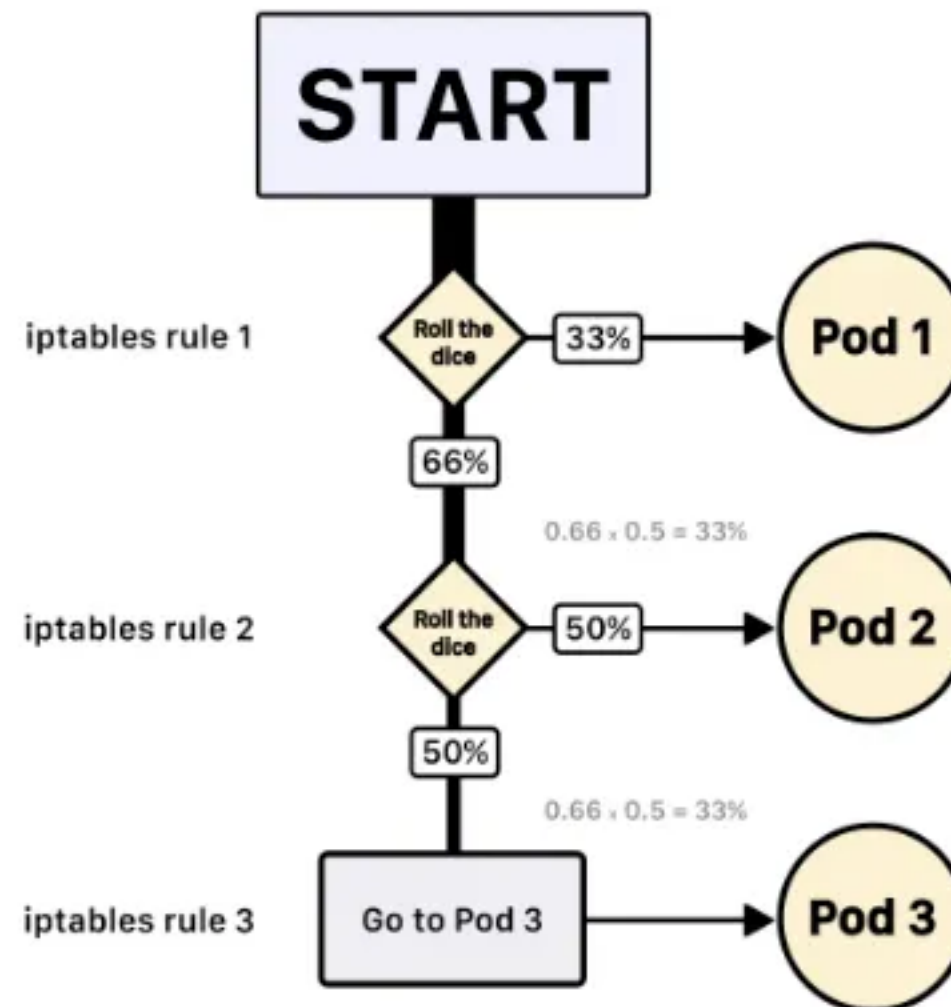
Default = **iptables**
IPVS, userspace



<https://kubernetes.io/docs/reference/networking/virtual-ips/#proxy-modes>



Kube-proxy



<https://learnkubernetes.com/kubernetes-long-lived-connections>



NodePort

Expose the service outside the cluster with port

Range of port 30000-32767

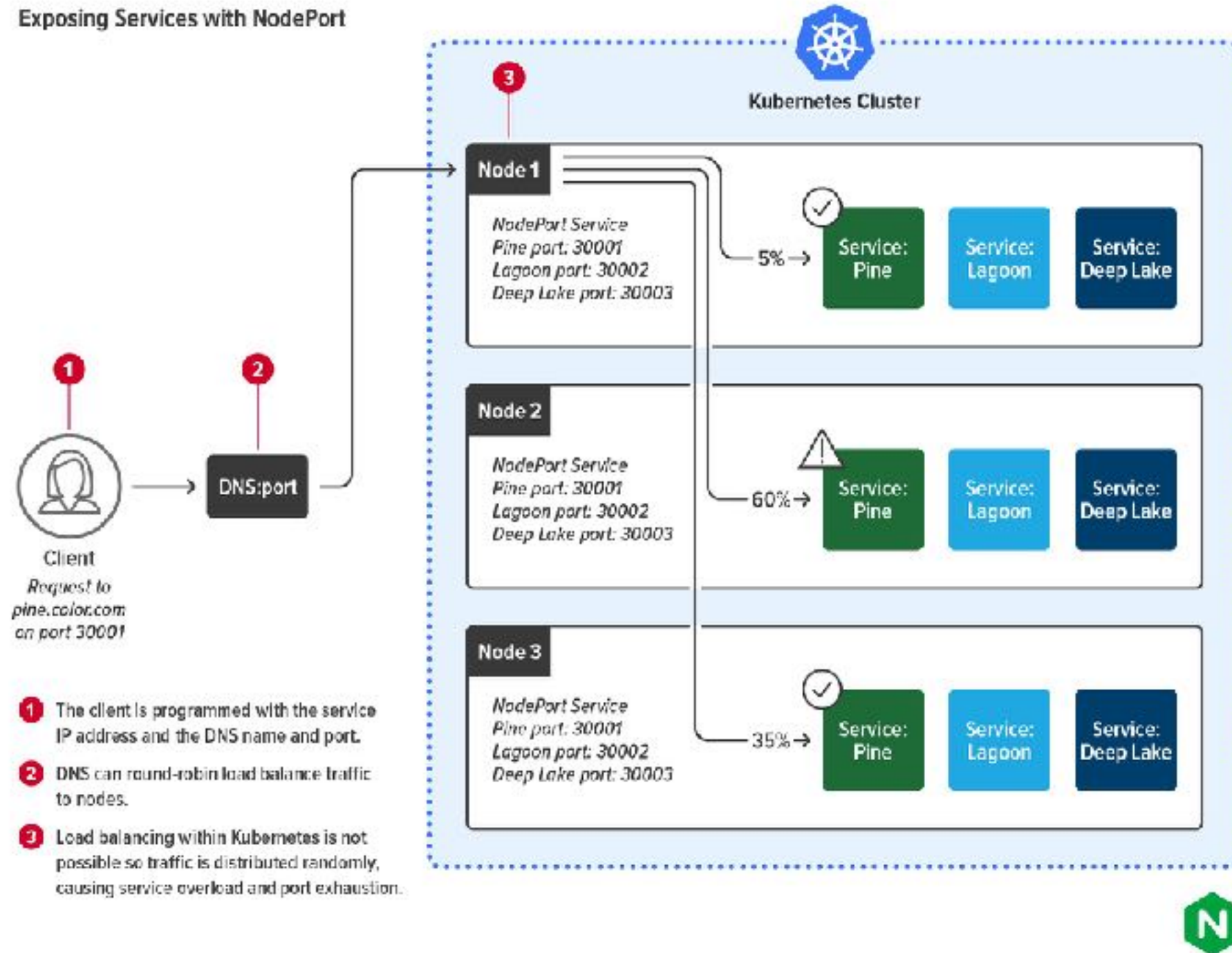
Use layer 4 routing rules and Linux **iptables** utilities

<https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>



Expose services with NodePort

Exposing Services with NodePort



<https://www.f5.com/company/blog/nginx/kubernetes-networking-101>



Load Balance (LB)

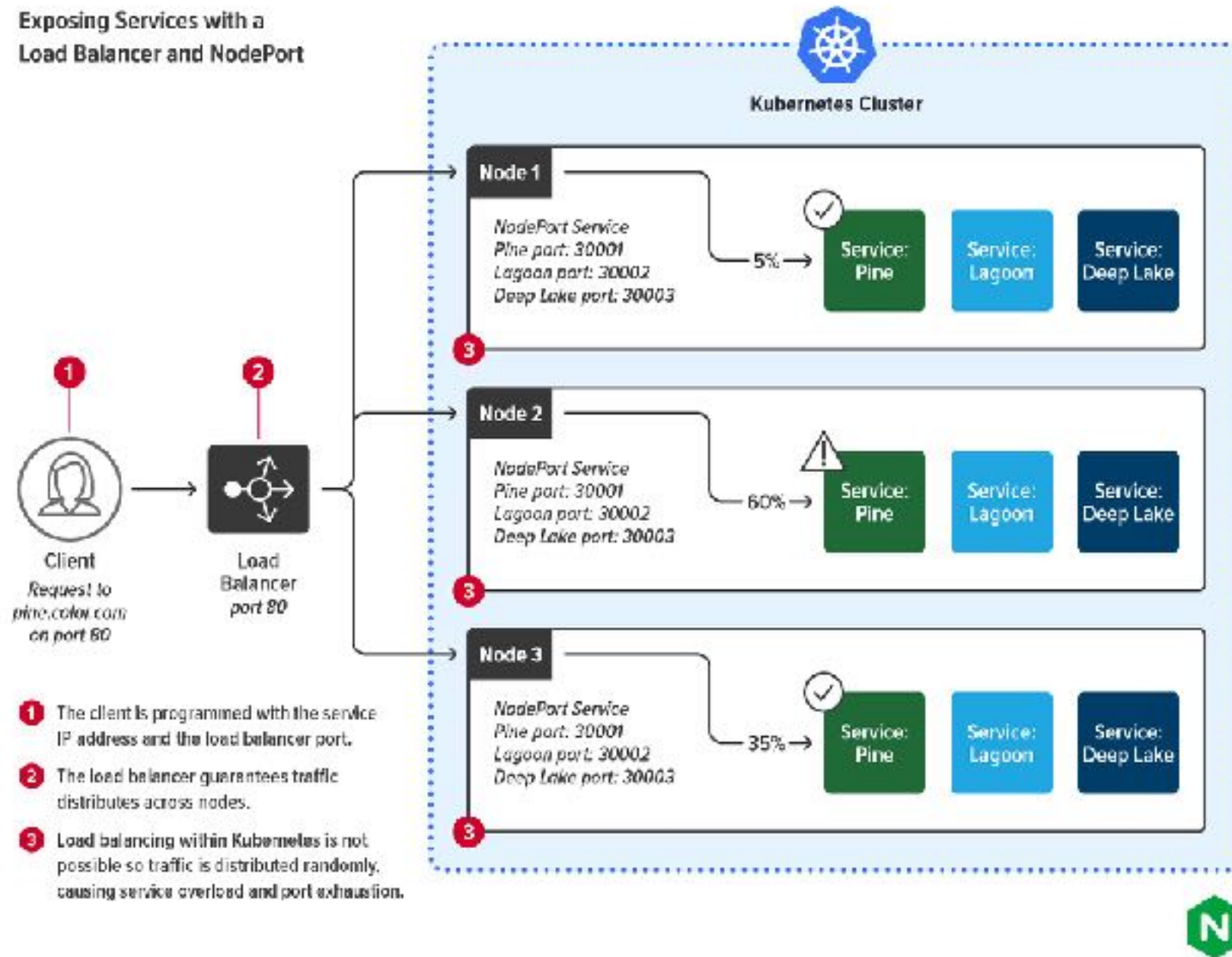
Exposes the Service externally using a cloud provider's load balancer

Good choice for small and static deployment

<https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>



Expose services with LB



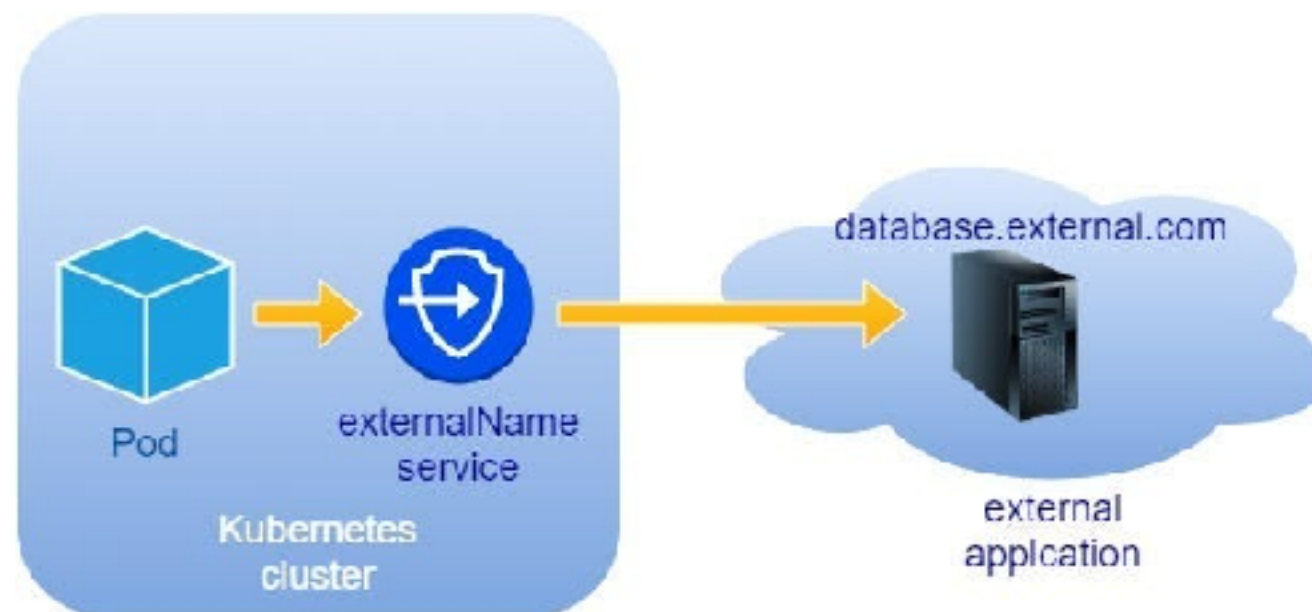
<https://www.f5.com/company/blog/nginx/kubernetes-networking-101>



ExternalName

Mapping a service to a domain name

This is commonly used to create a service within Kubernetes to represent an **external database**



<https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>



Ingress !!

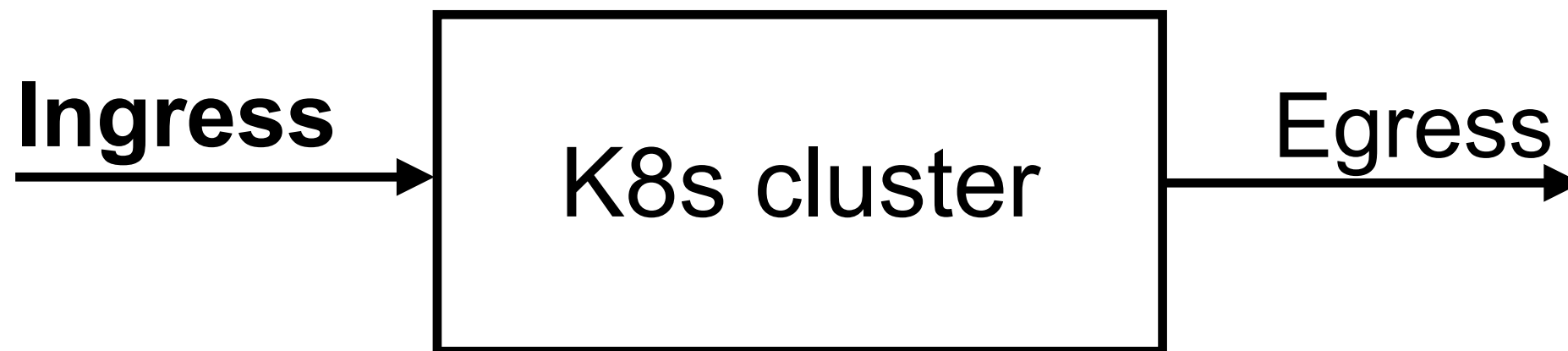
 **FEATURE STATE:** Kubernetes v1.19 [stable]

<https://kubernetes.io/docs/concepts/services-networking/ingress/>



Ingress ?

The traffic that enter to the cluster
A kind of resource in cluster

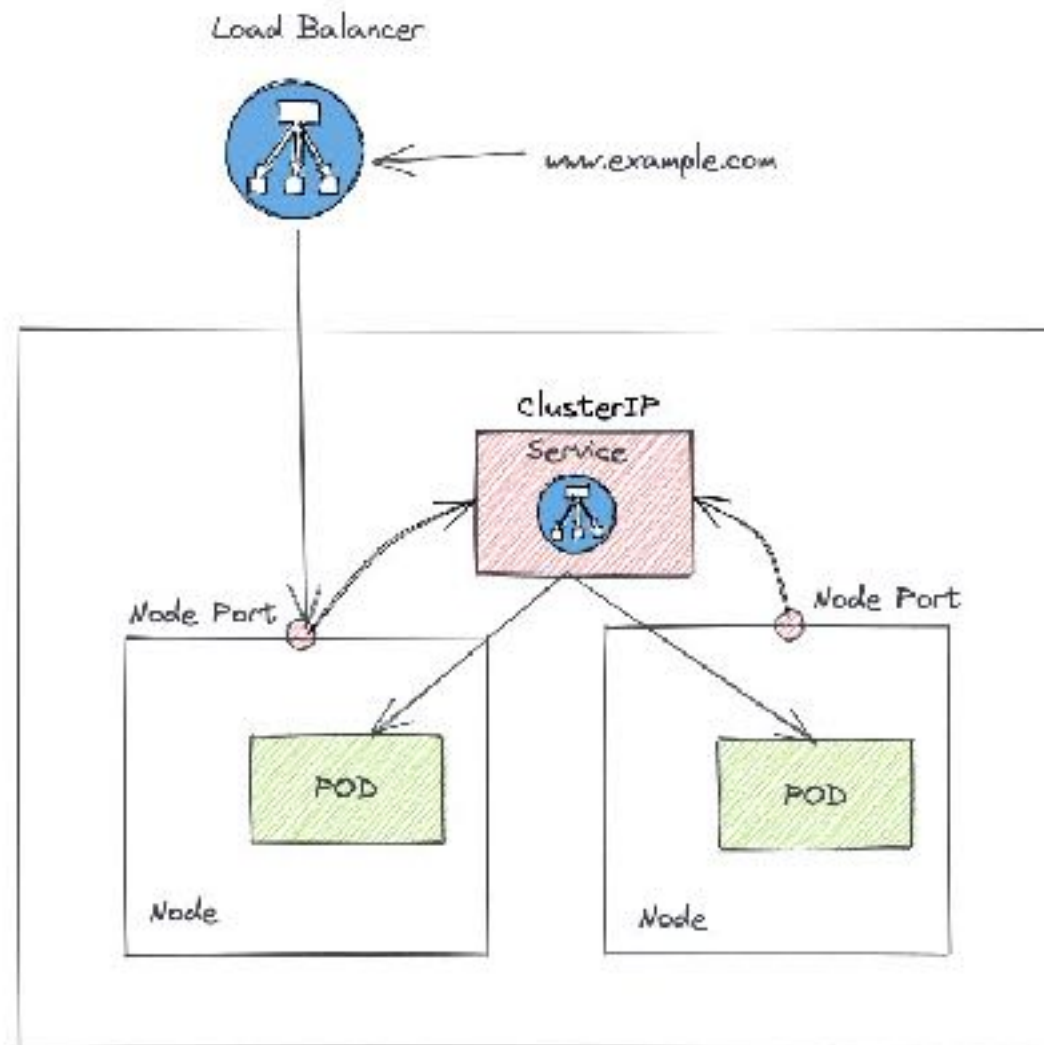


<https://kubernetes.io/docs/concepts/services-networking/ingress/>



Before Ingress ?

Use service with **NodePort** type

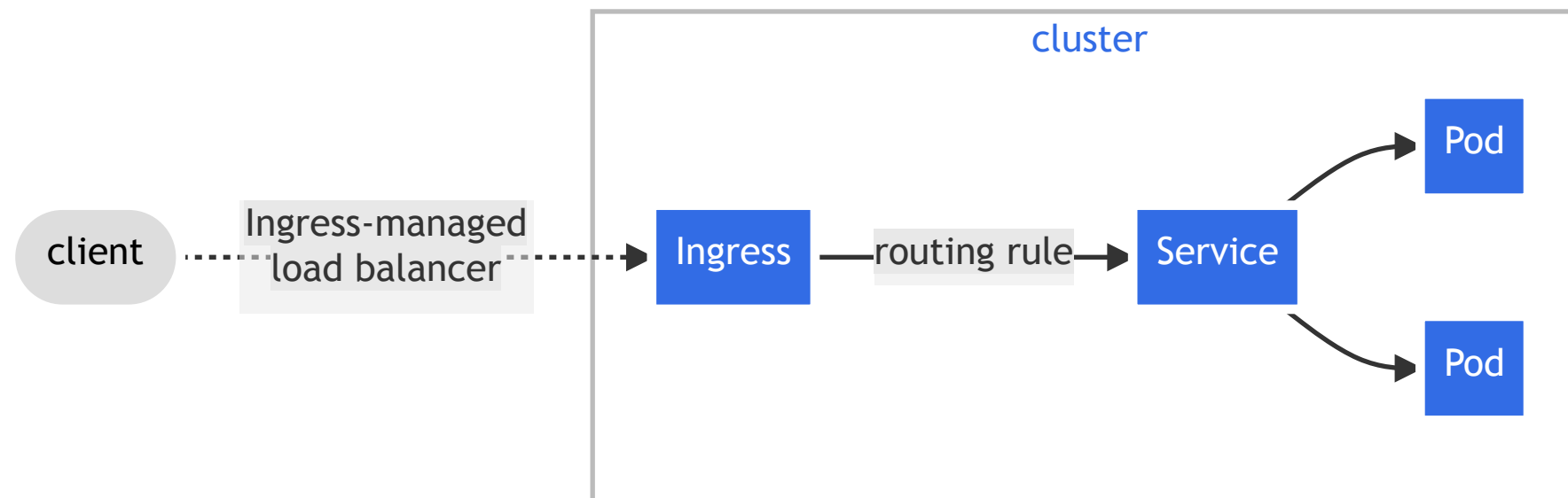


<https://devopscube.com/kubernetes-ingress-tutorial/>



Ingress ?

How to handle multiple services in same port ?
How to limit protocol to access ?



<https://kubernetes.io/docs/concepts/services-networking/ingress/>



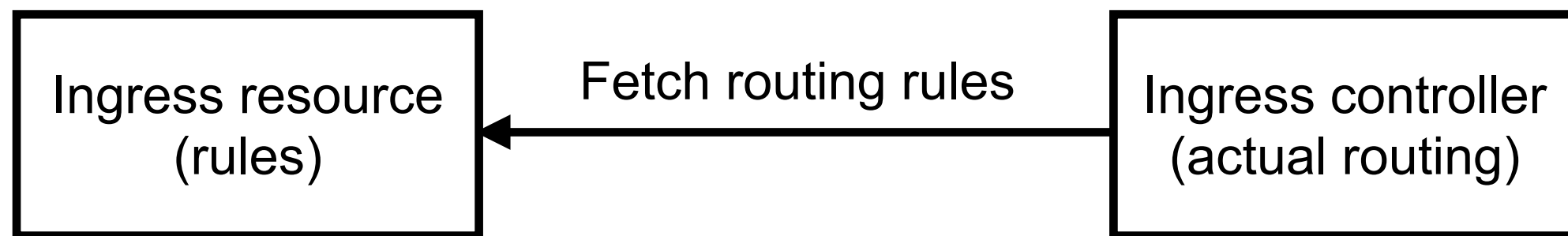
Ingress components ?

Ingress Resource

Store DNS routing rules in the cluster

Ingress Controller

Routing by accessing the DNS rules applied through resource



<https://kubernetes.io/docs/concepts/services-networking/ingress/>



Ingress Controller ?

Must be an ingress controller at least one controller

Nginx

AWS

CGE

APISIX

Alibaba

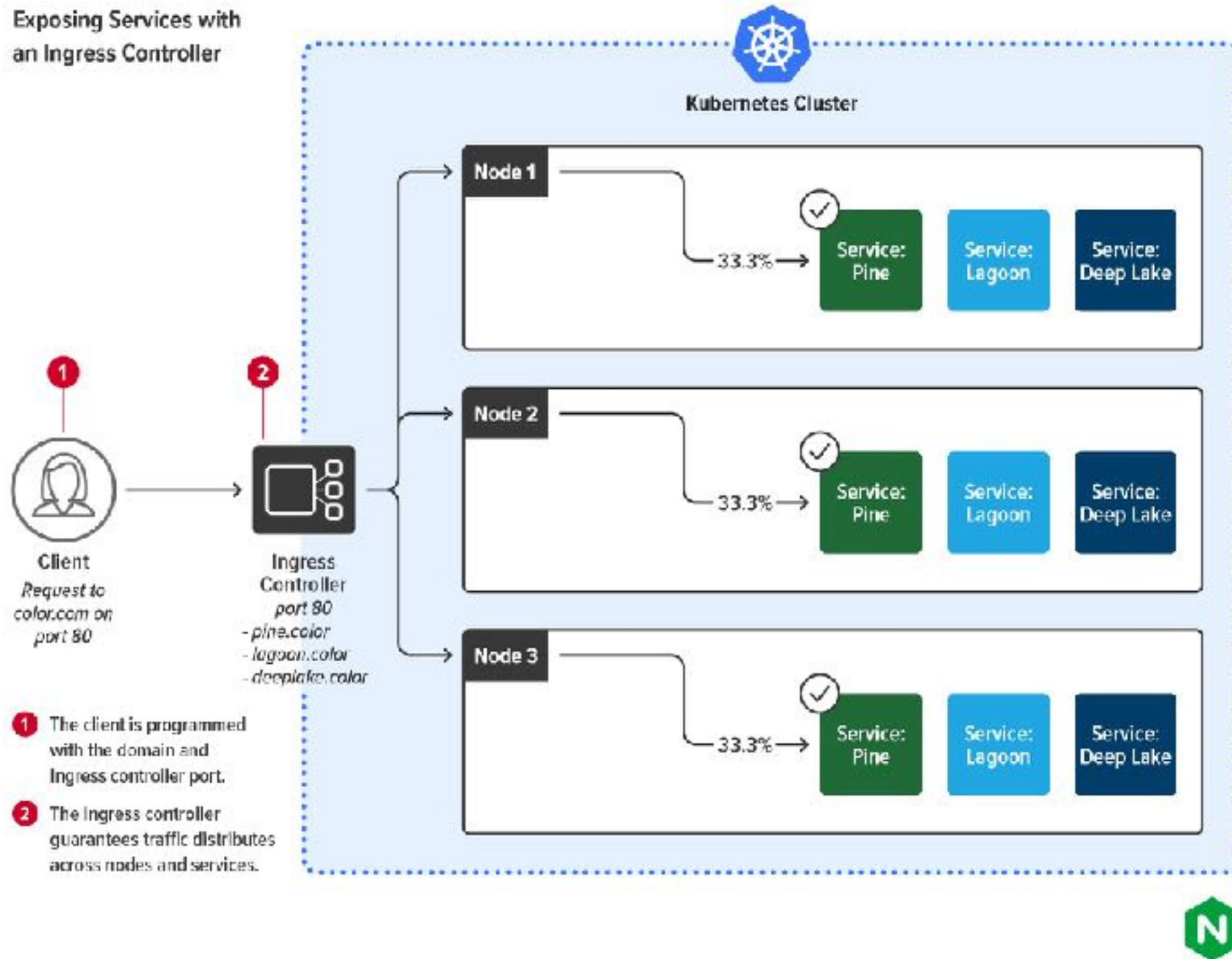
Kong

<https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>



Expose with Ingress Controller

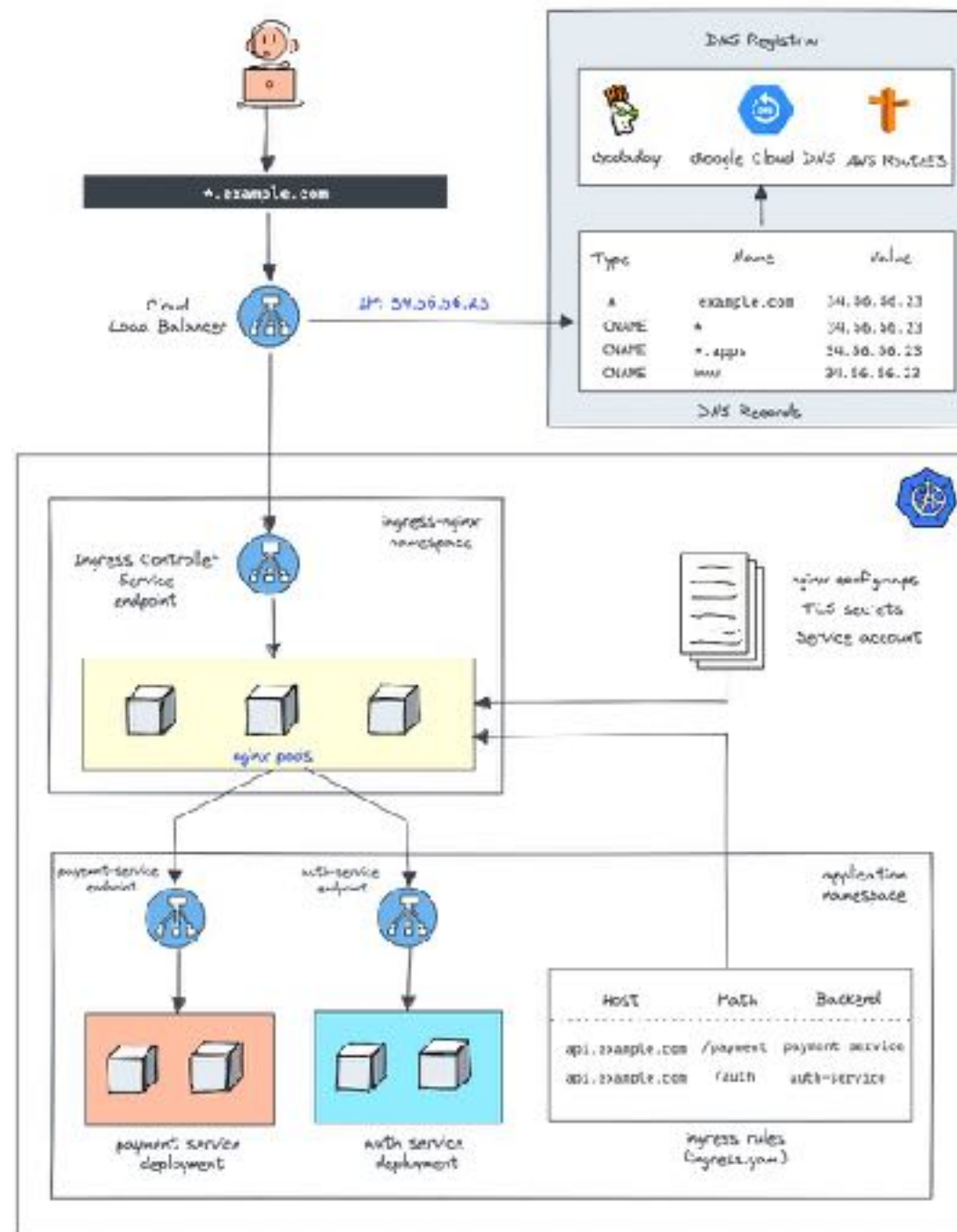
Exposing Services with an Ingress Controller



<https://www.f5.com/company/blog/nginx/kubernetes-networking-101>



Ingress and Ingress Controller



<https://devopscube.com/kubernetes-ingress-tutorial/>



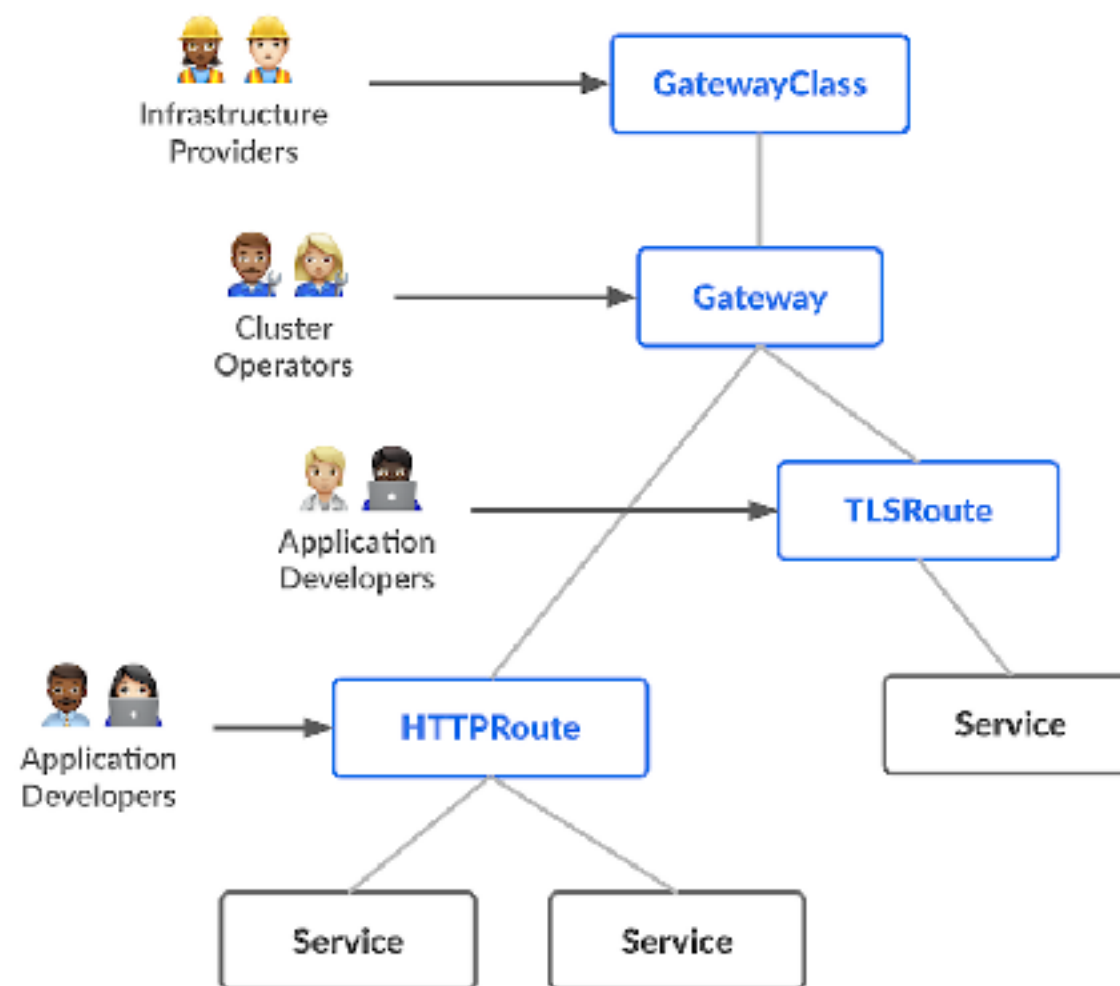
Gateway API

<https://kubernetes.io/docs/concepts/services-networking/gateway/>



Gateway API

Dynamic infrastructure provisioning
Advance traffic routing



<https://kubernetes.io/docs/concepts/services-networking/gateway/>



Gateway API

Request flow of HTTP traffic



<https://gateway-api.sigs.k8s.io/>

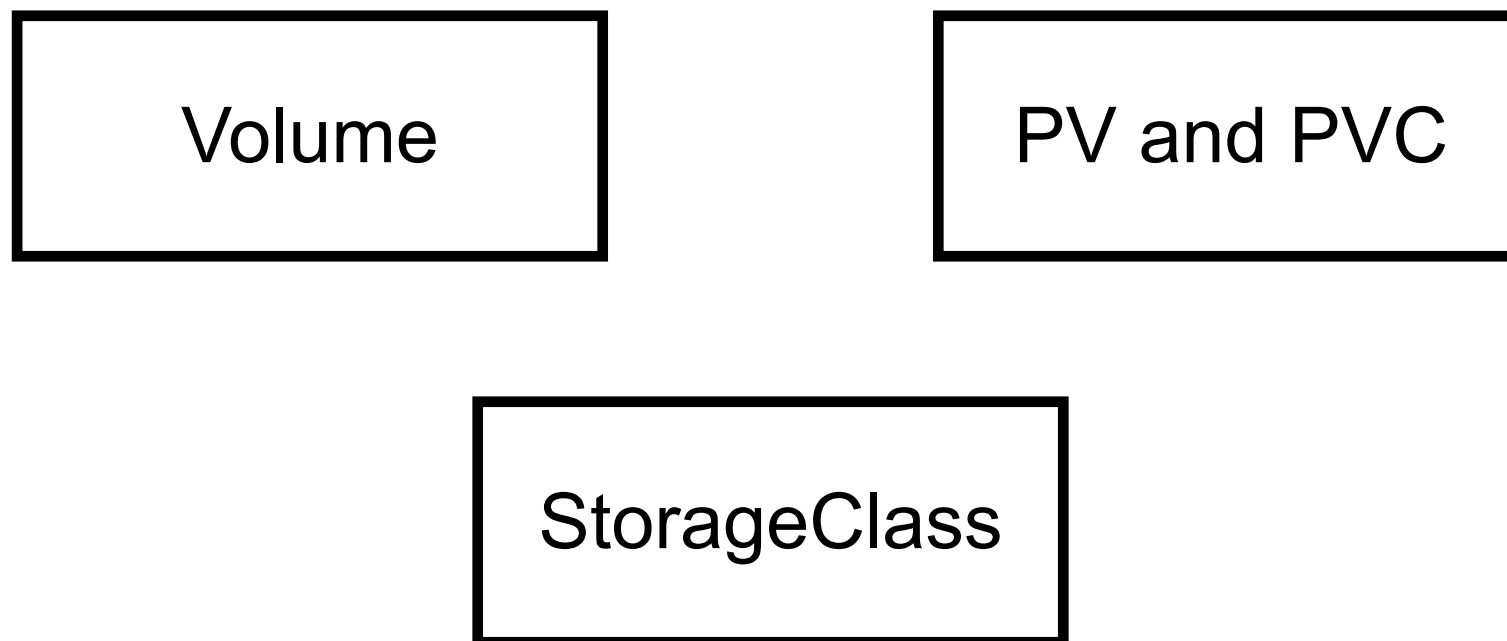


Storages



Storage in Kubernetes

Long-term and temporary storage to Pods in cluster
By default, each container has **ephemeral storage**



<https://kubernetes.io/docs/concepts/storage/>



Storage in Kubernetes

Volume Type	Description	Pod-Level	Persistent
emptyDir	Temporary directory on node	Yes	No
hostPath	Directory on node	Yes	Yes
awsElasticBlockStore / gcePersistentDisk / azureDisk / azureFile	Cloud provider-specific storage	No	Yes
nfs	Network File System share	No	Yes
persistentVolumeClaim	Use a Persistent Volume	No	Yes
configMap / secret / downwardAPI	Expose pod info and cluster info	Yes	No
csi	Use third-party storage providers	No	Depends on the specific CSI driver

<https://kubernetes.io/docs/concepts/storage/>



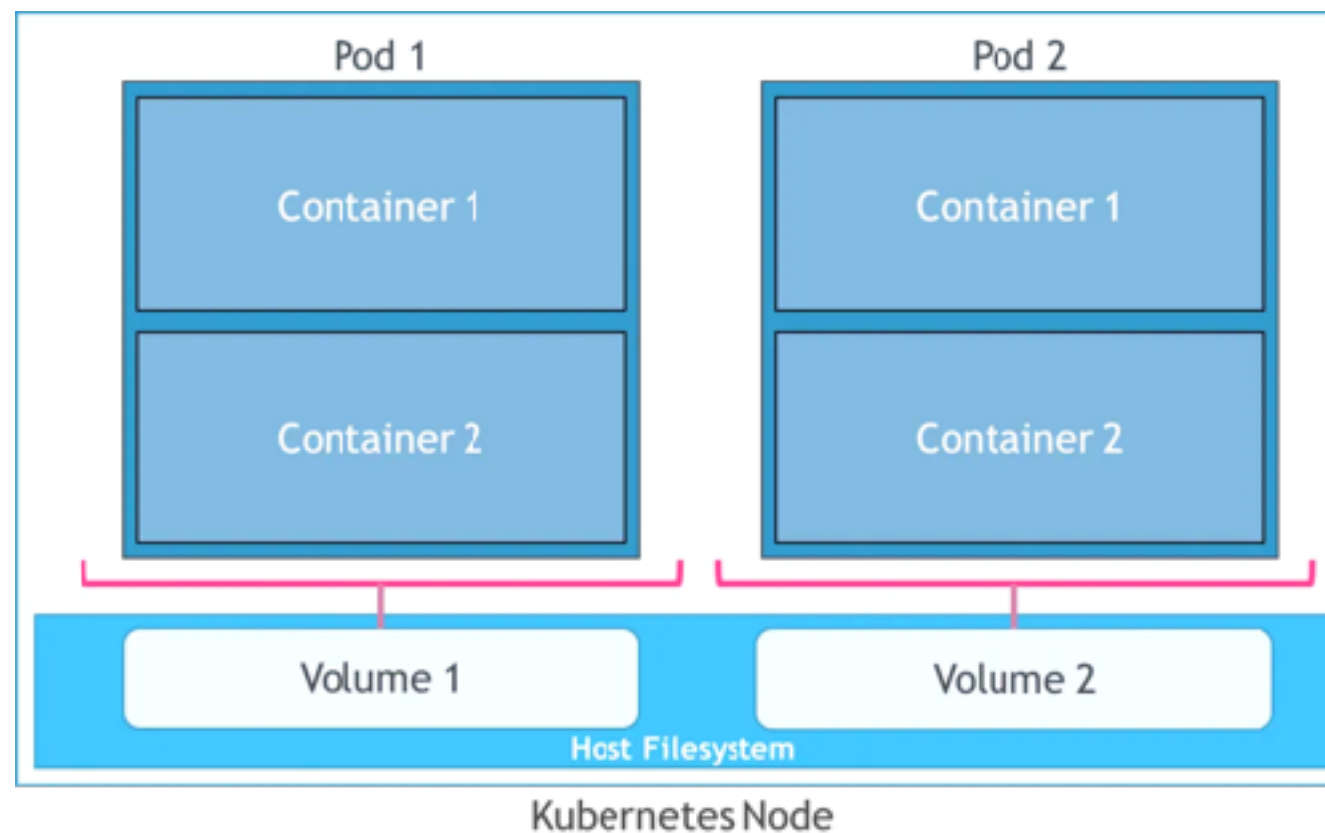
Volume

Storage unit associated with Pod

Shared data between containers in the same Pod

eg. EmptyDir, HostPath

Support all types of storages (local, network storage)



PV and PVC

Persistent volume

Storage unit in the cluster, provisioning by admin

Persistent volume

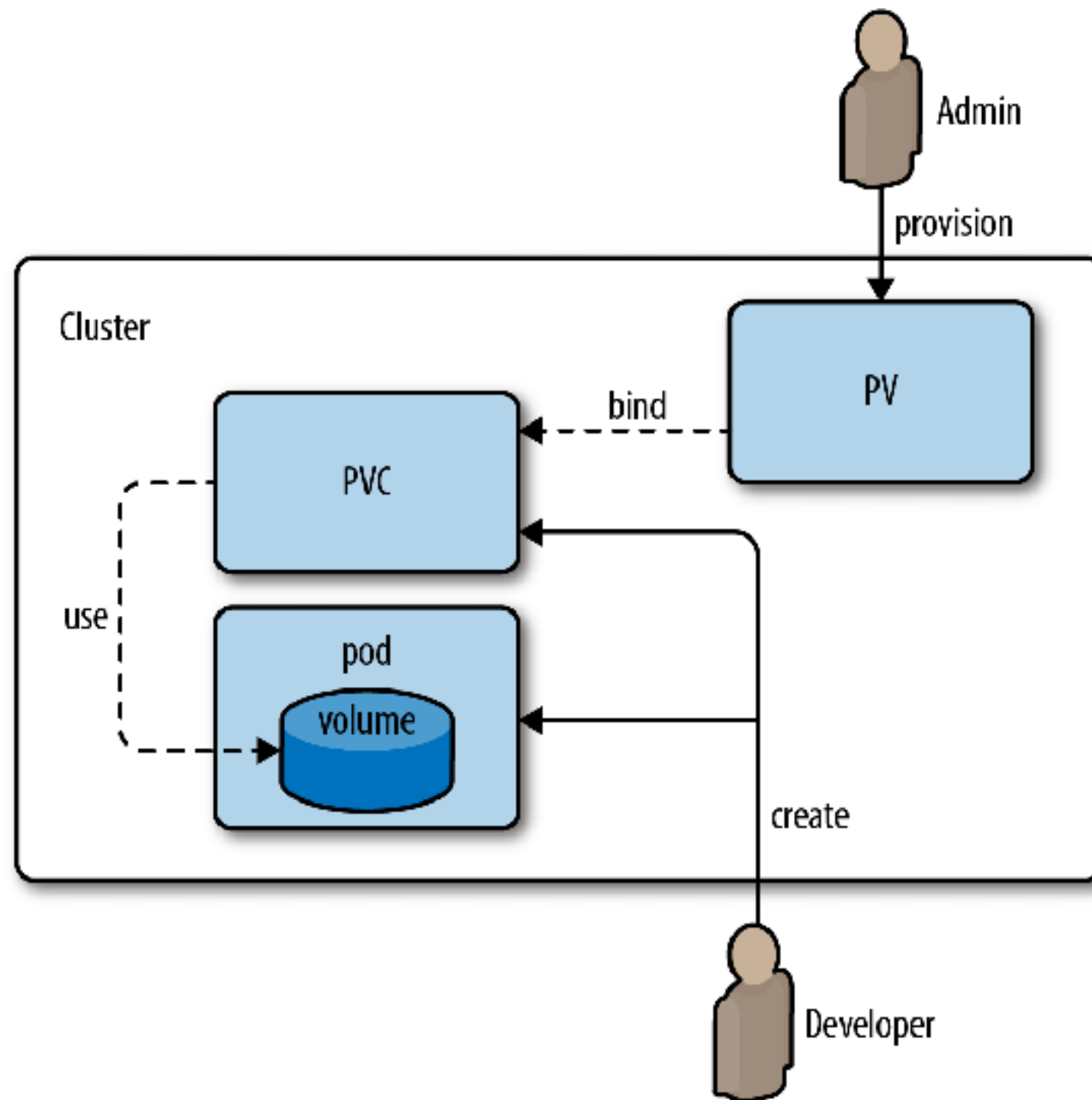
User's request for storage

PVC utilize PV resources

Provide more robust solution for long-term data storage



PV and PVC



StorageClass in Kubernetes

Dynamic volume provisioning

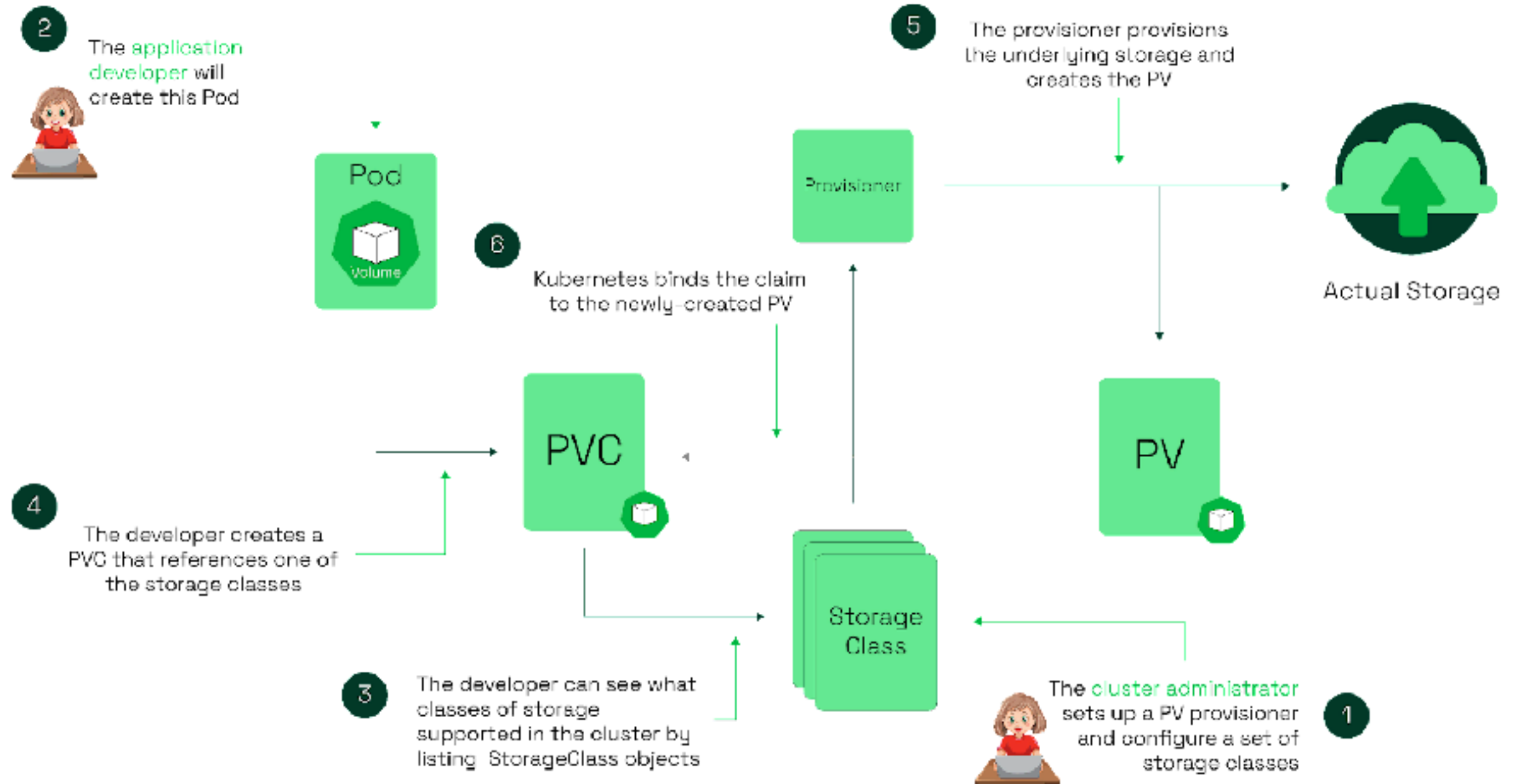
Define **classes** or **tiers** of storage

Volumes are provisioned with classes by user's need

<https://kubernetes.io/docs/concepts/storage/storage-classes/>



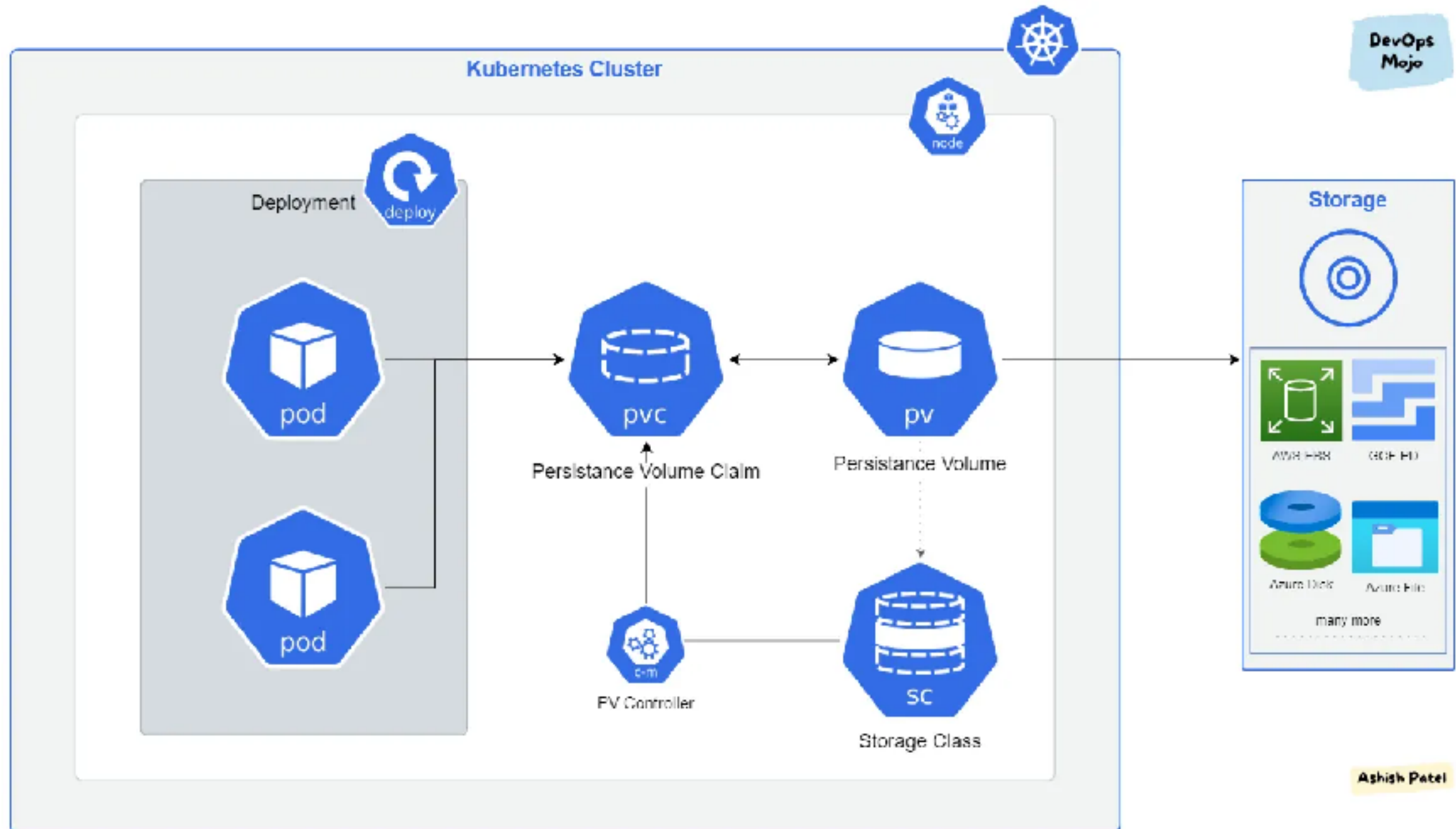
StorageClass in Kubernetes



<https://www.apptio.com/topics/kubernetes/best-practices/storage-class/>



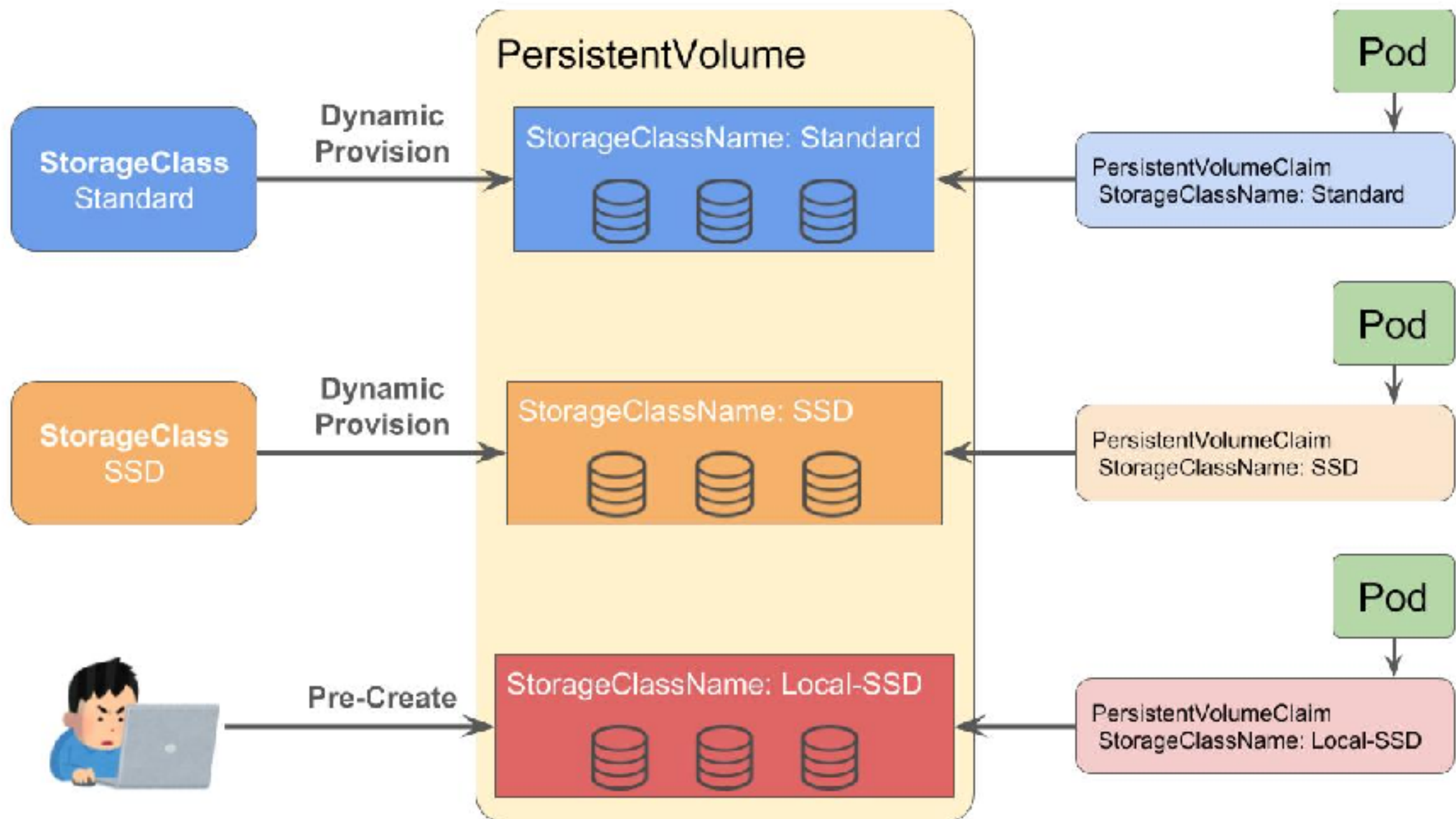
StorageClass in Kubernetes



<https://medium.com/devops-mojo>



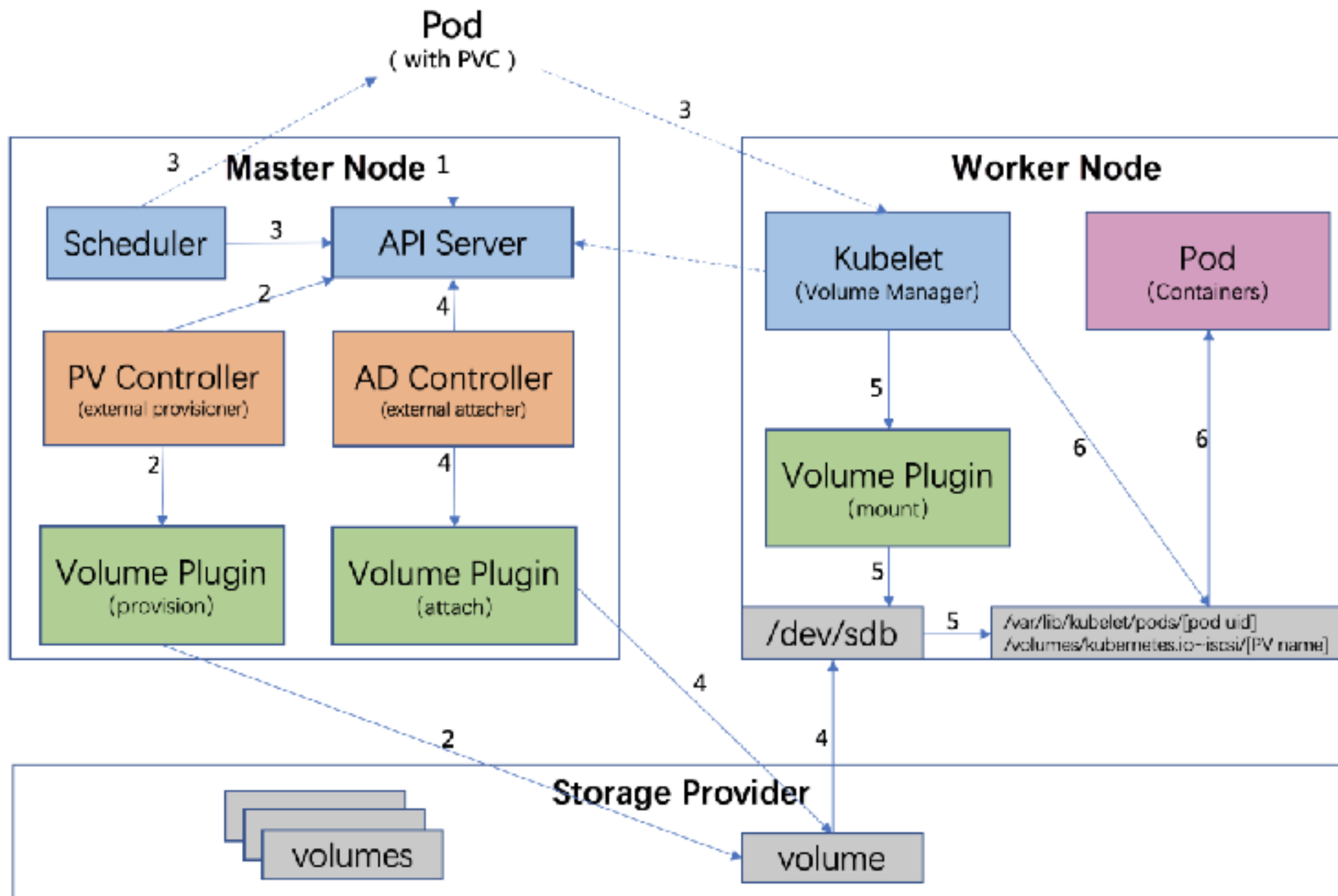
StorageClass in Kubernetes



<https://medium.com/devops-mojo>



Processes of Persistent Storage



https://www.alibabacloud.com/blog/kubernetes-persistent-storage-process_596505



Components ?

PV Controller

Binding PVs and PVCs and manage life cycle

Provision and delete operations on data volumes

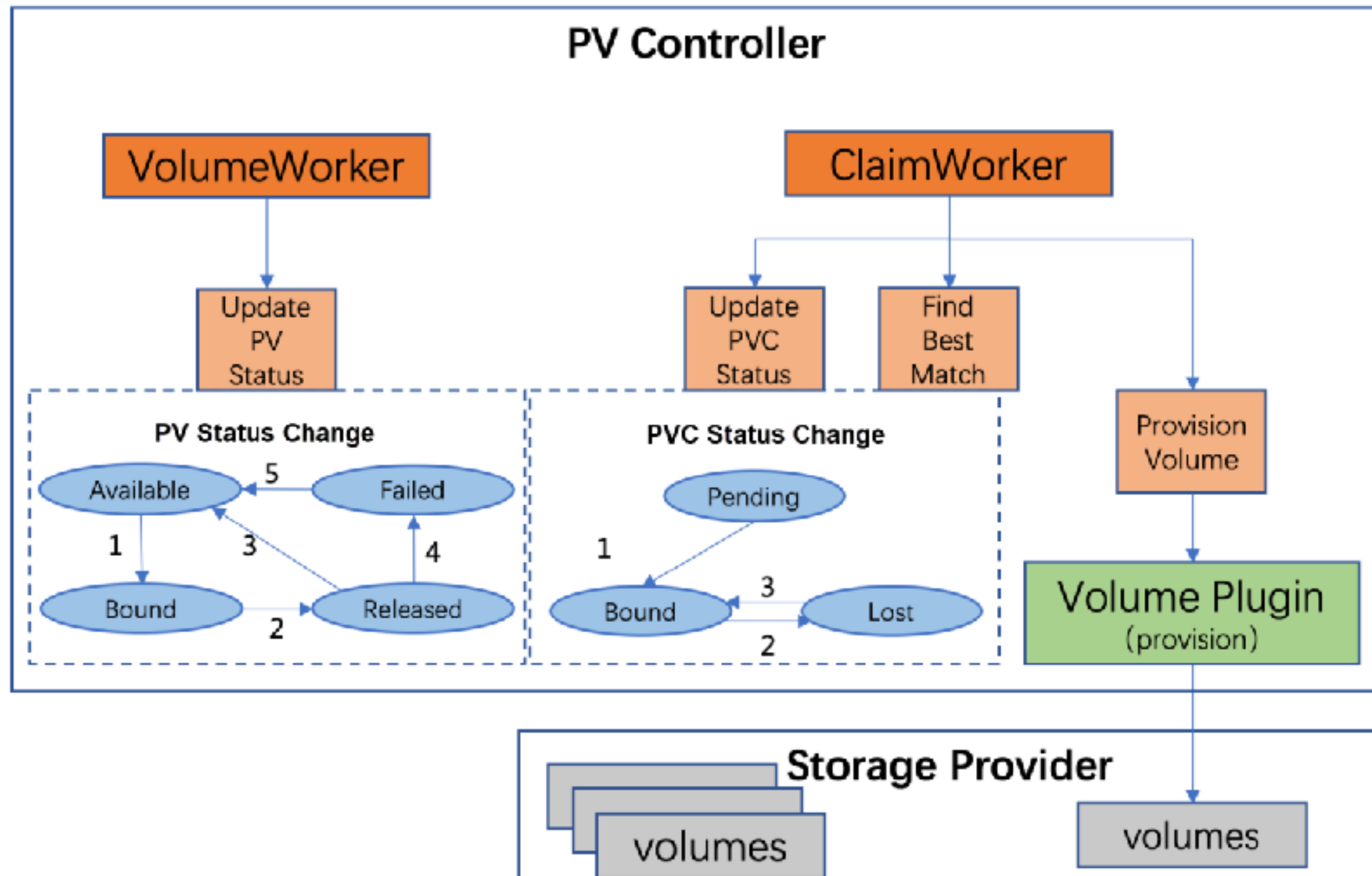
AD Controller

Attach and detach operations on data volumes

Attach devices to target nodes



PV Controller



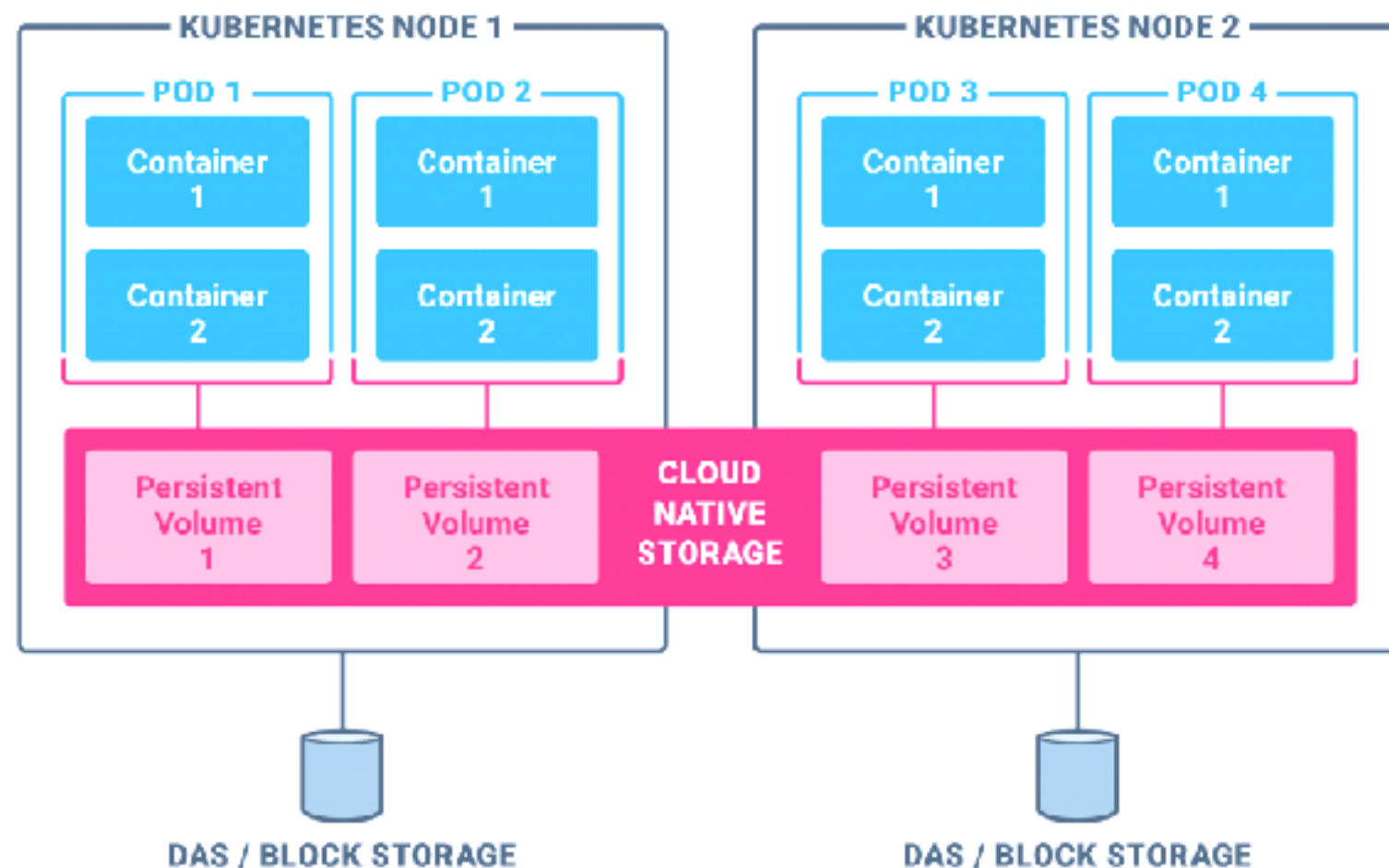
https://www.alibabacloud.com/blog/kubernetes-persistent-storage-process_596505



Overlay storage in Kubernetes

Working with Container Storage Interface (CSI)

Overlay Storage of Kubernetes



Source: Janakiram MSV

© 2025 THE NEW STACK

<https://thenewstack.io/how-kubernetes-provides-networking-and-storage-to-applications/>



Opensource Storage solutions ?



Rook



GlusterFS



Longhorn



StatefulSet



StatefulSet

Bringing the concept of ReplicaSets to **stateful** Pods

Enable running Pods in **cluster** mode

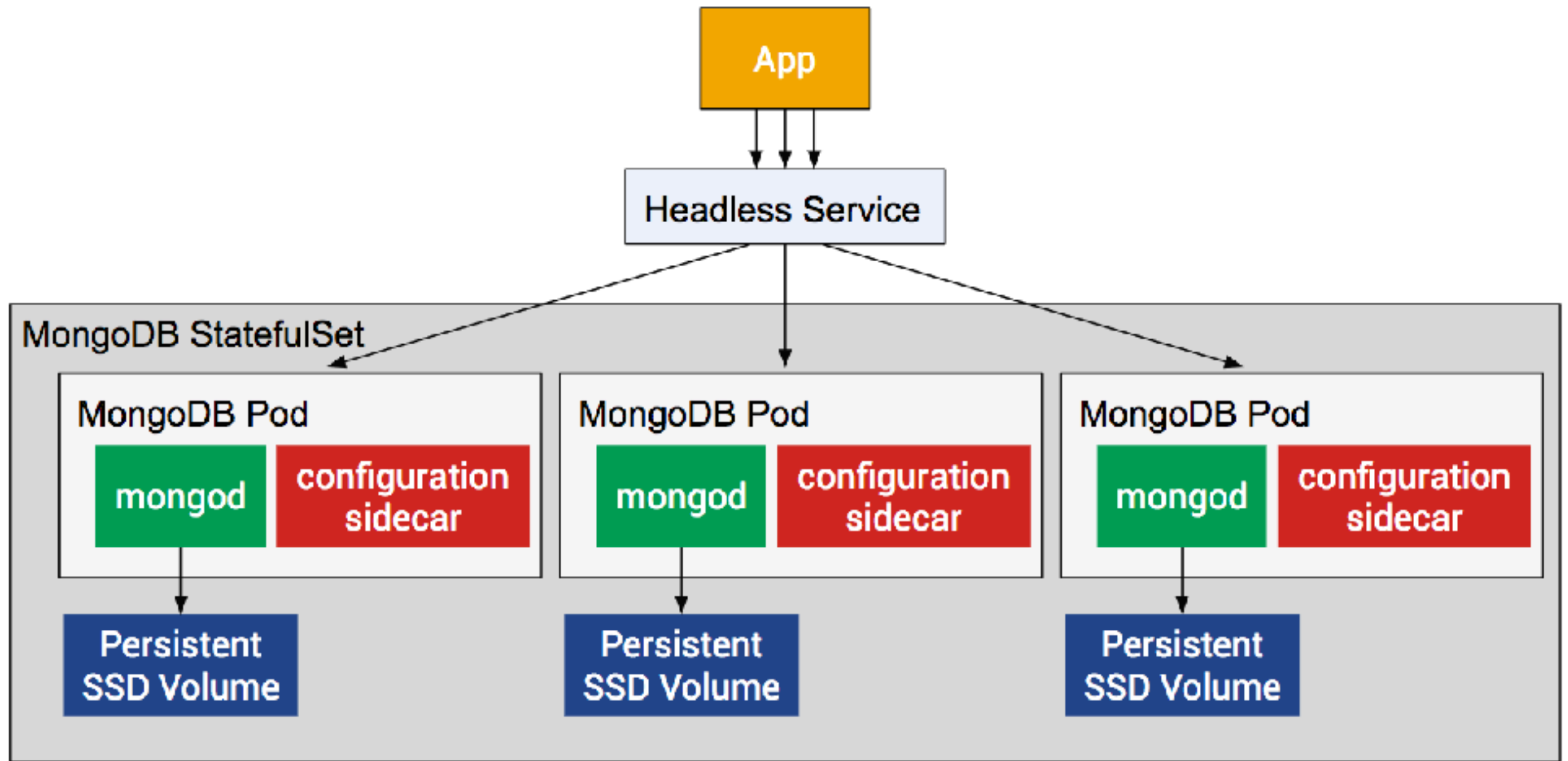
Ideal for deploy **highly** available database **workload**

Pods are created **sequentially**

Pods are terminated in **LiFo** (Last in, First out)



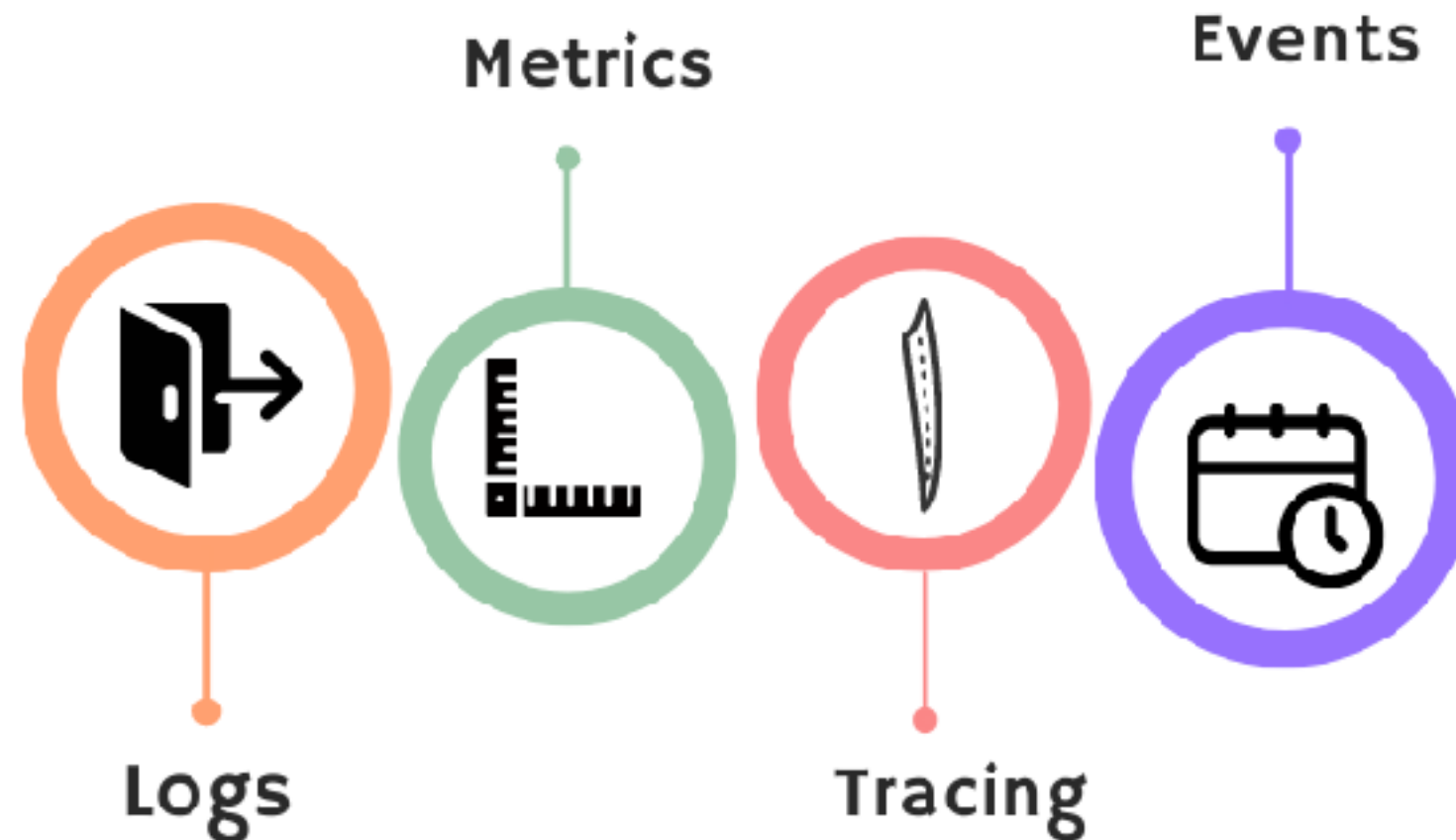
StatefulSet



Observability Monitoring



THE FOUR PILLARS OF API OBSERVABILITY



{APIToolkit.io}

<https://apitoolkit.io/blog/the-four-pillars-of-api-observability/>



Reasons to monitoring ?

Realtime alerts and early error detection
Better workload management and optimization
Easily troubleshooting
Realtime cost visibility
Powerful insight

<https://grafana.com/solutions/kubernetes/kubernetes-monitoring-introduction/>



Kubernetes monitoring ?

Node
view

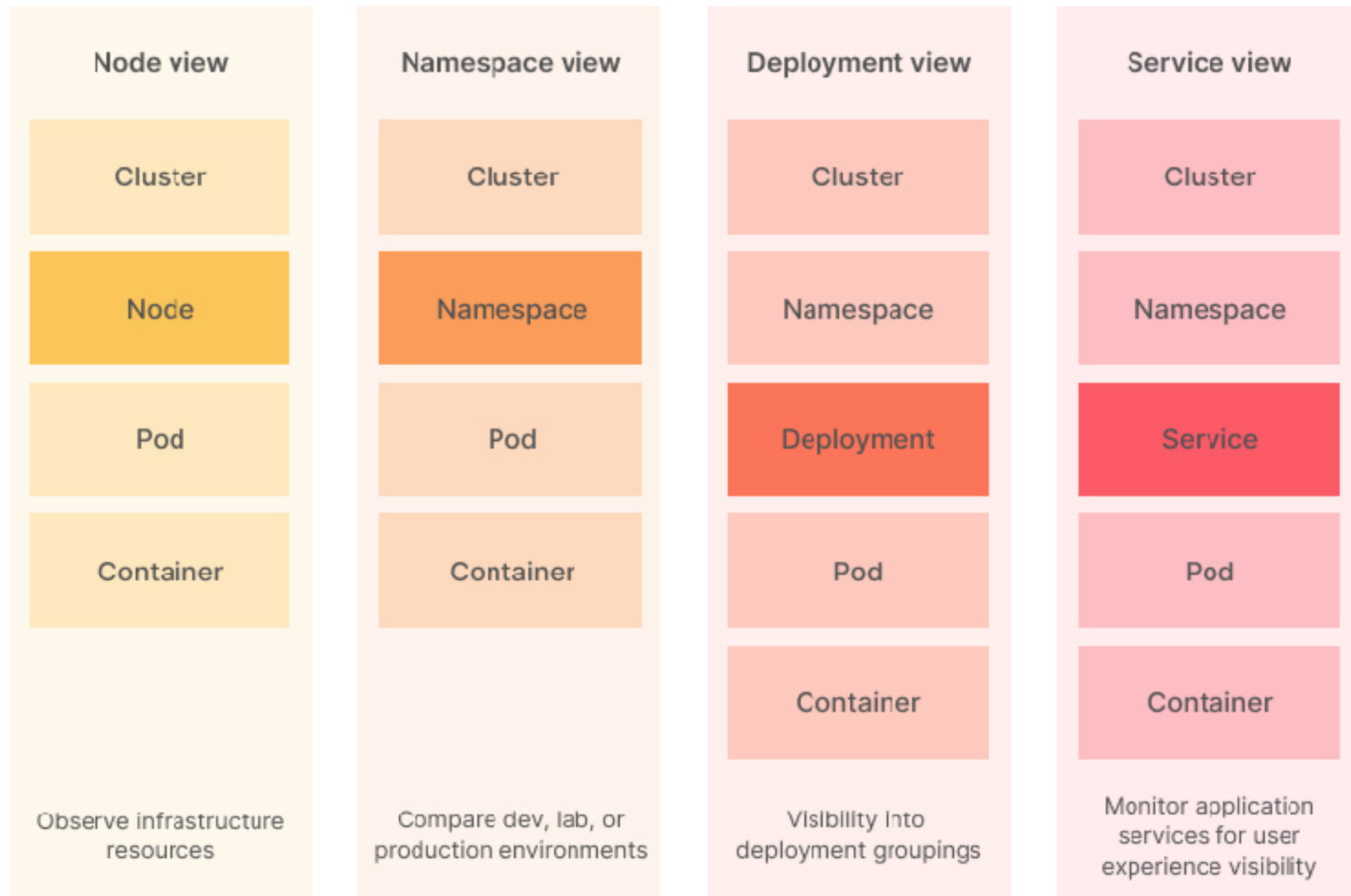
Namespace
view

Deployment
view

Service
view



Kubernetes monitoring ?



<https://grafana.com/solutions/kubernetes/kubernetes-monitoring-introduction/>



Performance metrics ?

Node resources
utilization

Object capacity

Health metrics

Storage health

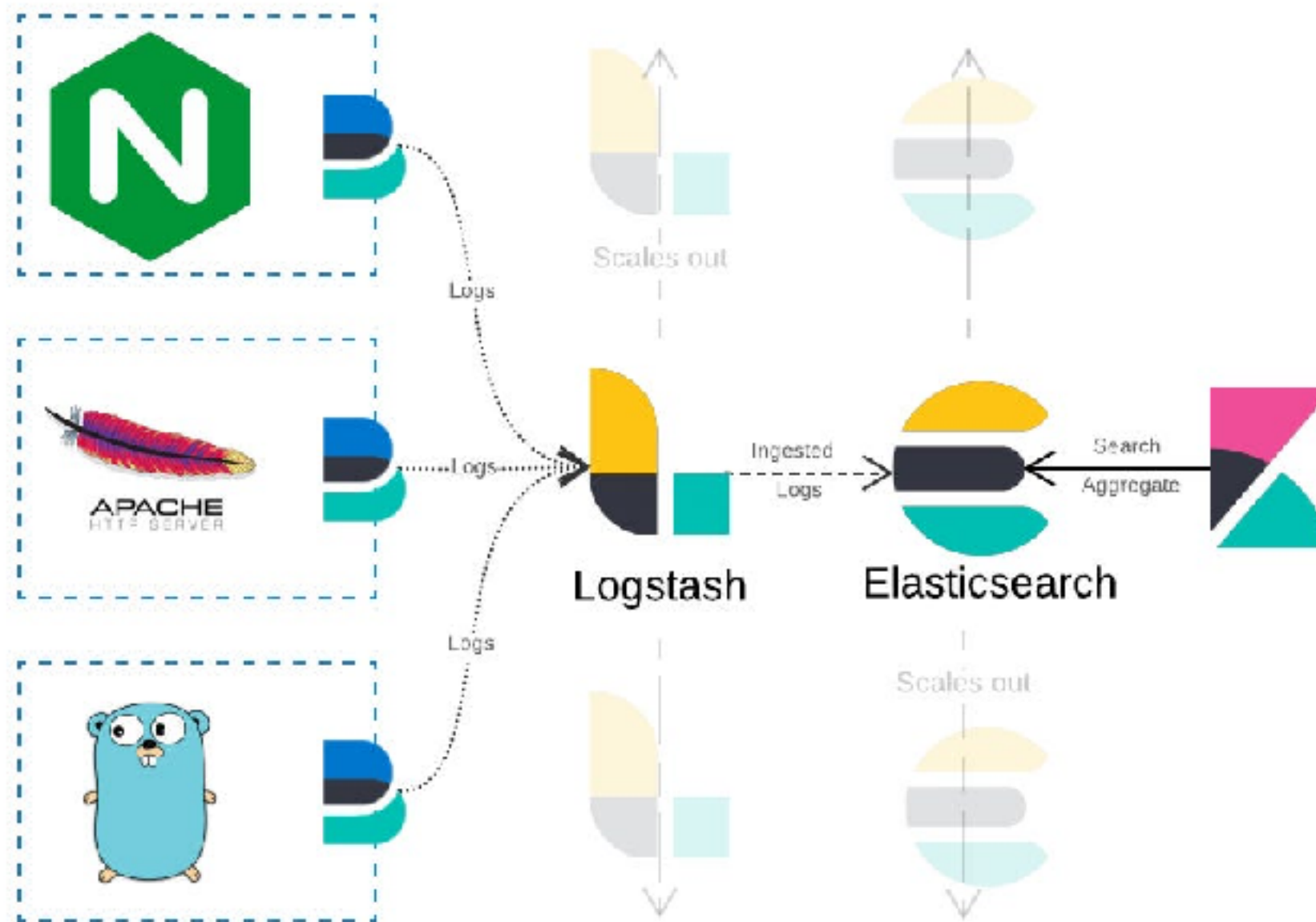
Kubelet
metrics



Technology Stacks ?



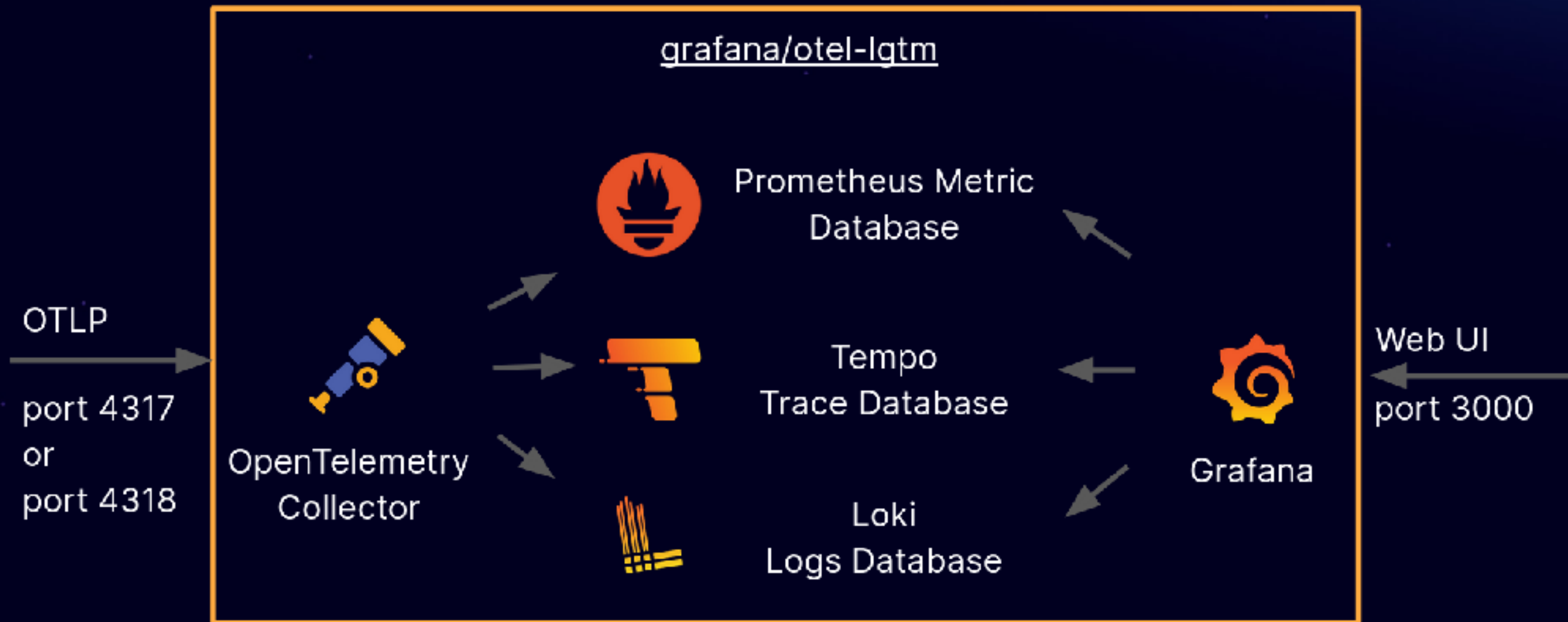
ELK Stack



<https://www.elastic.co/elastic-stack>



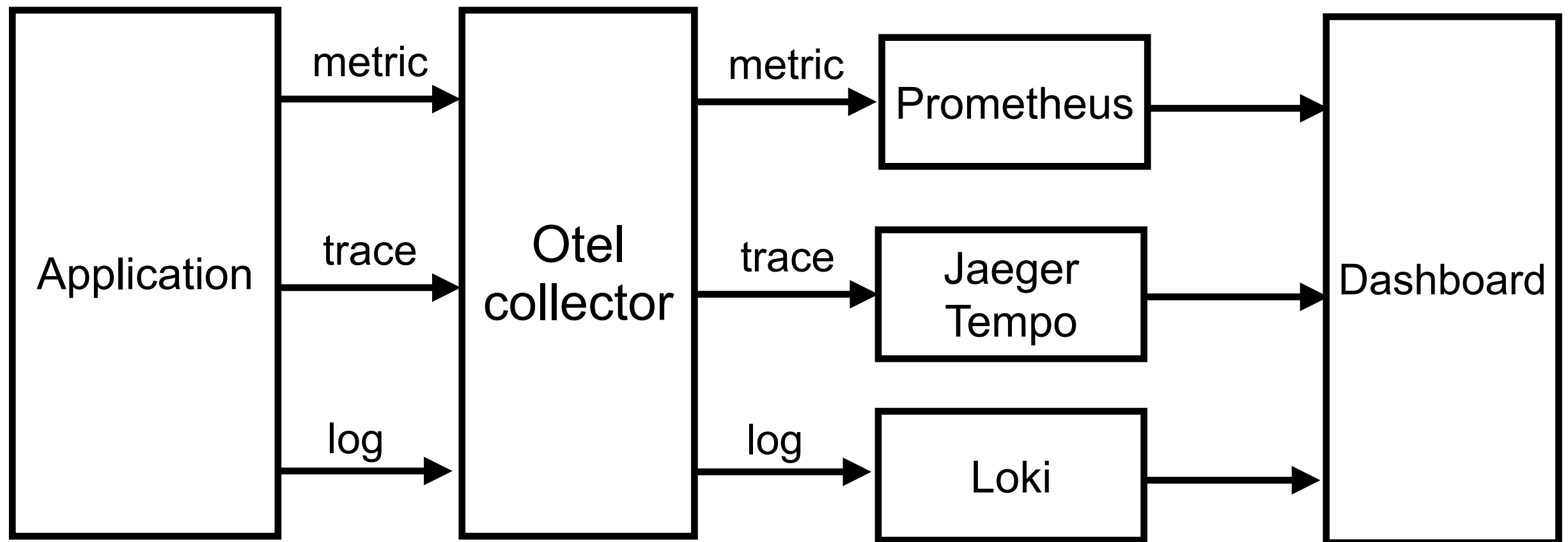
LGTM Stack



<https://github.com/grafana/docker-otel-lgtm/>



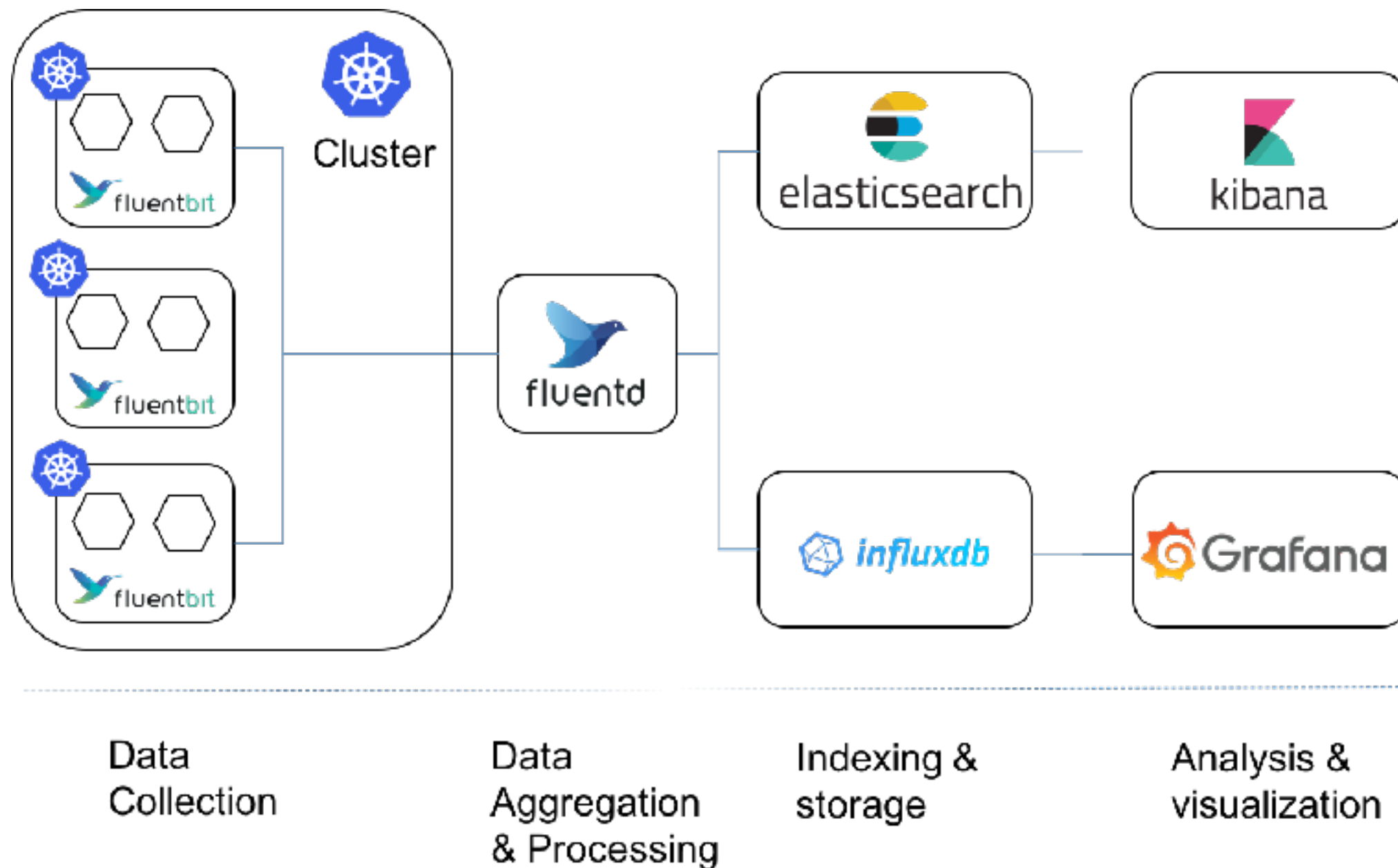
OpenTelemetry



<https://github.com/up1/workshop-lgtm-go>



Fluent and FluentBit



<https://dzone.com/articles/fluentd-vs-fluent-bit-side-by-side-comparison-logz>



Kubernetes Operations

Cluster
management

Application
management

Security

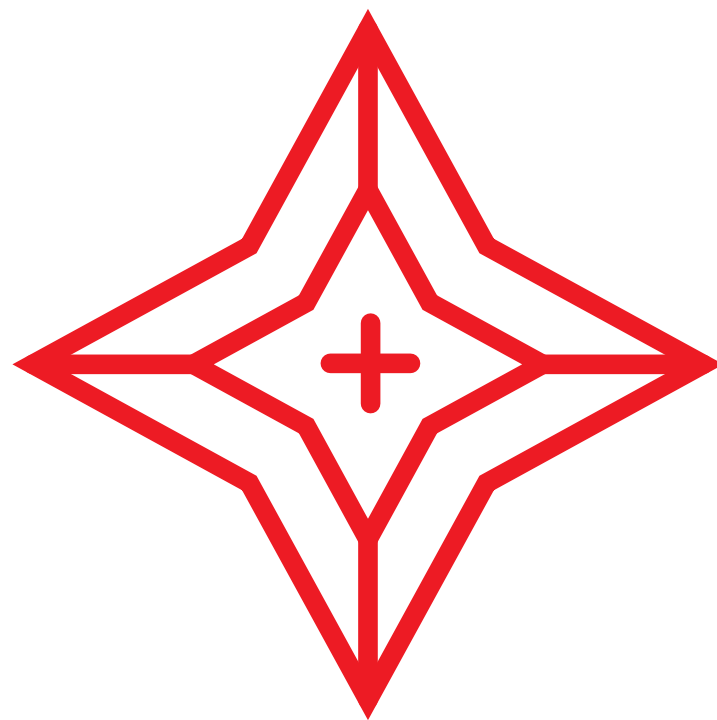


Steps to upgrade your cluster ?

<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>



Check cluster use deprecated API ?



Kubent

<https://github.com/doitintl/kube-no-trouble>



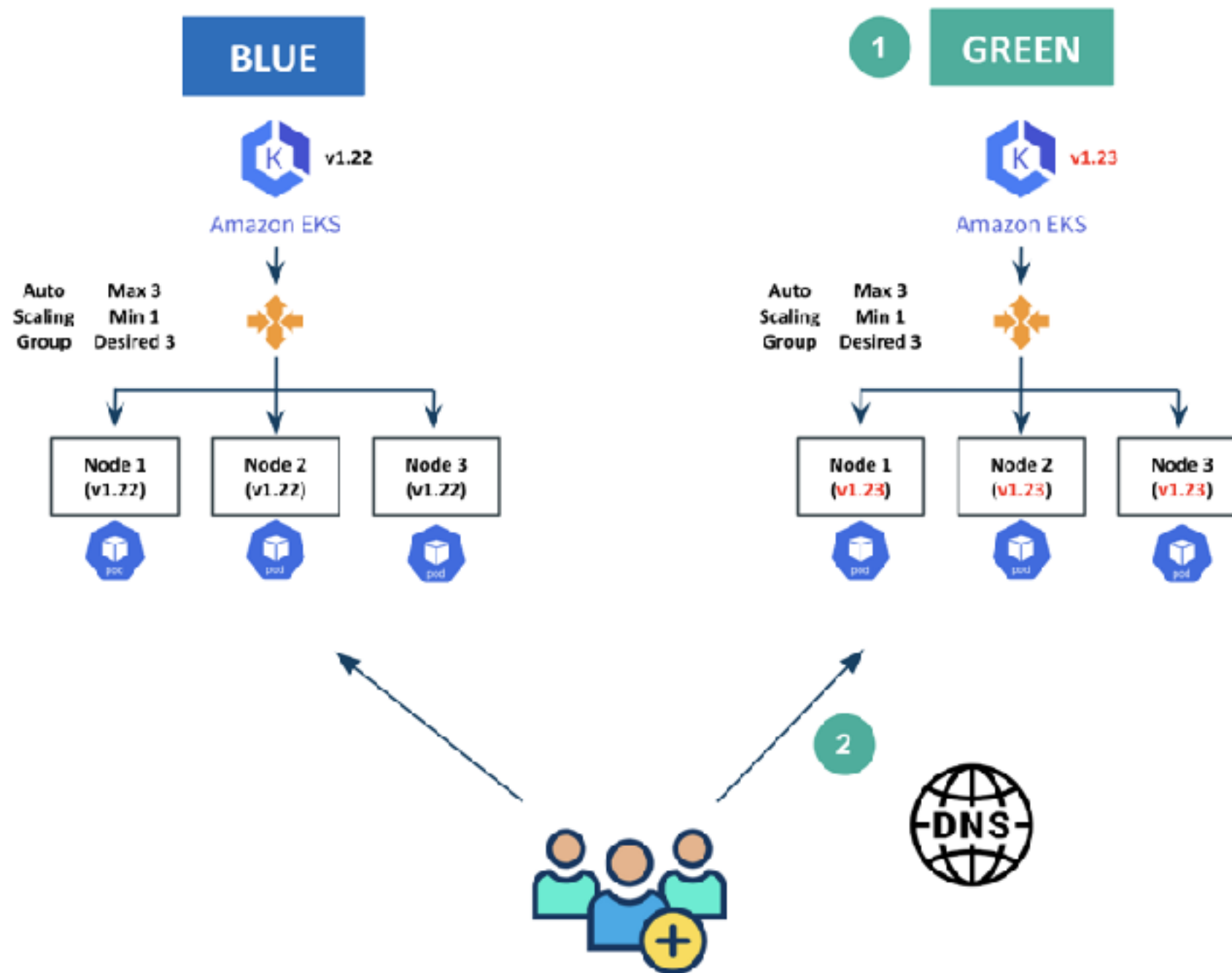
Upgrade cluster strategies ?

Rolling update

Blue/Green



Blue/Green update ?



<https://rafay.co/ai-and-cloud-native-blog/mastering-amazon-eks-upgrades-with-rafays-kubernetes-operations-platform/>



Try to upgrade in Testing environment !!



Troubleshooting Cluster !!

Cluster level

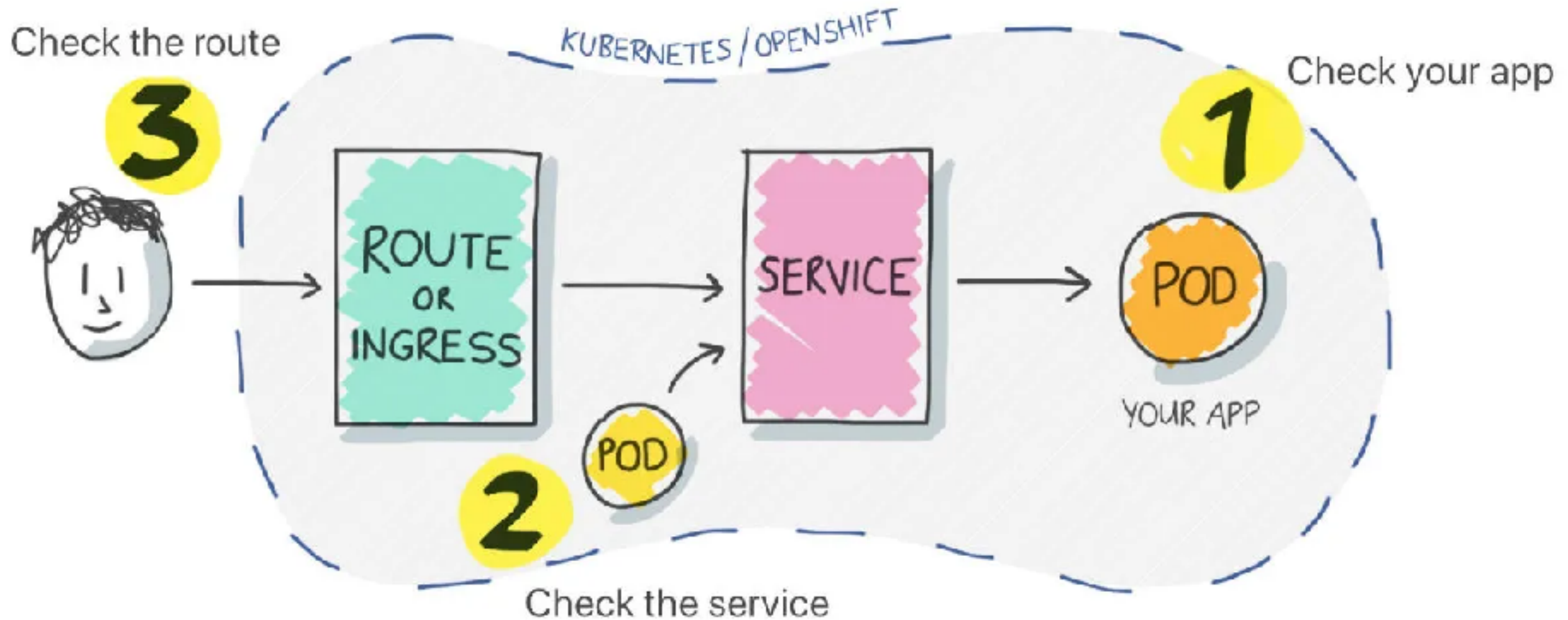
Application level

<https://kubernetes.io/docs/tasks/debug/debug-cluster/>





TROUBLESHOOTING APPS IN KUBERNETES

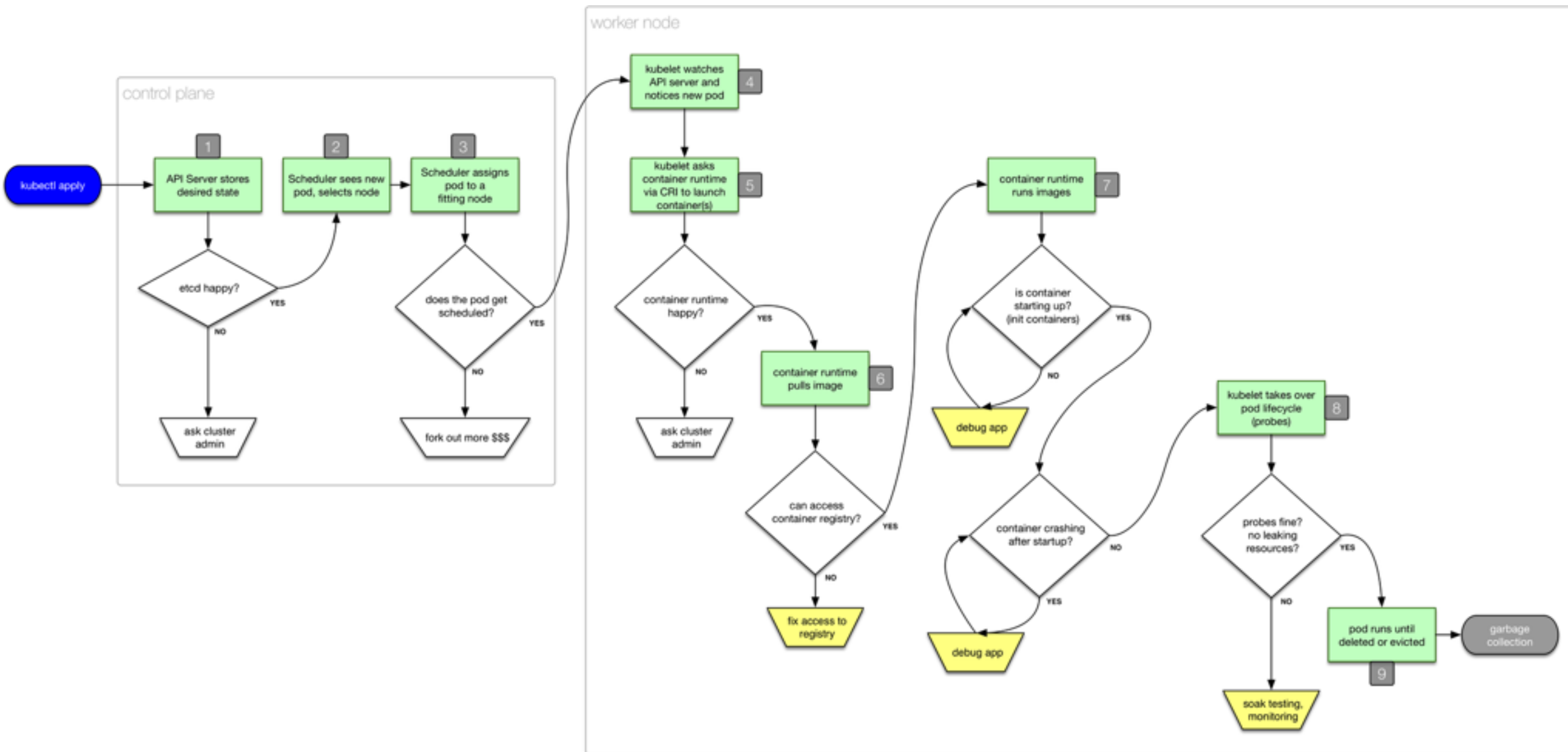


- 1 Check your *Pods*
- 2 Check the *Service*
- 3 Check the *Route or Ingress*

<https://www.tutorialworks.com/kubernetes-troubleshooting/>



Pods's Lifecycle ?



<http://troubleshooting.kubernetes.sh/>



Tools



KOps

Installation cluster
Upgrade and manag cluster



<https://github.com/kubernetes/kops>



K8sGPT



Giving Kubernetes Superpowers to Everyone

K8sGPT is an AI-powered tool that helps diagnose and fix Kubernetes issues with intelligent insights and automated troubleshooting.

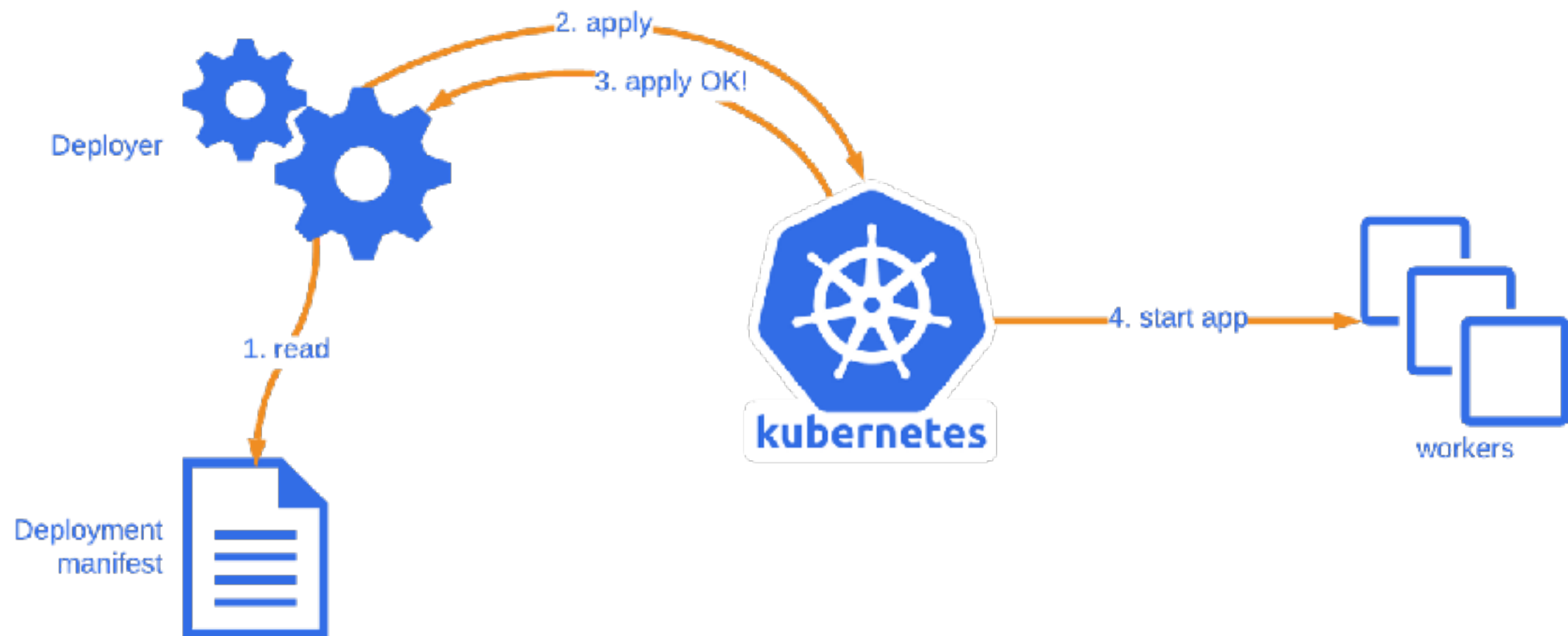
<https://k8sgpt.ai/>



Kubernetes best practices ?

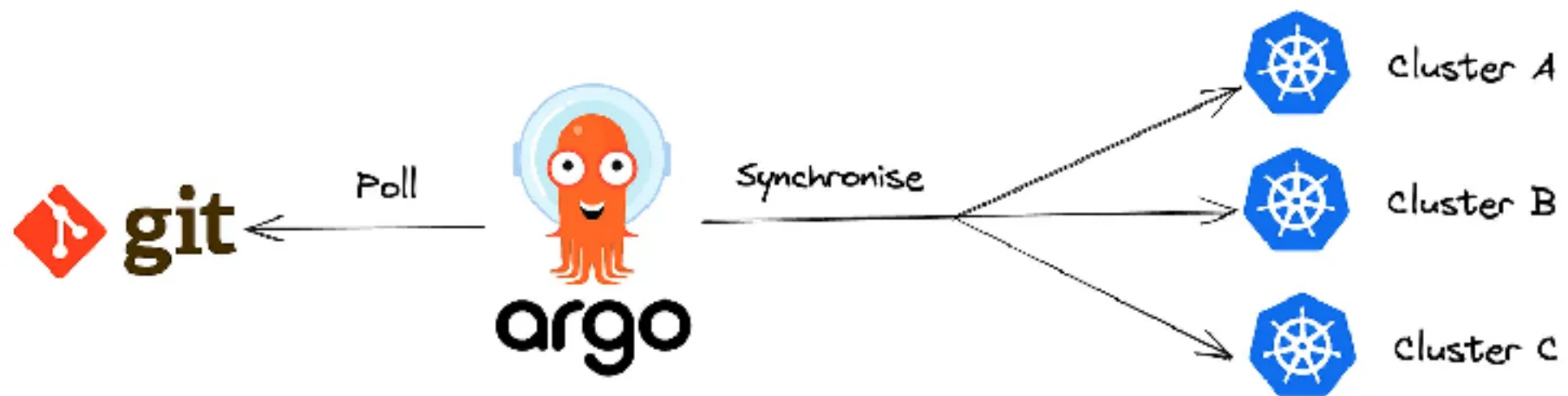


Simple deploy with kubectl !!



Use GitOps approach

Use git repository as **the source of truth** for defining the desired application state



<https://redhat-scholars.github.io/argocd-tutorial/argocd-tutorial/index.html>

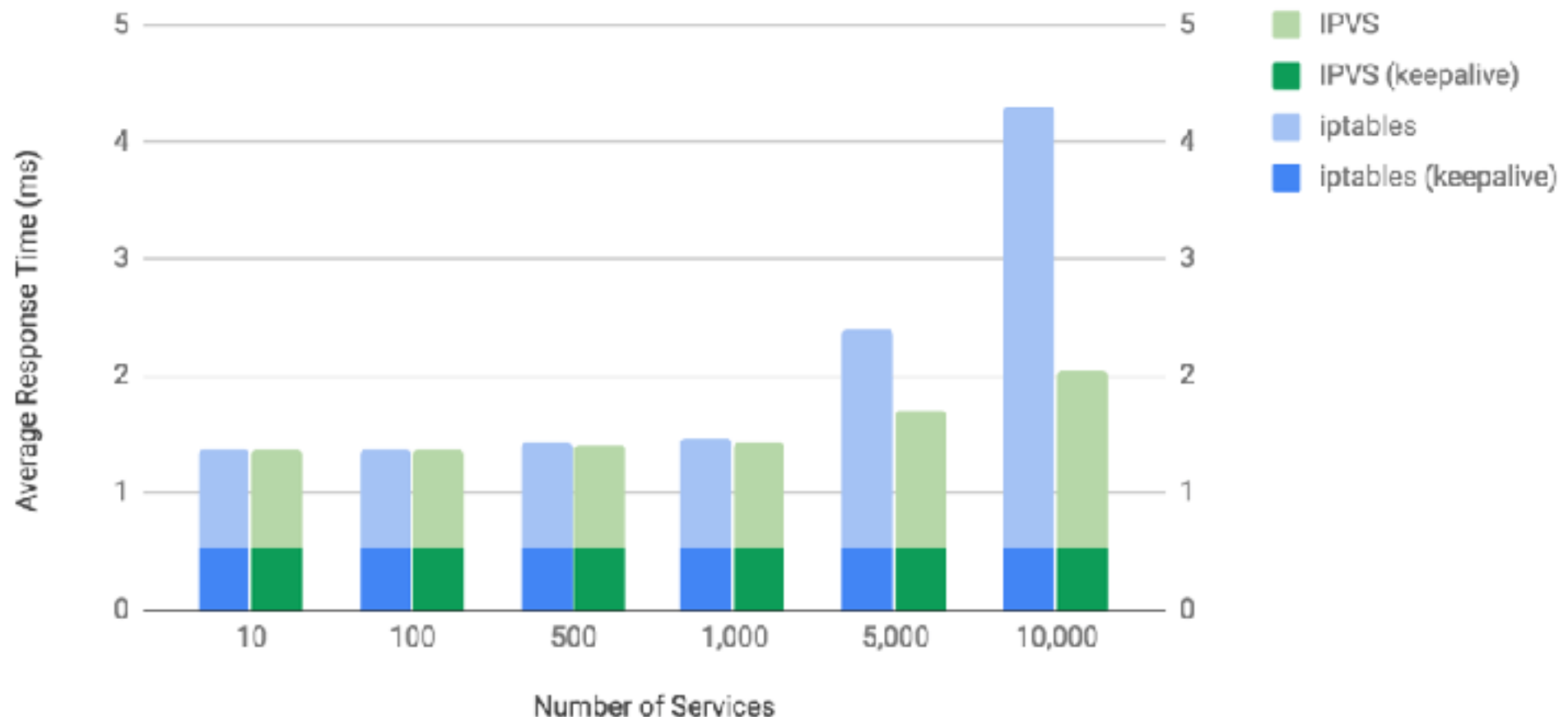


Use IPVS in service

IPVS (IP Virtual Server)

Linux kernel load balancer over layer 4

Round-Trip Response Time vs Number of Services



<https://www.tigera.io/blog/comparing-kube-proxy-modes-iptables-or-ipvs/>



Don't keep sensitive data in K8s secret

K8s secret == base64 encoding

AWS
secret manager

Hashicorp vault

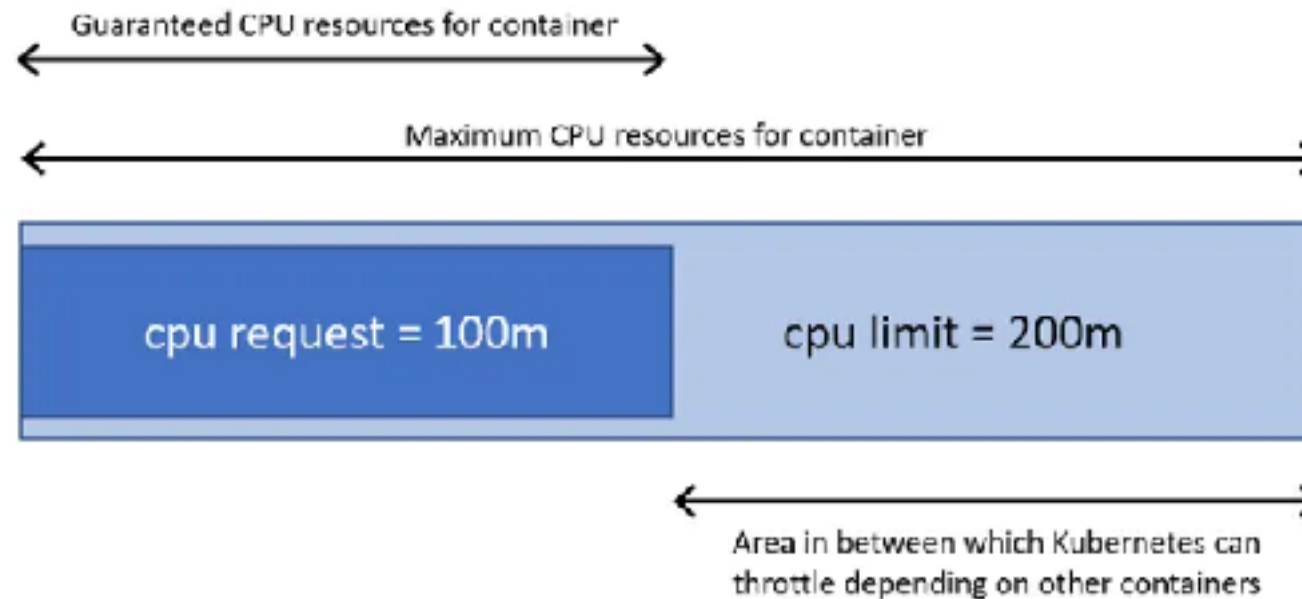
Azure key vault

Google
secret manager



Always set resources limit

```
resources:  
  limits:  
    cpu: 1  
    memory: 1Gi  
  requests:  
    cpu: 500m  
    memory: 512Mi
```



Use liveness and readiness probes

Liveness

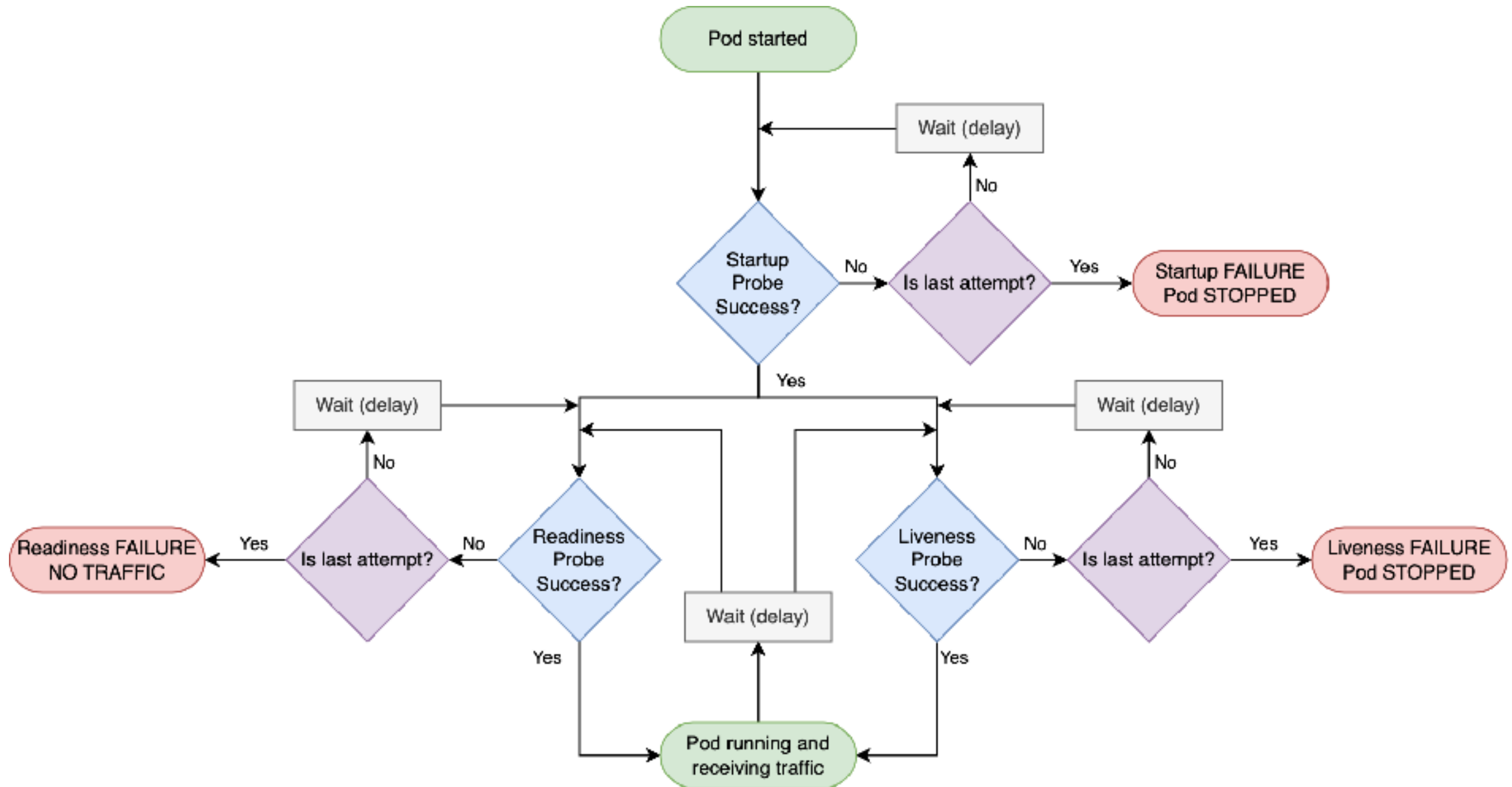
Readiness

Startup

<https://cloud.google.com/blog/products/containers-kubernetes/kubernetes-best-practices-setting-up-health-checks-with-readiness-and-liveness-probes>



Startup probe



<https://andrewlock.net/deploying-asp-net-core-applications-to-kubernetes-part-6-adding-health-checks-with-liveness-readiness-and-startup-probes/>

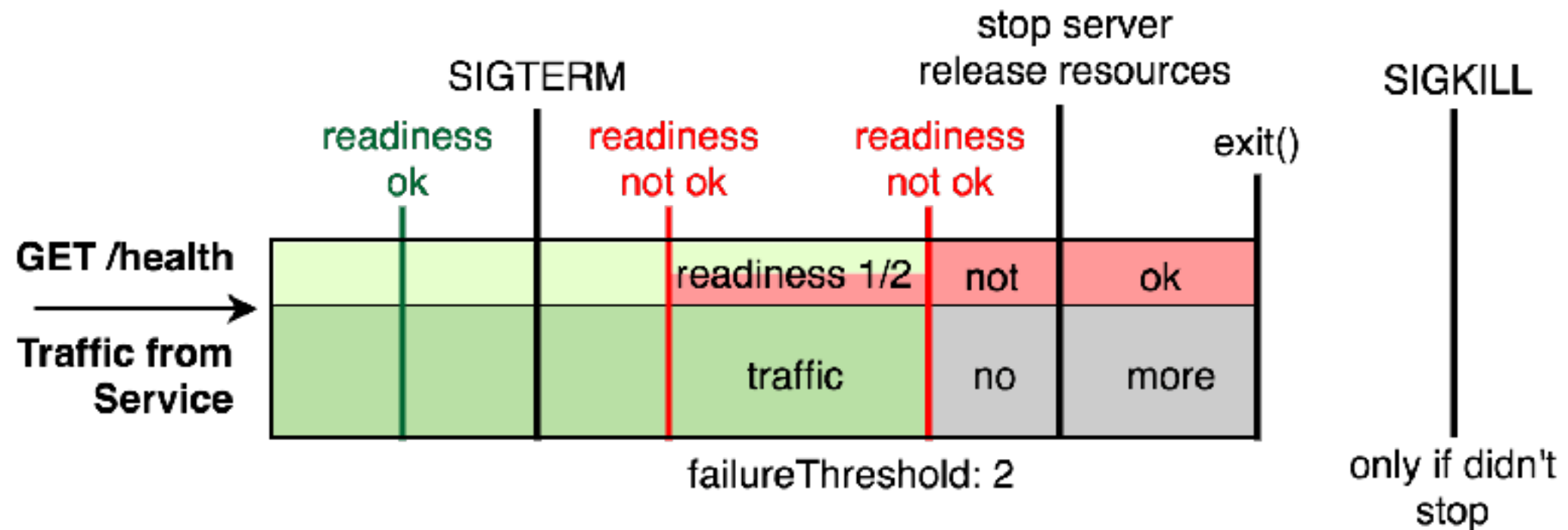


Terminating with grace

<https://learnkube.com/graceful-shutdown>



Graceful Shutdown in Kubernetes

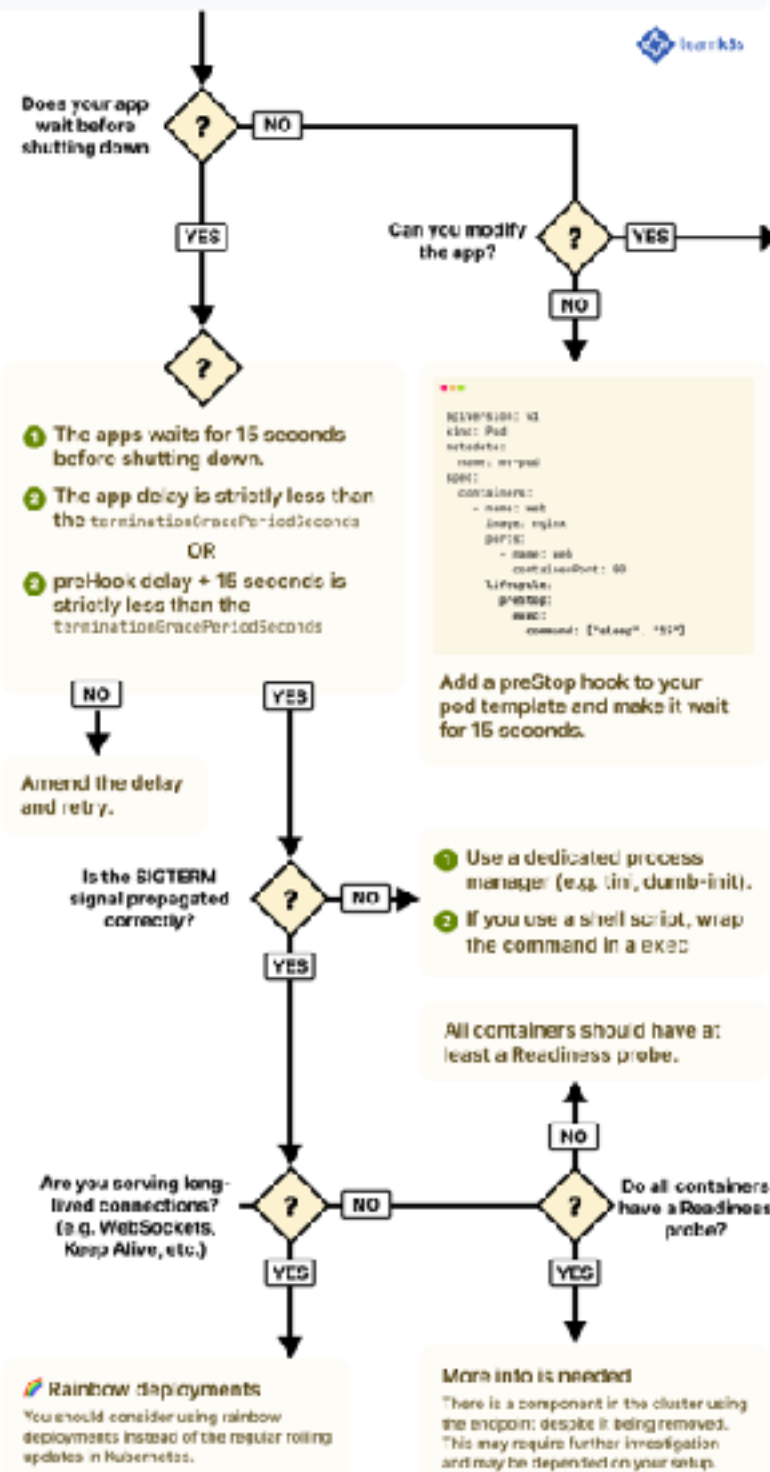


<https://learnk8s.com/graceful-shutdown>



I SEE 5XX ERRORS IN MY KUBERNETES DEPLOYMENTS

This flowchart helps you debug issues with pod graceful shutdown in Kubernetes.



<https://leanankhs.io/graceful-shutdown>

You should wait for 15 seconds and, in the next 15 seconds, close any connections and shut down the application.

Go

```

package main

import (
    "log"
    "os"
    "os/signal"
    "syscall"
)

func main() {
    sig := make(chan os.Signal, 1)
    done := make(chan bool, 1)
    // registers the channel
    signal.Notify(sig, syscall.SIGTERM)

    go func() {
        sig := os.Signal
        for {
            if caught := <math>\text{SIGTERM}</math>, shutting down")
            // Finish any outstanding requests, then...
            done = true
        }
    }()

    log.Println("Starting application")
    // Main logic goes here
    log.Println("Exiting")
}
    
```

Python

```

import signal, time, os

def shutdown(signum, frame):
    print("Caught SIGTERM, shutting down")
    # Finish any outstanding requests, then...
    exit(0)

if __name__ == '__main__':
    # Register handler
    signal.signal(signal.SIGTERM, shutdown)
    # Main logic goes here
    
```

Node.js

```

process.on('SIGTERM', () => {
    console.log('The service is about to shut down!');

    // Finish any outstanding requests, then...
    process.exit(0);
});
    
```

Java

```

public class App {
    public static void main(String[] args) {
        var shutdownListener = new Thread() {
            public void run() {
                // Main logic goes here
            }
        };
        Runtime.getRuntime().addShutdownHook(shutdownListener);
    }
}
    
```

.NET Core

```

using System;
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Hosting;
using Microsoft.Extensions.Logging;

public class Program
{
    public static void Main(string[] args)
    {
        CreateWebHostBuilder(args).Build().Run();
    }

    public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
    {
        return WebHost.CreateDefaultBuilder(args)
            .UseStartup<Startup>()
            .Build();
    }
}

public class Startup
{
    public Startup(IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            services.AddMvc().SetCompatibilityVersion(CompatibilityVersion.Default);
        });
    }
}
    
```



Production Checklist ?

Kubernetes production best practices

A curated checklist of best practices
designed to help you release to
production

<https://learnkubernetes.com/production-best-practices>



Q/A

